

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

Работа допущена к защите
Руководитель ОП
_____ А.В. Щукин
«_____» _____ 202_ г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
РАБОТА БАКАЛАВРА
РАЗРАБОТКА СРЕДСТВА АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ В
ПРОЦЕССЕ НЕПРЕРЫВНОЙ ИНТЕГРАЦИИ С ПОМОЩЬЮ БОЛЬШИХ
ЯЗЫКОВЫХ МОДЕЛЕЙ

по направлению подготовки 09.03.03 Прикладная информатика
Направленность (профиль) 09.03.03_03 Интеллектуальные инфокоммуникацион-
ные технологии

Выполнил
студент гр. з5130903/00301

Ю.Р. Ляпунова

Руководитель
ст. преподаватель ВШ ПИ

В.А. Пархоменко

Консультант
по нормоконтролю

В.А. Пархоменко

**САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ПЕТРА ВЕЛИКОГО**

Институт компьютерных наук и кибербезопасности

УТВЕРЖДАЮ

Руководитель ОП

_____ А.В. Щукин

« _____ » _____ 202_ г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

студенту Ляпуновой Юлии Руслановне гр. з5130903/00301

1. Тема работы: Разработка средства автоматизации тестирования в процессе непрерывной интеграции с помощью больших языковых моделей.
2. Срок сдачи студентом законченной работы: 11.01.2025.
3. Исходные данные по работе: исследование технологий [3.2], а также исследование о их внедрении [3.1].
 - 3.1. A Roadmap for Using Continuous Integration Environments / L. Yu [и др.] // Communications of the ACM. — 2024. — Т. 67, № 6. — С. 82—90. — DOI 10.1145/3631519. — URL: <https://dl.acm.org/doi/10.1145/3631519>.
 - 3.2. Procko T. T., Ochoa O. Graph Retrieval-Augmented Generation for Large Language Models: A Survey // 2024 Conference on AI, Science, Engineering, and Technology (AIxSET). — 2024. — С. 166—169. — DOI 10.1109/AIxSET62544.2024.00030.
4. Содержание работы (перечень подлежащих разработке вопросов):
 - 4.1. Исследование средств автоматизации тестирования в процессе непрерывной интеграции с помощью больших языковых моделей.
 - 4.2. Проектирование средства автоматизации тестирования в процессе непрерывной интеграции с помощью больших языковых моделей.
 - 4.3. Реализация средства автоматизации тестирования в процессе непрерывной интеграции с помощью больших языковых моделей.

- 4.4. Тестирование и апробация средства автоматизации тестирования в процессе непрерывной интеграции с помощью больших языковых моделей.
5. Перечень графического материала (с указанием обязательных чертежей):
- 5.1. Схема архитектуры разработанного средства автоматизации тестирования.
6. Консультанты по работе:
- 6.1. Ст. преподаватель ВШ ПИ, В.А. Пархоменко (нормоконтроль).
7. Дата выдачи задания: 02.10.2024.

Руководитель ВКР _____ В.А. Пархоменко

Задание принял к исполнению 02.10.2024

Студент _____ Ю.Р. Ляпунова

РЕФЕРАТ

На 118 с., 28 рисунков, 8 таблиц, 2 приложения

КЛЮЧЕВЫЕ СЛОВА: АВТОМАТИЗИРОВАННОЕ ТЕСТИРОВАНИЕ, LLM, RAG, CI.

Тема выпускной квалификационной работы: «Разработка средства автоматизации тестирования в процессе непрерывной интеграции с помощью больших языковых моделей».

Предметом исследования является технологии применения LLM для автоматизации тестирования программного обеспечения в процессе непрерывной интеграции. Цель данной работы заключается в разработке системы автоматизации тестирования программного обеспечения, использующей подходящие LLM, в процессе разработки этого обеспечения и непрерывной интеграции обновлений в него.

В рамках исследования был проведен сравнительный анализ существующих инструментов автоматизации тестирования. Изучены подходы к интеграции LLM в процессы генерации кода для автоматизированного тестирования. На основе полученных данных, с использованием метода синтеза, предложено решение, основанное на концепции RAG, которое направлено на устранение недостатков моделей, связанных с ограниченностью и статичностью контекста. Для проектирования и реализации данной системы был применен объектно-ориентированный подход.

Результатом работы является система представляет собой программное решение, использующее LLM и концепцию RAG, способное генерировать тесты с учетом изменений в коде в процессе CI.

Разработанная система ориентирована на применение в организациях, занимающихся разработкой программного обеспечения, где требуется автоматизация тестирования. Она позволяет снизить временные затраты на написание тестов и адаптироваться к изменениям в коде.

ABSTRACT

118 pages, 28 figures, 8 tables, 2 appendices

KEYWORDS: AUTOMATED TESTING, LLM, RAG, CI.

The subject of the graduate qualification work is «Development of a test automation tool for continuous integration testing using large language models».

The subject of the study is the technology of applying LLMs to automate software testing in the process of continuous integration. The purpose of this paper is to develop a software test automation system that uses suitable LLMs in the process of development of this software and continuous integration of updates to it.

The study included a comparative analysis of existing test automation tools. Approaches to integrating LLMs into code generation processes for automated testing were studied. Based on the obtained data, using the synthesis method, a solution based on the RAG concept was proposed, which aims to eliminate the disadvantages of the models related to context constraints and staticity. An object-oriented approach was used to design and implement this system.

The resulting system is a software solution utilizing LLM and the RAG concept, capable of generating tests taking into account the changes in the code during the CI process.

The developed system is oriented for application in software development organizations where test automation is required. It allows reducing the time spent on writing tests and adapt to changes in the code.

СОДЕРЖАНИЕ

Введение	9
Глава 1. Исследование средств автоматизации тестирования с помощью больших языковых моделей.....	11
1.1. Основы больших языковых моделей	11
1.1.1. Принципы работы больших языковых моделей	11
1.1.2. Возможности LLMs в анализе кода и автоматизации тестирования..	13
1.1.3. Сравнительный анализ традиционного автоматизированного тести- рования и подхода с использованием LLMs.....	16
1.1.4. Ограничения и вызовы в использовании LLMs.....	17
1.2. Связь LLMs с Retrieval-Augmented Generation	24
1.2.1. Введение в Retrieval-Augmented Generation	24
1.2.2. Принцип работы RAG	25
1.3. Непрерывная интеграция.....	28
1.3.1. Понятие непрерывной интеграции	28
1.4. Анализ существующих инструментов автоматизации тестирования в контексте рассмотренных технологий.....	30
1.4.1. Инструмент Postman + PostBot	30
1.4.2. Инструмент GitHub Copilot	32
1.4.3. Инструмент IntelliJ IDEA + AI Assistant.....	35
1.4.4. Инструмент Zencoder	37
1.4.5. Инструмент Cursor.....	39
1.4.6. Сравнение инструментов	40
1.5. Выводы по главе 1	44
Глава 2. Проектирование средства для автоматизированного тестирования API.....	45
2.1. Анализ основных требований.....	45
2.2. Использование системы.....	47
2.2.1. Основные участники	47
2.2.2. Основные случаи использования.....	47
2.3. Диаграмма состояний	48
2.3.1. Основные состояния системы.....	48
2.4. Функциональное моделирование системы	50
2.4.1. IDEF0-диаграмма уровня A-0.....	50
2.4.2. IDEF0-диаграмма уровня A0	51
2.4.3. IDEF0-диаграмма для функционального блока A6	55

2.4.4. Регламенты использования LLM	57
2.5. Проектирование архитектуры системы.....	59
2.5.1. Проектирование процесса генерации тестов.....	60
2.5.2. Проектирование процесса актуализации базы знаний.....	61
2.5.3. Особенности проектирования системы.....	63
2.6. Выводы по главе 2	66
Глава 3. Реализация системы автоматизированного тестирования API.....	67
3.1. Процесс оркестрации разрабатываемой системы	67
3.1.1. Преднастройка процесса оркестрации	68
3.1.2. Разработка пайплайна.....	69
3.2. Диаграмма классов	70
3.2.1. Класс WebhookController.....	71
3.2.2. Класс RabbitMQConfig.....	72
3.2.3. Класс QueueListener.....	73
3.2.4. Класс TestGenerationWorker.....	74
3.2.5. Класс KnowledgeWorker.....	74
3.2.6. Класс RAGService	75
3.2.7. Класс GitHubService	75
3.2.8. Класс ModelCollector	76
3.2.9. Класс EndpointMethodLocator	77
3.2.10. Класс DiffParser.....	77
3.2.11. Класс PatchParser	78
3.2.12. Класс EmbeddingService	78
3.2.13. Класс VectorStorageService	78
3.2.14. Класс ChatService.....	79
3.3. Функционал извлечения данных из PR	79
3.4. Парсинг извлеченных данных.....	81
3.5. Формирование расширенного промта	83
3.6. Выводы по главе 3	84
Глава 4. Оценка качества разработанной системы автоматизированного тестирования API.....	85
4.1. Выбор подхода к тестированию.....	85
4.1.1. Виды тестирования	85
4.1.2. Проекты для выполнения тестирования.....	87
4.2. Модульное тестирование	88
4.3. Функциональное тестирование.....	90

4.3.1. Оптимизация проекта Spring PetClinic.....	90
4.3.2. Настройки пайплайна	91
4.3.3. Результаты генерации тестов и покрытие исходного кода.....	93
4.3.4. Проверка устойчивости тестов к мутациям исходного кода	94
4.3.5. Анализ первого выжившего мутанта.....	94
4.3.6. Анализ второго выжившего мутанта	96
4.4. Тестирование особенностей RAG	96
4.4.1. Пустая векторная БД + LLM обученная на программном коде	97
4.4.2. Обогащенная векторная БД + LLM необученная на программном коде	97
4.4.3. Обогащенная векторная БД + LLM обученная на программном коде	98
4.5. Нагрузочное тестирование	103
4.6. Выводы по главе 4	106
Заключение	109
Список сокращений и условных обозначений.....	111
Словарь терминов.....	112
Список использованных источников.....	113
Приложение 1. Руководство по настройке и запуску разработанной системы	119
Приложение 2. Код разработанной системы.....	126

ВВЕДЕНИЕ

В условиях стремительного развития информационных технологий и увеличивающейся сложности программного обеспечения процессы разработки и обеспечения его качества становятся все более требовательными к автоматизации и оптимизации. Одним из ключевых способов обеспечения качества является тестирование программных продуктов, которое позволяет выявлять и устранять дефекты на ранних стадиях жизненного цикла разработки. Однако традиционные методы тестирования часто сталкиваются с проблемами масштабируемости, временных затрат и необходимости постоянного обновления тестовых сценариев в условиях непрерывной интеграции.

Современные подходы к автоматизации тестирования стремятся использовать возможности машинного обучения и искусственного интеллекта для повышения эффективности и точности процесса тестирования. В частности, большие языковые модели демонстрируют потенциал в генерации и анализе тестовых сценариев на основе естественного и программного языков, что открывает возможности для автоматизации задач в области обеспечения качества программного обеспечения [26].

Большинство существующих решений в области автоматизации тестирования являются платными и зависят от зарубежных облачных сервисов, что создает риски для конфиденциальности данных и устойчивости процессов тестирования. В этом контексте использование собственной системы автоматизации тестирования на базе больших языковых моделей, работающей на внутренних серверах компании, обеспечивает независимость от внешних провайдеров, соблюдение требований конфиденциальности и снижение затрат.

Разработка данной системы имеет практическую значимость, поскольку позволяет сократить временные и ресурсные затраты на процесс тестирования и адаптировать процессы обеспечения качества к современным требованиям гибкой разработки.

Актуальность исследования обусловлена необходимостью обеспечения качества и повышения скорости тестирования программного обеспечения в условиях растущей сложности IT-проектов. В условиях санкций и ограничений доступа к зарубежным сервисам значение приобретает разработка решений, которые обеспечивают импортозамещение и соответствуют требованиям конфиденциальности.

Объект исследования — процесс автоматизации тестирования программного обеспечения.

Предмет исследования — технологии применения больших языковых моделей для автоматизации тестирования программного обеспечения в процессе непрерывной интеграции.

Цель работы — разработка системы автоматизации тестирования программного обеспечения, использующей подходящие большие языковые модели, в процессе разработки этого обеспечения и непрерывной интеграции обновлений в него.

В соответствии с целью исследования необходимо решить следующие задачи:

1. Исследовать принципы работы больших языковых моделей и их применимость в автоматизации тестирования.
2. Провести сравнительный анализ существующих аналогов автоматизации тестирования на основе больших языковых моделей.
3. Разработать архитектуру системы автоматизации тестирования по средствам использования большой языковой модели в процессе непрерывной интеграции.
4. Реализовать прототип системы.
5. Протестировать и оценить систему, включая снижение временных и ресурсных затрат.

Теоретической основой выпускной квалификационной работы послужили исследования технологий [6; 13; 17; 4], а также исследование о их внедрении [5].

Методологической базой исследования стали общенаучные методы, в том числе эмпирические методы, направленные на анализ аналогов инструментов автоматизации тестирования. С использованием моделирования создана теоретическая база для проектирования архитектуры системы автоматизации тестирования. Структурные особенности системы изучались с помощью методов анализа и синтеза, что позволило выделить ключевые функции системы. Оценка разработанной системы проводилась с применением метода измерения, включающего анализ качества генерируемых тестов и оценку нагрузки на систему.

ГЛАВА 1. ИССЛЕДОВАНИЕ СРЕДСТВ АВТОМАТИЗАЦИИ ТЕСТИРОВАНИЯ С ПОМОЩЬЮ БОЛЬШИХ ЯЗЫКОВЫХ МОДЕЛЕЙ

В данной главе рассматриваются возможные технологии и концепции для автоматизации процессов тестирования ПО. Основная цель главы — проанализировать технологии, подходы и инструменты которые могут быть применены для автоматизированного тестирования.

В параграфе 1.1 представлены основные принципы работы больших языковых моделей, их возможности в анализе кода и автоматизации тестирования, а также ограничения и вызовы. В параграфе 1.2 рассматривается концепция Retrieval-Augmented Generation, ее принципы и преимущества. Параграф 1.3 посвящен процессу непрерывной интеграции. В параграфе 1.4 рассмотрены существующие инструменты, которые используют большие языковые модели для автоматизации тестирования. В заключение, в параграфе 1.4.6 проводится их сравнительный анализ.

1.1. Основы больших языковых моделей

Большие языковые модели стали важным инструментом в современной разработке программного обеспечения (далее — ПО) благодаря их способности анализировать, обрабатывать и генерировать текстовую информацию на основе большого контекста. Эти модели позволяют не только решать задачи обработки естественного языка, но и эффективно работать с программным кодом, включая анализ, рефакторинг и генерацию тестов.

В контексте автоматизации тестирования в процессах непрерывной интеграции большие языковые модели открывают новые возможности для повышения эффективности. Они способны обрабатывать сложные данные, такие как документация, комментарии к коду и истории изменений, чтобы создавать тестовые сценарии.

1.1.1. Принципы работы больших языковых моделей

Большие языковые модели (Large Language Models, далее — LLMs) — это модели машинного обучения, предназначенные для обработки и генерации текстов на естественном языке. LLMs базируются на архитектуре трансформеров, предложенной Васвани и соавторами в 2017 году [26]. Эта архитектура стала

революционной, так как она заменила традиционные рекуррентные нейронные сети (Recurrent Neural Network, далее — RNN) и конволюционные нейронные сети (Convolutional Neural Network, далее — CNN), которые использовались ранее для обработки текста. Трансформеры применяют механизм внимания, который позволяет моделям учитывать полный контекст текста, что делает их эффективными для задач обработки естественного языка и программного кода.

Основные принципы работы LLMs [26, с. 3— 7]:

1. Механизм внимания (далее, — self-attention). Основой работы трансформеров является механизм внимания. Данный механизм позволяет модели учитывать взаимосвязи между всеми токенами (словами или символами) в тексте, независимо от их положения. Каждый токен анализируется в контексте всей последовательности, сравнивая с остальными токенами. В процессе расчета self-attention модель определяет «вес» (важность) каждого токена относительно текущего, что позволяет выявлять ключевые зависимости, даже если слова находятся далеко друг от друга. Эти веса используются для формирования более точного представления о значении токенов, обеспечивая модели способность учитывать длинные контексты и сложные языковые конструкции. Благодаря этому self-attention стал важной составляющей архитектуры трансформеров, позволяя моделям эффективно работать с текстами любой длины.
2. Параллельная обработка. В традиционных RNN текст обрабатывается последовательно, что делает обучение медленным и ограничивает их применение на больших наборах данных. Трансформеры способны обрабатывать весь текст одновременно. Это позволяет значительно ускорить обучение, а также эффективно работать с длинными последовательностями текста, что делает трансформеры более подходящими для современных задач обработки естественного языка и программного кода.
3. Многоуровневая структура. Многоуровневая структура трансформеров обеспечивает их способность анализировать текст на разных уровнях абстракции. Каждый слой трансформера обрабатывает данные с определенной целью: нижние слои фокусируются на выявлении базовых грамматических структур, таких как связи между словами и их грамматические роли, в то время как верхние слои анализируют сложные зависимости и общий контекст текста. Эта иерархическая обработка позволяет модели формировать глубокое понимание текста. Помимо этого, каждый

слой включает механизмы нормализации данных, которые обеспечивают стабильность обучения, и нелинейные функции активации, позволяющие моделям улавливать сложные нелинейные зависимости в данных. Такая архитектура делает трансформеры универсальными инструментами для задач анализа и генерации текста.

4. Обучение LLMs осуществляется в два этапа: предварительное обучение и дообучение. На этапе предварительного обучения цель состоит в том, чтобы научить модель понимать общий язык и текст. Для этого модель обрабатывает огромные объемы данных, включая тексты книг, статей и форумов. В процессе обучения она учится предсказывать следующее слово в тексте или восстанавливать пропущенные слова в предложении. Например, модель GPT-3 обучена на тексте объемом 570 ГБ, что позволило ей овладеть широким спектром языковых задач [54]. На этапе дообучения модель адаптируется для выполнения конкретных задач, таких как генерация кода, написание автотестов или анализ документации. Этот этап позволяет моделям быть универсальными: от работы с естественным языком до выполнения специфических задач, таких как обработка кода или создание тестов.

1.1.2. Возможности LLMs в анализе кода и автоматизации тестирования

Каждая большая языковая модель разрабатывается для выполнения определенного круга задач, которые зависят от данных, использованных при ее обучении. Например, модели, обученные на текстовых данных, демонстрируют превосходство в обработке естественного языка, тогда как модели, натренированные на коде, оптимизированы для задач, связанных с программированием. В результате, LLMs обученные на программном коде обеспечивают генерацию кода и создание тестов, анализ и оптимизацию существующего кода, а также помогают в исправлении багов.

Использование LLMs в работе с программным кодом обосновано способностью обрабатывать неструктурированные данные, такие как документация, комментарии к коду и истории изменений. Например, Codex и аналогичные модели анализируют техническую документацию, извлекая из нее функциональные требования. Этот процесс позволяет моделям понимать ключевые аспекты работы ПО [26].

Помимо обработки неструктурированных данных, LLMs активно применяются для статического анализа кода, который включает:

- выявление ошибок и уязвимостей. Модели, обученные на огромных наборах данных, содержащих миллионы строк кода, находят нарушения стандартов кодирования, синтаксические ошибки и потенциальные баги, например, утечки памяти или ошибки при работе с ресурсами. Согласно исследованиям Salesforce, модель CodeT5 продемонстрировала на 9% более высокую точность в выявлении ошибок по сравнению с предыдущими подходами [18]. Помимо этого, модели доказали свою эффективность для анализа потенциальных уязвимостей, таких как SQL-инъекции, XSS-атаки и ошибки управления памятью. Это подтверждается исследованием, где показано, как интеграция LLMs в веб-приложения помогает снизить риск XSS-атак и SQL-инъекций [11];
- оптимизация алгоритмов и рефакторинг кода. Модели предлагают улучшения в коде, основываясь на лучших практиках и соответствующие современным стандартам разработки, таких как использование встроенных функций языка вместо ручных реализаций, повышая стабильность ПО. Это особенно полезно для устаревших проектов с неструктурированным кодом. Как описано в статье о рефакторинге кода с использованием искусственного интеллекта (далее — ИИ), модели могут автоматически устранять избыточность, улучшать читаемость и помогать избегать анти-паттернов, таких как дублирование кода или чрезмерная вложенность [8].

Необходимо выделить способность LLMs автоматически генерировать документацию на основе анализа кода. Это упрощает понимание проекта для новых членов команды и облегчает сопровождение ПО. Исследование [10] показывает, что автоматическая генерация документации повышает производительность команд на 15–20%, устраняя необходимость в ручном описании каждой функции или модуля.

Еще одной ключевой областью применения LLMs является генерация автоматических тестов. Большие языковые модели для генерации программного кода могут разрабатывать unit-тесты для функций, интеграционные тесты для выявления проблем интеграции между различными модулями тестируемого ПО, а также тесты для edge-case сценариев (редкие и экстремальные случаи, которые сложно предугадать вручную). Исследования OpenAI показали, что Codex, специализи-

рованная версия GPT, обученная на публичных репозиториях GitHub, достигает точности до 71% в задачах генерации тестов для простых программных функций, таких как манипуляции с массивами или работа с данными [54]. Помимо этого, инструмент TestGen-LLM, разработанный в Meta¹, демонстрирует эффективность LLMs в автоматическом улучшении существующих unit-тестов. Этот инструмент протестирован на платформах Instagram² и Facebook³, где обеспечил значительное повышение покрытия кода и качества тестов [7]. Также в исследовании [21] представлено использование LLM-агентов для автоматической генерации тестов. Авторы демонстрируют, что LLM-агенты превосходят существующие системы тестирования, особенно в задачах генерации тестов для сложных программных сценариев.

Сравнение основных моделей для генерации программного кода представлено в сравнительной таблице представленной в табл.П1.1, где в качестве критериев выбраны следующие:

- название модели;
- кол-во параметров — влияет на производительность и вычислительные требования для использования модели;
- поддерживаемые языки программирования (далее — ЯП) — влияет на сколько модель применима к конкретным задачам;
- особенности — ключевые функции;
- лицензия — определяет, доступна ли модель для некоммерческого или коммерческого использования, а также стоимость;
- MultiPL-E (pass@1;pass@10;pass@100) [52] — является расширенным бенчмарком, позволяющим оценить производительность модели в задачах генерации программного кода на нескольких языках программирования. Метрика рассчитывается для трех случаев: вероятность успешного решения задачи с первой попытки, вероятность успеха в пределах 10 попыток и вероятность успеха в пределах 100 попыток.

Выбранные параметры отражают ключевые характеристики моделей: вычислительные возможности, универсальность, функциональность, доступность и точность работы на реальных задачах, что позволяет провести систематическое и обоснованное сравнение моделей.

¹Признана экстремистской организацией в РФ.

²Продукт компании Meta, которая признана экстремистской организацией в РФ.

³Продукт компании Meta, которая признана экстремистской организацией в РФ.

1.1.3. Сравнительный анализ традиционного автоматизированного тестирования и подхода с использованием LLMs

Проведение сравнительного анализа необходимо для объективной оценки преимуществ и ограничений использования больших языковых моделей в тестировании программного обеспечения по сравнению с традиционным ручным подходом. Такой анализ способствует обоснованию целесообразности применения LLMs в реальной практике разработки и демонстрирует их влияние на эффективность и качество тестирования. Результаты анализа представлены в табл.4.3.

Выбор критериев для сравнения традиционного автоматизированного тестирования и подхода с использованием LLMs основывается на необходимости отражения ключевых аспектов, важных в практике разработки ПО. Эти критерии охватывают скорость выполнения, затраты на ресурсы, покрытие кода и адаптацию к изменениям — факторы, которые влияют на эффективность команд и качество продуктов. Критерии также включают вероятность человеческой ошибки и точность генерируемых тестов, так как они важны для надежности ПО. Дополнительно включены аспекты, связанные с технической сложностью и поддержкой. Эти критерии отражают вызовы, с которыми сталкиваются команды при масштабировании проектов и работе с современными системами. Таким образом, выбор критериев направлен на создание всестороннего анализа, основанного на практическом опыте и научной базе.

Перспективы применения подхода с использованием LLMs подтверждаются рядом исследований, подчеркивающих их эффективность и преимущества в автоматизации тестирования.

В частности, исследование произведенное компанией McKinsey демонстрирует, что интеграция ИИ в процессы разработки может привести к значительному увеличению продуктивности. В отчете указано на сокращение времени разработки до 50% при использовании ИИ-инструментов. Это позволяет командам быстрее выпускать новые версии продукта [63].

Дополнительно, в одной из работ [23], касающейся использования ИИ для автоматизации тестирования, отмечается, что полнота покрытия кода увеличивается на 15%, а общая стоимость тестирования сокращается на 25% по сравнению с традиционными подходами. Это связано с возможностью модели генерировать тесты для различных входных данных, включая граничные и редкие случаи.

Таблица 1.1

Сравнение традиционного автоматизированного тестирования и тестирования при использовании LLMs после анализа исследований [26; 10; 21; 7]

Критерий	Традиционное автоматизированное тестирование	Тестирование при LLMs
Скорость выполнения	медленный процесс, требует значительных временных затрат	высокая, автоматическая генерация тестов
Затраты на ресурсы	высокие, включают оплату труда и временные затраты	низкие, благодаря автоматизации
Покрытие кода	ограниченное, зависит от навыков и времени разработчика	высокое, покрывает больше сценариев, включая редкие случаи
Адаптация к изменениям	требует ручного обновления	автоматическое обновление тестов при изменении кода
Вероятность человеческой ошибки	высокая вероятность пропуска ошибок или упущений	практически отсутствует, модель минимизирует субъективные ошибки
Качество генерируемых тестов	зависит от квалификации QA-инженера; возможны недостатки	высокое, благодаря обучению на широких наборах данных
Обучение и сопровождение новых специалистов	требует дополнительных ресурсов для обучения новых сотрудников	помогают быстрее адаптировать новых сотрудников через автоматизацию
Учет редких и граничных случаев	редко учитываются, особенно при ограниченном времени	генерируют тесты для граничных и редких случаев с высокой точностью
Поддержка сложных архитектур	трудно покрывать сложные архитектуры или системы с высокой связностью	поддерживают сложные архитектуры, обеспечивая гибкость и адаптивность

Описанные выше преимущества моделей делают их частью современного инструментария для автоматизации процессов разработки. С развитием технологий роль LLMs в разработке будет только усиливаться, обеспечивая более высокий уровень надежности и безопасности ПО.

1.1.4. Ограничения и вызовы в использовании LLMs

Большие языковые модели представляют собой активно развивающуюся технологию в области обработки данных. Тем не менее, их использование сопровождается рядом вызовов и ограничений. Эти аспекты необходимо учитывать, чтобы достичь эффективного и ответственного применения технологий.

1.1.4.1. Отсутствие истинного понимания

Одним из ключевых ограничений LLMs является отсутствие у них истинного понимания программной логики и намерений разработчика. Эти модели не обладают сознанием и осмысленным восприятием алгоритмов или структур данных, а

функционируют на основе вероятностного сопоставления паттернов, извлеченных из огромных обучающих наборов данных. Это ограничивает их способность к точному интерпретированию сложных архитектур и контекста, так как основывается исключительно на статистической корреляции, а не на глубоком понимании контекста или причинно-следственных связей.

Например, при анализе кода с неявными зависимостями между модулями, модель может не распознать потенциальную проблему, связанную с неправильным использованием ресурсов или нарушением потоков данных. Исследования показывают, что LLMs хорошо справляются с типичными задачами, описанными выше, но часто терпят неудачу в ситуациях, требующих понимания долгосрочных эффектов изменений, например, влияния рефакторинга на производительность системы [23; 63].

Кроме того, LLMs склонны к «галлюцинациям» в контексте программирования, когда они предлагают решения или исправления, которые на первый взгляд кажутся корректными, но при выполнении оказываются нефункциональными. Например, модель может сгенерировать код, который соответствует стандартам синтаксиса, но не учитывает контекстные ограничения, такие как допустимые значения переменных или доступ к необходимым API. Это подчеркивается в работе [10], где отмечается, что модели демонстрируют высокую производительность в типичных задачах, но не способны справляться с ситуациями, где требуется глубокий анализ взаимосвязей между компонентами системы.

Еще одной сложностью является недостаточная адаптация моделей к специфическим доменам программирования. Например, если LLM анализирует код на языке, не распространенном в обучающих данных, или сталкивается с уникальными архитектурными паттернами, она может выдавать неточные рекомендации или вовсе не распознавать проблему [2].

Таким образом, отсутствие истинного понимания у LLMs в программировании ограничивает их применение в задачах, требующих причинно-следственного анализа, интерпретации сложных систем или анализа долгосрочных последствий изменений. Для повышения надежности и полезности моделей важно интегрировать механизмы дополнительной проверки и адаптации к специфическим доменам кода [2].

1.1.4.2. Этические и социальные риски

Этические и социальные риски становятся все более значимыми по мере интеграции LLMs в профессиональные и социальные сферы. В контексте программирования эти риски особенно заметны, поскольку LLMs не только автоматизируют задачи, но и становятся инструментами, влияющими на качество, безопасность и справедливость ПО.

Одной из ключевых проблем является возможность распространения предвзятости. LLMs обучаются на больших наборах данных, которые могут содержать скрытые предвзятости, отражающие социальные, культурные или технические предпочтения разработчиков и источников данных. В программировании это может проявляться, например, в генерации кода, который не соответствует требованиям инклюзивности или нарушает лучшие практики безопасности. Модели могут усиливать существующие предубеждения, так как они воспроизводят паттерны из обучающих данных [15]. Это особенно опасно в сценариях, где результат работы модели принимается без тщательной проверки человеком.

Кроме того, существует угроза распространения дезинформации через автоматически сгенерированные тексты или комментарии к коду. LLMs могут создавать содержимое, которое выглядит убедительно, но фактически является некорректным. В программной разработке это может привести к внедрению уязвимостей или неэффективных решений. Например, LLMs могут генерировать код, который на первый взгляд кажется корректным, но содержит критические ошибки, влияющие на работоспособность и безопасность системы [10].

Этические риски также связаны с приватностью и интеллектуальной собственностью. Модели, обученные на публичных репозиториях кода, могут непреднамеренно воспроизводить защищенный авторским правом код или конфиденциальную информацию. Это поднимает вопросы юридической ответственности разработчиков таких систем и пользователей, которые применяют эти модели в своих проектах. Известен случай, когда против организаций Microsoft и OpenAI выдвинут коллективный иск, в котором их обвиняли в неправомерном использовании открытого исходного кода для обучения своих моделей ИИ. Истцы утверждали, что компании нарушили условия лицензий на открытое ПО, интегрировав такой код в свои коммерческие продукты без надлежащего соблюдения лицензионных требований [28].

Еще одним аспектом является влияние на рынок труда. Автоматизация задач, таких как генерация тестов, исправление ошибок и даже написание функционального кода, вызывает опасения среди программистов о возможной потере рабочих мест или обесценивании их навыков. Согласно опросу, проведенному компанией Pluralsight среди 1200 руководителей и ИТ-специалистов, почти 65% опрошенных обеспокоены столь ускоренным внедрением ИИ в сферу. Хотя модели обещают ускорить разработку и улучшить качество, их внедрение требует от разработчиков новых подходов к взаимодействию с инструментами, что может усилить профессиональное неравенство. Также отмечается, что LLMs могут снижать барьеры для входа в профессию и обучение ИИ повышает конкурентоспособность ИТ-специалистов на рынке труда (96% опрошенных используют ИИ для повышения своей квалификации), но одновременно создают угрозу для квалифицированных специалистов из-за снижения спроса на ручные задачи [1].

Также нельзя игнорировать проблему ответственности за ошибки. Если LLM сгенерирует код с критической уязвимостью или нарушением законодательства, возникает вопрос: кто несет ответственность — разработчики модели, пользователь или компания, интегрировавшая ее в процесс? Этот вопрос до сих пор остается открытым и требует дальнейшего законодательного регулирования.

Таким образом, использование LLMs в программировании сопровождается серьезными этическими и социальными вызовами. Для их минимизации необходимо внедрение строгих механизмов контроля данных, на которых обучаются модели, разработка стандартов их применения и повышение осведомленности пользователей об ограничениях и рисках технологий. Только комплексный подход позволит использовать потенциал LLMs без угрозы для общества, приватности и профессиональной среды.

1.1.4.3. Высокие вычислительные затраты

Большие языковые модели требуют огромных вычислительных ресурсов для обучения, развертывания и эксплуатации. Эти затраты являются одной из ключевых проблем в их использовании, ограничивая доступность технологий для многих организаций и вызывая значительное влияние на окружающую среду.

Обучение LLMs включает обработку огромных объемов данных с использованием современных графических процессоров (Graphics Processing Unit, далее — GPU) и специализированных тензорных процессоров (Tensor Processing Unit,

далее — TPU). По данным исследования, выбросы CO_2 для вычислений зависят от региона и источников энергии. В среднем 1 кВт·ч электроэнергии приводит к выбросу 0,4–0,5 кг CO_2 . Таким образом, обучение крупной модели может производить выбросы углерода, сопоставимые с полетом на самолете из Нью-Йорка в Сан-Франциско и обратно [20]. Это вызывает серьезные экологические вопросы, особенно в эпоху борьбы с изменением климата. Более детальный разбор затрат для обучения некоторых моделей LLMs представлен в табл.1.2.

Таблица 1.2

Сравнение затрат для обучения LLMs

	GPT-3 [12]	YaML 100B [64]	Bloom [27]	Falcon 180B [37]	CodeGeeX [29]
Кол-во параметров (млрд)	175	100	176	180	13
Используемые GPU	1024 x NVIDIA V100	800 x NVIDIA A100	384 x NVIDIA A100	4096 x NVIDIA A100	1536 x Ascend 910 AI
Продолжительность обучения (дни)	30	65	117	71	60
Общие вычислительные ресурсы (GPU-часы)	737280	1248000	1079232	7000000	2211840
Энергопотребление (кВт·ч)	221184	374400	1082657	2101757	686131
Минимальный углеродный след (тонн CO_2)	88.47	149.76	433.06	840.7	274.45
Максимальный углеродный след (тонн CO_2)	110.59	187.2	541.33	1050.88	343.07

Используемые формулы [20]:

$$\text{Общее кол-во GPU-часов} = \text{Кол-во используемых GPU} \times \text{Время обучения}; \quad (1.1)$$

$$\begin{aligned} \text{Энергопотребление} = & \text{Потребление одного GPU (Вт)} \times \\ & \times \text{Кол-во GPU} \times \text{Время работы (час)}; \end{aligned} \quad (1.2)$$

$$\text{Углеродный след} = \text{Электропотребление (кВт·ч)} \times \text{Выброс } CO_2 \text{ (кг)}. \quad (1.3)$$

Среднее значения потребления электроэнергии для GPU при полной загрузке:

- NVIDIA V100 потребляет около 300 Вт энергии;
- NVIDIA A100 потребляет около 300 Вт энергии;
- Ascend 910 AI потребляет около 310 Вт энергии.

Основываясь на данных исследования, для вычисления минимального и максимального углеродного следа взяты показатели выброса 0,4 кг CO_2 и 0,5 кг CO_2 , соответственно.

Высокие вычислительные затраты также проявляются в процессе вывода (этап, на котором обученная модель используется для выполнения реальных задач). Даже при развертывании модели для конечных пользователей требуется значительная вычислительная мощность для обработки запросов в реальном времени, особенно если речь идет о миллионах пользователей. Это объясняется тем, что каждая операция предсказания требует доступа к массивной нейронной сети, что создает нагрузку на серверы и увеличивает энергопотребление.

Эти затраты особенно высоки для небольших компаний, не имеющих доступа к специализированным вычислительным центрам. Это создает барьеры для использования LLMs в индустрии и усиливает концентрацию технологий в руках крупных корпораций, таких как OpenAI, Google, Microsoft, Яндекс и Сбер. Такие компании обладают ресурсами для разработки и обслуживания моделей, но их доминирование также порождает вопросы о доступности и монополизации технологий.

Более того, затраты на оборудование и инфраструктуру являются лишь частью проблемы. Поддержание стабильной работы LLMs требует значительных инвестиций в охлаждение дата-центров, обеспечение надежности сетей и кадровые ресурсы для управления системами.

Для уменьшения вычислительных затрат предпринимаются попытки оптимизации моделей и алгоритмов обучения. Например, используются методы передачи знаний, когда большая модель обучает меньшую, сохраняя при этом основную функциональность. Также активно разрабатываются новые архитектуры нейронных сетей, которые снижают количество необходимых параметров без существенного снижения производительности.

Таким образом, высокие вычислительные затраты остаются одним из основных вызовов при использовании LLMs. Они ограничивают доступность технологий, усиливают социально-экономическое неравенство и вносят существенный вклад в углеродный след. Для преодоления этих проблем необходимы совместные усилия разработчиков, исследователей и политиков, направленные на оптимизацию технологий, внедрение более энергоэффективных решений и регулирование использования вычислительных мощностей.

1.1.4.4. Ограниченная способность к обновлению знаний

Большие языковые модели играют важную роль в автоматизации задач в ИТ-сфере. Однако их ограниченная способность к обновлению знаний остается существенным препятствием для их эффективного применения в быстро меняющейся индустрии. Это связано с природой их архитектуры и процесса обучения. Модель обучается на статичном наборе данных, который представляет собой срез знаний до определенного момента времени [22]. После завершения обучения модель сохраняет только те паттерны, которые она извлекла из этих данных, и не способна автоматически обновлять свои знания о новых технологиях, языках программирования или инструментах, появившихся после окончания процесса обучения.

Одним из ключевых ограничений является отсутствие динамического механизма добавления новых данных в уже обученную модель. Например, модель, обученная в 2023 году, не будет знать о новых функциях популярных ЯП, выпущенных после этой даты, или об изменениях в современных фреймворках. Это создает риск устаревших рекомендаций или неточной информации при использовании LLMs в разработке ПО.

Модели, обученные на статичных наборах данных, имеют ограниченную способность к адаптации, поскольку их знания зафиксированы на этапе обучения. Чтобы модель могла работать с актуальной информацией, требуется повторное обучение или дообучение с использованием новых данных. Этот процесс не только требует значительных вычислительных ресурсов, но и связан с риском, когда модель теряет часть ранее приобретенных знаний, пытаясь усвоить новые. В задачах программирования модель может начать предлагать решения, которые соответствуют новым стандартам, но при этом забыть об обратной совместимости, что может быть критичным для разработки. Этот эффект известен как «катастрофическое забывание» [14].

В дополнение, ограниченная способность к обновлению знаний влияет на доверие ИТ-специалистов к LLMs. Разработчики ожидают, что модели предоставят актуальные рекомендации, но сталкиваются с устаревшими или нерелевантными результатами.

Выводы из проведенного анализа ограничений и вызовов использования больших языковых моделей свидетельствуют о наличии значительных барьеров, которые необходимо учитывать при их применении в автоматизации тестирования.

Однако выявленные недостатки, такие как отсутствие истинного понимания, этические и социальные риски, высокие вычислительные затраты и ограниченная способность к обновлению знаний, могут быть частично или полностью решены в рамках разработки системы, описанной в данной выпускной квалификационной работе. Для их минимизации необходимо внедрение строгих механизмов контроля данных, на которых обучаются модели, разработка стандартов их применения, повышение осведомленности пользователей об ограничениях и рисках технологий, а также разработки новых методов, таких как энергоэффективное дообучение, адаптивные архитектуры и интеграция с динамическими источниками данных, которые смогут поддерживать актуальность и практическую применимость моделей в быстро развивающемся технологическом мире.

1.2. Связь LLMs с Retrieval-Augmented Generation

Имея результаты в задачах обработки естественного языка, использование LLMs связано с рядом ограничений. В контексте этих ограничений концепция Retrieval-Augmented Generation становится важным направлением для повышения эффективности и практической применимости языковых моделей.

1.2.1. Введение в Retrieval-Augmented Generation

Retrieval-Augmented Generation (далее — RAG) — это подход, который комбинирует возможности больших языковых моделей с механизмами извлечения информации из внешних источников [19]. Его цель — преодолеть ключевые ограничения LLMs, среди которых:

- отсутствие истинного понимания и проблема галлюцинаций решается благодаря наличию актуальную информацию из внешних источников, повышая точность и достоверность генерируемого контента. В статье [17] отмечается снижение вероятности генерации недостоверной информации за счет обращения к внешним базам знаний (далее — БЗ);
- ограниченная способность к обновлению знаний также преодолевается благодаря использованию внешних источников информации без необходимости повторного обучения. В работе [3] подчеркивается, что RAG обеспечивает непрерывное обновление знаний;

- уменьшение вычислительных затрат достигается путем снижения объема обработки данных. В обычных LLMs требуется обрабатывать весь контекст внутри модели, что требует значительных ресурсов. В RAG извлекается только релевантная информация из внешней БД, сокращая объем данных, которые обрабатывает основная модель [19]. Кроме того, возможно использование модели меньшего размера для подготовки структурированного и детализированного контекста.

В отличие от традиционных LLMs, которые полагаются исключительно на данные, на которых они обучались, RAG использует динамические БЗ, позволяя моделям получать актуальную информацию в реальном времени. Это дает возможность значительно расширить контекст работы моделей и минимизировать ограничения, связанные с фиксированными наборами данных. Основное преимущество RAG заключается в том, что он позволяет моделям работать с актуальными данными, извлекая их из внешних БЗ, таких как специализированные БД или корпоративные хранилища.

1.2.2. Принцип работы RAG

Перед началом работы системы необходимо подготовить БЗ, которая будет использоваться для извлечения данных. Документы кодируются в векторные представления и сохраняются в специализированных индексах.

На этапе извлечения система, используя механизм поиска, ищет релевантную информацию в БД, индексах документов или других внешних источниках [9]. Этот процесс включает следующие шаги [17; 3]:

1. Кодирование запроса. Запрос пользователя преобразуется в плотный вектор (высоко-размерный числовой), который улавливает не только семантическое значение слов, но и контекст, в котором они используются. Этот процесс выполняется с использованием нейронных сетей или специализированных моделей для извлечения, которые предварительно обучены для задач поиска.
2. Сравнение с БЗ. Запрос сравнивается с векторами, представляющими документы в БД. Для этого используются такие метрики, как косинусное сходство или евклидово расстояние. Цель состоит в том, чтобы найти документы, чьи векторные представления наиболее близки к запросу, так как это указывает на их релевантность.

3. Ранжирование результатов. Извлеченные документы ранжируются по степени их соответствия запросу. Методы ранжирования могут учитывать не только схожесть векторов, но и дополнительные факторы, такие как популярность документа, его источник или дата публикации.
4. Возвращение данных. Релевантные документы (или их части) возвращаются в генеративную модель, которая использует их для создания ответа.

На этапе генерации ответа генеративная модель принимает извлеченную информацию и использует ее для формирования осмысленного ответа. Этот процесс позволяет учитывать актуальные данные, которые не включены в исходный набор для обучения модели [16].

На рис.1.1 приведена схема функционирования концепции RAG.

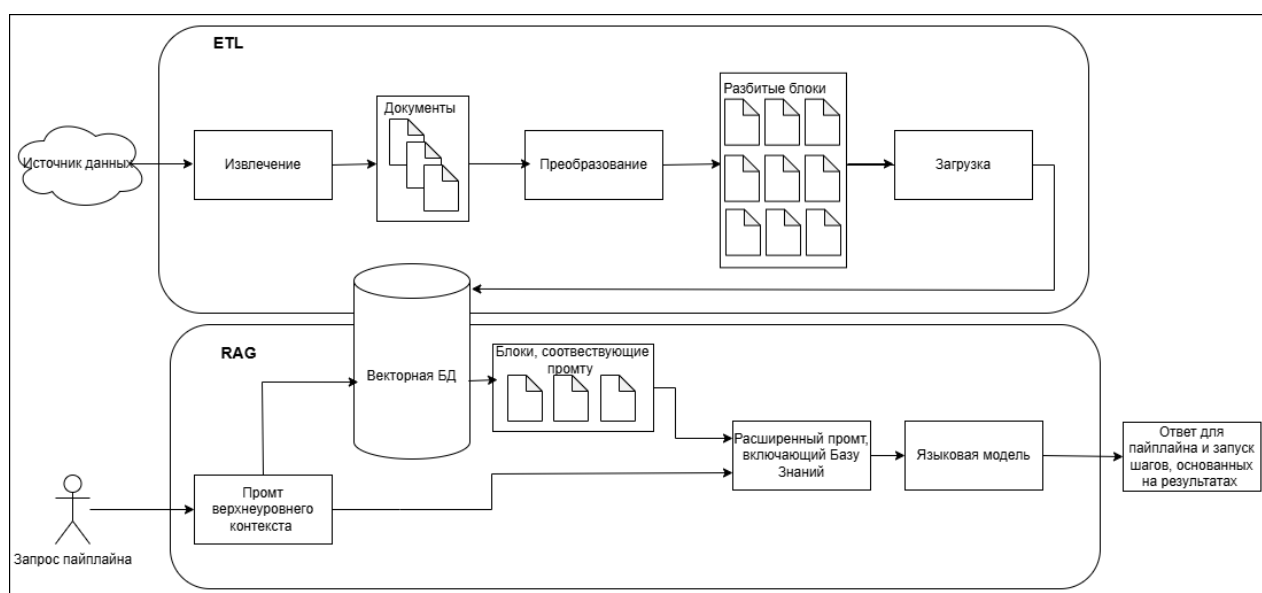


Рис.1.1. Принцип работы концепции RAG [17]

RAG представляет собой подход, который значительно расширяет возможности автоматизированного тестирования благодаря сочетанию механизмов извлечения релевантной информации и генеративного потенциала LLMs. Рассмотрим основные преимущества RAG в контексте автоматизации тестирования [3]:

- актуализация тестов в реальном времени. Современные ПО часто подвергаются регулярным изменениям, включая добавление новых функций, изменения существующих модулей или их интерфейсов, исправления ошибок, которые могут влиять на поведение системы. Эти изменения требуют обновления существующих тестов и создания новых сценариев

для проверки модифицированного кода. В ручных процессах тестирования такая адаптация часто занимает значительное время и может быть подвержена человеческим ошибкам. Автоматизация этого процесса с использованием RAG позволяет минимизировать эти риски и ускорить актуализацию тестов. Помимо этого, он позволяет формировать тесты, соответствующие контексту, которые учитывают новые функции или исправления в ПО. Благодаря постоянной синхронизации с БЗ тесты всегда соответствуют текущему состоянию системы;

- расширение покрытия кода. Один из ключевых факторов, влияющих на качество ПО, — это уровень покрытия кода тестами. Его расширение означает увеличение количества и разнообразия тестовых сценариев, которые проверяют функциональность, производительность и устойчивость программы. При ручном или традиционном автоматизированном тестировании многие сценарии остаются незатронутыми из-за ограничения времени и ресурсов, сложности выявления edge-case сценариев или недостаточной информации о поведении системы. Эти ограничения приводят к тому, что значительная часть кода остается не покрыта тестами, что увеличивает риск дефектов в релизе;
- снижение вероятности ошибок. Ручное тестирование и разработка тестов часто сопряжены с человеческими ошибками, такими как невнимательность, ограниченное понимание требований или упущение редких сценариев. RAG снижает вероятность таких ошибок за счет автоматизации и интеграции механизмов извлечения релевантной информации, что обеспечивает более точную генерацию тестовых сценариев;
- ускорение процесса тестирования. Скорость выполнения тестов является важным фактором в современных процессах разработки ПО, особенно в условиях применения методологий непрерывной интеграции. Концепция RAG значительно ускоряет процесс тестирования, автоматизируя задачи, которые в традиционных подходах требуют значительных временных затрат и ресурсов. Использование RAG позволяет командам быстрее реагировать на изменения, минимизировать время простоя и поддерживать скорость выпуска обновлений. Это особенно важно в условиях современных практик разработки, где задержки на этапе тестирования могут замедлить выпуск обновлений и снизить конкурентоспособность продукта.

Использование концепции RAG в автоматизации тестирования открывает новые возможности для повышения эффективности, качества и скорости разработки тестов. Кроме того, внедрение механизмов контроля и валидации генерируемых тестов минимизирует риски ошибок, а оптимизация архитектуры модели и подходов к обучению способствует снижению вычислительных затрат. Сочетание извлечения и генерации позволяет раскрыть потенциал LLMs, делая их более практичными для решения задач, требующих работы с постоянно обновляющейся информацией. Таким образом, концепция RAG не только решает многие вызовы, чем повышает их практическую ценность, что подтверждается разработкой и применением системы в рамках данного исследования.

1.3. Непрерывная интеграция

Непрерывная интеграция является ключевым процессом в современной разработке ПО. Эта практика позволяет ускорить разработку и значительно повысить ее качество за счет своевременного выявления и устранения ошибок. Внедрение процесса непрерывной интеграции помогает обеспечить стабильность и надежность разрабатываемого продукта. В данной главе рассматриваются основные концепции непрерывной интеграции, роль автоматизированного тестирования и особенности их применения для работы с большими языковыми моделями.

1.3.1. Понятие непрерывной интеграции

Непрерывная интеграция (Continuous Integration, далее — CI) — это методология разработки ПО, направленная на частое слияние кода из разных веток в общий репозиторий. Обычно в проекте задействовано несколько разработчиков, поэтому важно хранить весь код, над которым идет работа, централизованно. Также разработка ПО требует, как правило, различных платформ и инструментов, поэтому в этом случае важно иметь постоянный механизм проверки изменений. В идеале объединение должно выполняться автоматически несколько раз в день. Основная цель CI — выявление и устранение ошибок на ранних стадиях разработки, что снижает стоимость исправлений и повышает качество продукта [38].

Методы и практики CI:

- частые обновления кода. Разработчики регулярно объединяют свои изменения с основной веткой, избегая накопления конфликтов;

- автоматизация процессов сборки и тестирования. Центральная часть CI. Программный код автоматически компилируется в исполняемые форматы каждый раз, когда разработчики вносят изменения. Это ускоряет обнаружение проблем, связанных с интеграцией. После сборки запускаются различные уровни тестов (unit-тесты, интеграционные, системные), чтобы обеспечить качество кода. Это позволяет быстро выявлять и исправлять ошибки, гарантируя качество кода. Таким образом, автоматизация этих процессов значительно повышает эффективность и скорость разработки, минимизируя время, затрачиваемое на ручные процессы проверки и тестирования, тем самым поддерживая высокое качество конечного продукта;
- интеграция с системами контроля версий (Version Control System, далее — VSC). Основа для управления изменениями кода в CI. Они позволяют разработчикам их отслеживать и управлять ими в коде, обеспечивая возможность возвращения к предыдущим версиям и совместную работу над проектом. Эти системы автоматизируют процесс сохранения различных версий и изменений кода, что упрощает слияние кода и решение конфликтов. Благодаря системам контроля версий, внесенные изменения могут быть легко интегрированы и проверены в общей кодовой базе, что является ключевым для эффективной непрерывной интеграции. Примеры популярных инструментов включают: Git, Subversion, Mercurial, Perforce Helix Core, Microsoft Team Foundation Server [24];
- использование CI-инструментов. Данные инструменты являются ключевыми компонентами в процессе CI. Они автоматизируют и управляют процессами сборки, тестирования и интеграции кода. Эти инструменты позволяют настроить автоматические рабочие процессы, которые активируются при каждой фиксации изменений в систему контроля версий. Они также могут интегрироваться с другими инструментами и платформами, обеспечивая гибкость и универсальность в процессе разработки ПО. Примеры популярных инструментов включают: Jenkins, GitLab CI и CircleCI [24];
- обратная связь. CI предоставляет быстрые отчеты о состоянии системы, указывая на возникшие проблемы.

Внедрение непрерывной интеграции начинается с регулярной отправки изменений в систему управления версиями, чтобы все участники проекта работали

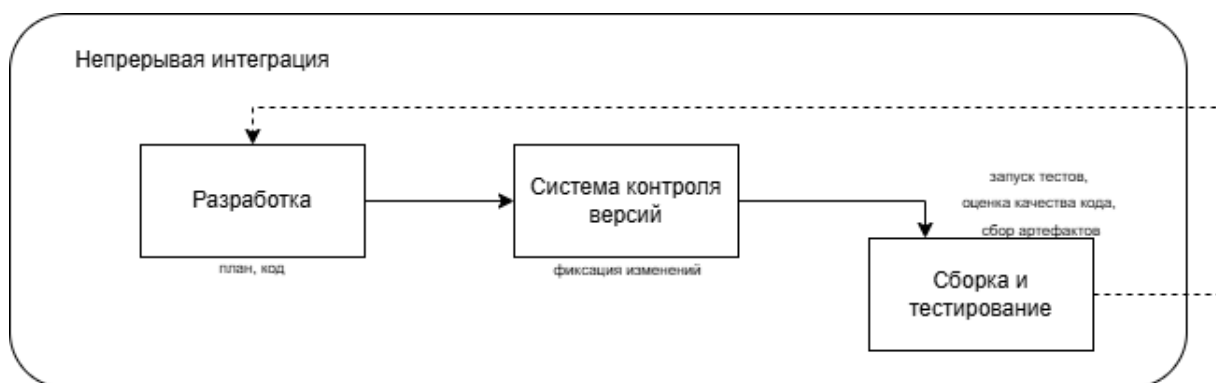


Рис.1.2. Процесс непрерывной интеграции [38]

с одинаковым кодом. Каждая фиксация изменений становится триггером для сборки и серии автоматизированных тестов, чтобы проверить поведение кода и убедиться, что изменение ничего не сломало. Непрерывная интеграция полезна и сама по себе, а еще она становится первым шагом к реализации CI-пайплайна. Внедрив эту практику в процесс разработки, команды имеют согласованный подход, который приводит к улучшению совместной работы и качества кода.

1.4. Анализ существующих инструментов автоматизации тестирования в контексте рассмотренных технологий

Для разработки системы автоматизированного тестирования с использованием больших языковых моделей и концепции RAG в процессе CI необходимо изучить существующие аналоги. Это позволит определить текущий уровень развития технологий, выявить сильные и слабые стороны аналогичных решений и обосновать актуальность разрабатываемой системы.

1.4.1. Инструмент *Postman + PostBot*

Postman [46] — это платный инструмент для разработки, тестирования и документирования API. Он предоставляет удобный интерфейс для отправки запросов и анализа ответов. Инструмент можно адаптировать для автоматизации API-тестирования, включая генерацию запросов и анализ ответов на основе создания коллекции API-запросов, импорта спецификации API (например, OpenAPI/Swagger) или ранее выполненных запросов AI, через функцию Postbot, которая доступна в виде дополнительного платного модуля.

Преимущества использования Postman + PostBot:

- удобство интерфейса;

- интеграция с LLMs;
- интеграция с CI — поддерживает экспорт коллекций и запуск тестов через Newman;
- гибкость настроек — возможность настройки кастомных скриптов для любых задач тестирования.

Недостатки использования Postman + PostBot:

- требуется кастомизация — интеграции AI требуют написания кастомных скриптов только на языке JavaScript, что увеличивает сложность настройки;
- ограничение в генерации сложных тестов — может генерировать базовые тесты, но для edge-case сценариев или интеграционных тестов требуется значительное вмешательство;
- отсутствие концепции RAG;
- риск нарушения конфиденциальности данных — использует облачные сервисы для обработки запросов и генерации тестов.

Использование данного инструмента доступно по следующим тарифным планам:

- бесплатный — для индивидуальных пользователей или небольших команд до 3 человек с лимитом в 50 действий на пользователя в месяц, используя Postbot;
- базовый план — \$28 за пользователя в месяц или \$276 в год;
- профессиональный план — \$48 за пользователя в месяц или \$456 в год;
- корпоративный план — стоимость \$58 за пользователя в месяц.



Рис.1.3. Postman + PostBot

Данный инструмент не может напрямую анализировать код. В его распоряжении может быть только то, что будет явно вставлено в чат для контекста. Не имеет возможности запрашивать данные из внешних или внутренних баз данных. Написания тестов возможно только на языке JavaScript, что создает дополнитель-

ную сложность в поддержании кода. Пример сгенерированного теста представлен на рис.1.3. Помимо этого, нарушение конфиденциальности при отправке запроса на сервера Postman и возможность оплаты только в иностранной валюте приводят к тому, что данный инструмент невозможно использовать на территории РФ в условиях санкций.

1.4.2. Инструмент GitHub Copilot

GitHub Copilot [39] — это платный инструмент автодополнения кода. Copilot интегрируется с IDE, такими как Visual Studio Code и IDE от JetBrains, и помогает разработчикам генерировать код, включая фрагменты для тестов. Хотя инструмент не специализирован для автоматизированного тестирования, его можно использовать для написания unit-тестов и других сценариев тестирования с минимальным вмешательством пользователя. Copilot анализирует текущий контекст в IDE, включая функцию или всю кодовую базу репозитория, с которым работает разработчик. На основе комментариев, названий функций или уже существующего кода, Copilot предполагает, что требуется сгенерировать.

Преимущества использования GitHub Copilot:

- ускорение написания кода;
- поддержка популярных ЯП — работает с большинством современных языков, включая Python, Java, JavaScript, Go, Ruby, и другие;
- интеграция с IDE — совместимость с Visual Studio Code и другими популярными IDE делает Copilot удобным для ежедневной работы;
- простота использования — не требует настройки инфраструктуры, готов к использованию сразу после установки;
- обучение на обширных данных — может генерировать код, основанный на большом количестве публичных репозиториях, что позволяет ему создавать тесты для типичных задач.

Недостатки использования GitHub Copilot:

- не поддерживает концепцию RAG — работает с данными внутри репозитория и не может обращаться во внешнюю базу знаний за дополнительными данными, необходимыми для контекста;
- ограниченный список доступных языковых моделей;
- отсутствие интеграции с CI;

- риск генерации неправильного кода — может генерировать код, который выглядит корректным, но содержит логические ошибки или не соответствует бизнес-требованиям;
- риск нарушения конфиденциальности данных — фрагменты кода, с которым работает пользователь, отправляются на удаленные серверы, где они обрабатываются существующей моделью машинного обучения. Это означает, что исходный код, включая конфиденциальные или проприетарные данные, может быть передан за пределы локальной среды разработки;
- ограниченная универсальность — фокусируется на написании кода и unit-тестов, но не поддерживает комплексное тестирование (например, API или интеграционные).

Также важно учитывать, что для использования данного инструмента существует несколько вариантов подписки в зависимости от потребностей:

- Copilot Free — бесплатный, но ограниченный доступ к определенным функциям Copilot, включая до 2000 завершений кода и 50 сообщений в чате ежемесячно, что ограничивает регулярное использование данного инструмента;
- Copilot Pro — стоимость составляет \$10 в месяц или \$100 в год для индивидуальных пользователей с доступом ко всем функциям Copilot;
- Copilot Business — стоимость составляет \$19 за пользователя в месяц для организаций и команд, предоставляя расширенные возможности и управление политиками на уровне организации;
- Copilot Enterprise — стоимость составляет \$39 за пользователя в месяц для крупных предприятий, предлагая дополнительные функции и настройки.

При тестировании данного инструмента, было выявлено, что Copilot может генерировать рабочие API тесты, но только при дополнительном участии разработчика, так как в сгенерированном коде не хватало импортов классов проекта, установленных библиотек и необходимой инфраструктуры для инициализации используемых клиентов. Пример генерации API тестов представлен на рис.1.4a. При необходимости использовать дополнительные данные (политики написания кода, бизнес-документации, спецификации) нужно дублировать файлы в каждом репозитории, чтобы использовать для контекста внутри IDE, так как возможный контекст ограничивается рамками репозитория. Возможность выбора файлов для контекста представлена на рис.1.4b. Copilot представляет ограниченный список языковых моделей и не дает возможности использовать другие языковые модели.

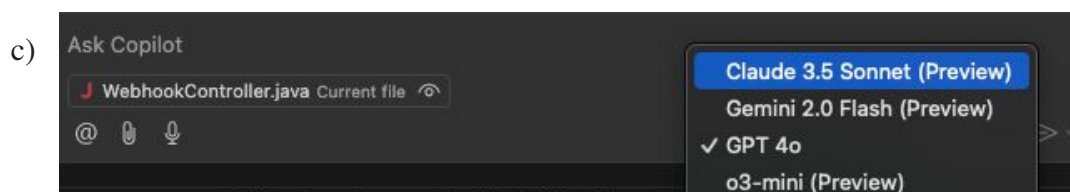
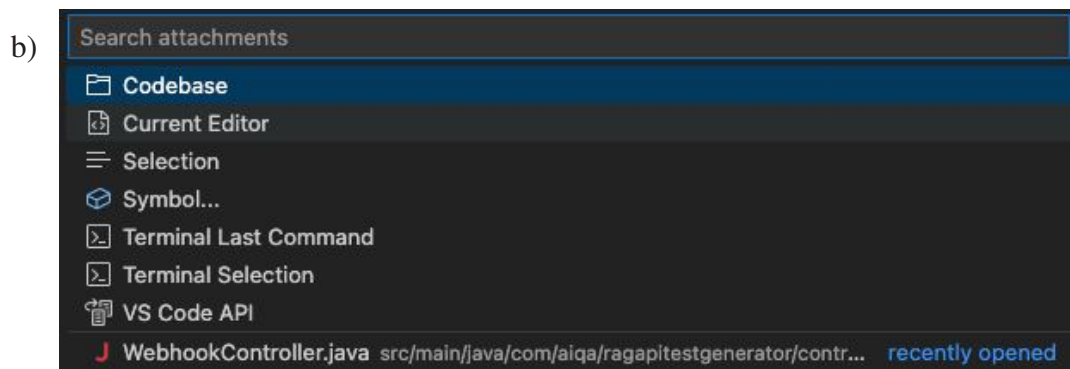
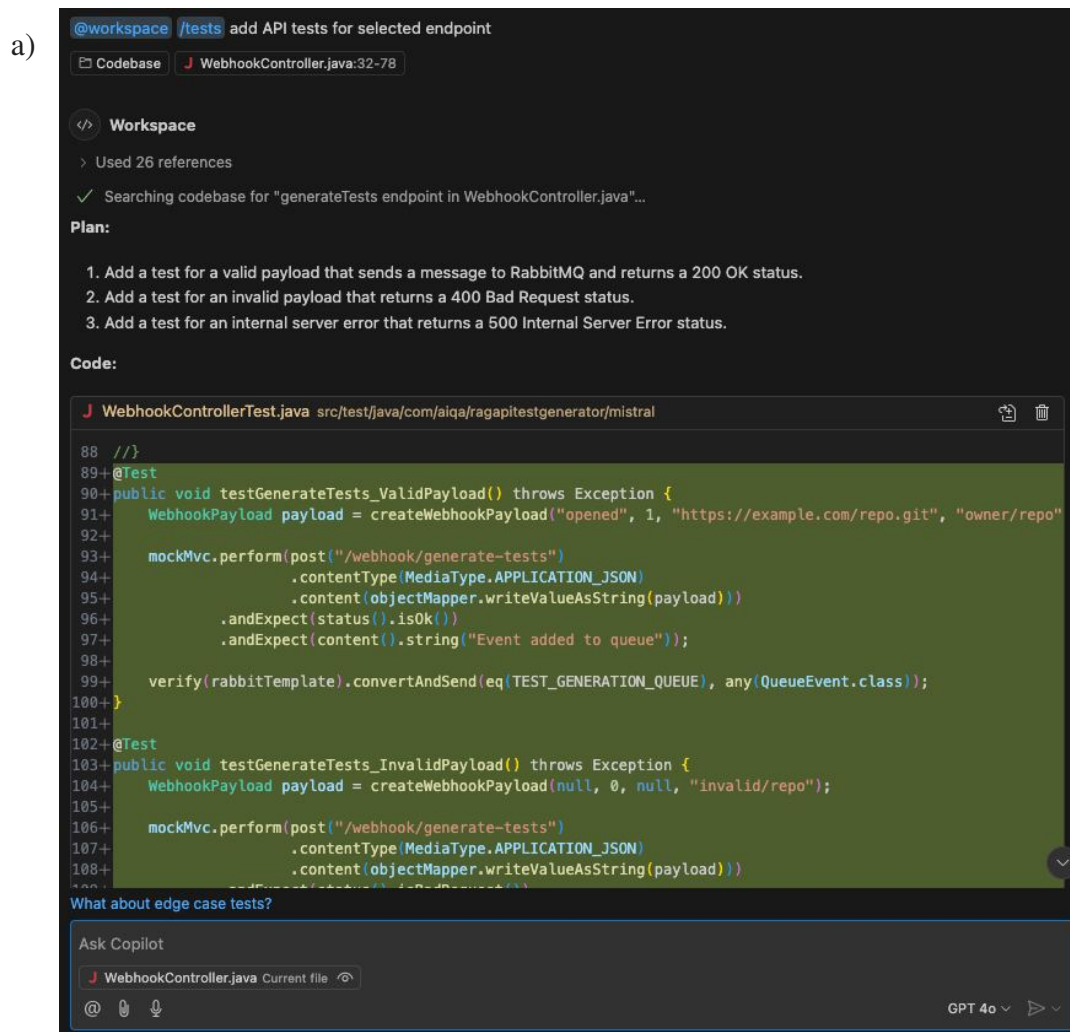


Рис.1.4. GitHub Copilot, включая: *a* — использование; *b* — список возможного расширения контекста; *c* — список доступных моделей

Доступные LLMs для использования представлены на рис. 1.4с. Помимо минусов выше, использование данного инструмента может привести к нарушению политики безопасности, так как запросы на генерацию ответа отправляются на сервера компаний, представляющие языковые модели. А также, в условиях санкций, российские компании и организации не могут его применять или могут использовать в условиях санкций и доступной оплаты в иностранной валюте.

1.4.3. Инструмент IntelliJ IDEA + AI Assistant

AI Assistant от JetBrains [43] — это платный интеллектуальный помощник, интегрированный в среду разработки IntelliJ IDEA, который использует ИИ для упрощения написания кода, тестирования и документирования. Данный инструмент может автоматически генерировать модульные тесты, анализировать код и давать рекомендации по улучшению его качества. Он позволяет разработчикам автоматизировать процесс тестирования, создавая тестовые сценарии на основе структуры кода и комментариев.

Преимущества использования IntelliJ IDEA + AI Assistant:

- генерация тестов с помощью AI — автоматическое создание на основе анализа кода;
- глубокая интеграция с IntelliJ IDEA — встроенный в среду разработки инструмент, не требующий установки стороннего ПО;
- объяснение кода — анализ существующего кода и предоставление пояснений, что упрощает рефакторинг и тестирование;
- гибкость и адаптация — возможность кастомизации тестов и внесения корректив в предложенные AI сценарии;
- интеграция с CI — тесты могут быть автоматически сгенерированы и интегрированы в процесс непрерывной интеграции.

Недостатки использования IntelliJ IDEA + AI Assistant:

- ограничение в генерации сложных тестов — инструмент эффективен для модульного тестирования, но сложные интеграционные сценарии требуют доработки вручную;
- отсутствие концепции RAG — работает с данными внутри репозитория и не может обращаться во внешнюю базу знаний за дополнительными данными, необходимыми для контекста;

- нет автоматической актуализации контекста — AI не обновляет тесты динамически при изменении кода;
- зависимость от документации — генерация тестов во многом зависит от качества комментариев в коде;
- риск конфиденциальности данных — AI Assistant использует облачные сервисы для анализа кода, что может представлять угрозу для конфиденциальных проектов.

Использование данного инструмента доступно по следующим тарифным планам⁴:

- бесплатный — пробная бесплатная 7-дневная версия с доступом ко всем функциям с ограничениями по времени и объему использования;
- профессиональный план — \$12.5 за пользователя в месяц или \$121 в год;
- корпоративный план — стоимость \$25 за пользователя в месяц или \$248 в год.

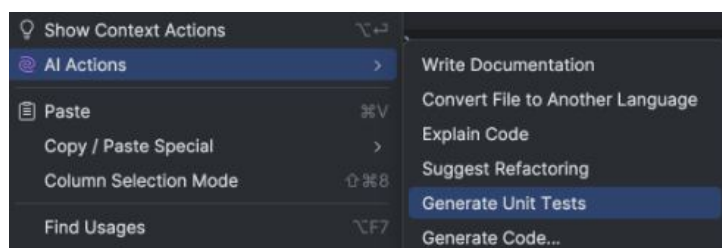


Рис.1.5. IntelliJ IDEA + AI Assistant

При тестировании AI Assistant от JetBrains было выявлено, что работает он похожим образом, что и GitHub Copilot. Он может генерировать рабочие API тесты при дополнительном взаимодействии разработчика с чатом ИИ, но не может использовать базу знаний за пределами репозитория. Удобен для написания unit-тестов, так как имеет готовую функцию, включающую необходимый контекст для генерации. Демонстрация возможности выбора функции генерации unit-тестов представлена на рис.1.5. Ограничения использования такие же, как и у GitHub Copilot. Использование данного инструмента может привести к нарушению политики безопасности, так как запросы на генерацию ответа отправляются на сервера компаний. Оплата возможна только в иностранной валюте.

⁴AI Assistant доступен только в платных версиях сред разработки, поэтому для его использования необходима также платная версия Ultimate IntelliJ IDEA

1.4.4. Инструмент Zencoder

Zencoder [65] — это платный интуитивный агент генерации кода с помощью ИИ с возможностью встраивания в виде плагина в популярные IDE (VSCode и IDE от JetBrains). Может генерировать разнообразные тесты, исправлять, а также самостоятельно вставлять сгенерированный код в файлы.

Преимущества использования Zencoder:

- ускорение написания кода посредством автодополнения;
- поддержка более 70 ЯП;
- интеграция с IDE — Visual Studio Code и продуктами JetBrains;
- простота использования — не требует настройки инфраструктуры, готов к использованию сразу после установки;
- автоматическая вставка изменений в файлы;
- возможность расширения контекста за счет прикрепления файлов не входящих в репозиторий и кодовую базу.

Недостатки использования Zencoder:

- отсутствие интеграции с CI;
- риск нарушения конфиденциальности данных—все запросы отправляются на сервера компании Zencoder;
- ИИ агент проходит beta-тестирование.

Использование данного инструмента доступно по следующим тарифным планам:

- бесплатный — до 10 разработчиков с урезанным функционалом ИИ агента;
- профессиональный план — \$19 за пользователя в месяц с расширенным функционалом ИИ агента, но без пользовательских настроек;
- корпоративный план — стоимость \$39 за пользователя в месяц с полным функционалом и дополнительной настройкой для компании.

Данный плагин работает похожим образом как GitHub Copilot и AI Assistant в IntelliJ IDEA, но имеет важное преимущество. Zencoder может самостоятельно вносить изменения в файлы, тем самым пропуская этап дополнительных действий со стороны разработчика. Имеется возможность использовать кодовую базу репозитория как у инструментов выше, но также возможно подгружать файлы с компьютера, не входящих в репозиторий. Генерирует рабочие API тесты со всеми необходимыми импортами и инфраструктурой. Пример использования расположен на рис.1.6. После проведенных тестов можно сделать вывод, что является удобным,

a)

Cover by end-to-end tests WebhookController

Attached context ~ Codebase WebhookController.java

Zencoder
2 references used

To cover the `WebhookController` with end-to-end tests, you need to simulate HTTP requests to the endpoints defined in the controller and verify the responses. This can be achieved using a testing framework like Spring Boot's `MockMvc` for simulating HTTP requests and assertions.

Here's a step-by-step guide to creating end-to-end tests for the `WebhookController`:

- 1. Set Up Your Test Class:** Annotate your test class with `@SpringBootTest` and `@AutoConfigureMockMvc` to enable Spring Boot's testing features and configure `MockMvc`.
- 2. Inject MockMvc:** Use `@Autowired` to inject `MockMvc` into your test class.
- 3. Write Test Methods:** Create test methods for each endpoint in the `WebhookController`. Use `MockMvc` to perform HTTP requests and assert the expected outcomes.

Here's an example of how you might write these tests:

```

import com.aiqa.ragapitestgenerator.model.WebhookPa
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation
import org.springframework.boot.test.autoconfigure.
import org.springframework.boot.test.context.Spring

```

b)

New chat

Coding Agent (Beta): Uses advanced AI workflows to perform common development tasks, such as bug fixing, refactoring, or new feature development. Recommended for complex queries.

Context ☒ Coding agent Beta

Рис.1.6. Zencoder, включая: *a* — использование; *b* — функция включения ИИ-агента

сильным инструментом генерации кода. Один из минусов, найденных в процессе тестирования - иногда вставляет дублирующий код в существующие классы, тем самым делая код нерабочим. Имеет такие же минусы как у инструментов выше, заключающиеся в невозможности использования в пределах РФ и нарушении конфиденциальности. Помимо этого отсутствует возможность использования CI. Но большим плюсом является генерация готового кода, минимизирующего участие разработчика.

1.4.5. Инструмент Cursor

Cursor IDE [62] — платная среда разработки, разработанная на основе VSCode, с использованием внешних языковых моделей, но с наличием оплаченных подписок для более глубокого использования. Полноценный инструмент генерации кода, а также автоматического внесения изменений в репозиторий без участия разработчика. Может генерировать любые тесты с учетом различных тест-кейсов. Является популярным инструментом в сообществе разработчиков.

Преимущества использования Cursor IDE:

- ускорение написания кода посредством автодополнения;
- поддержка большого количества ЯП;
- автоматическая вставка изменений в файлы;
- возможность расширения контекста за счет прикрепления файлов не входящих в репозиторий и кодовую базу.

Недостатки использования Cursor IDE:

- отсутствие интеграции с CI;
- риск нарушения конфиденциальности данных — все запросы отправляются на сервера компаний представленных языковых моделей;
- не имеет возможности работать как встраиваемый плагин в других IDE.

Использование данного инструмента доступно по следующим тарифным планам:

- бесплатный — ограниченное количество запросов к ИИ;
- профессиональный план — \$20 за пользователя в месяц;
- корпоративный план — стоимость \$40 за пользователя в месяц с удобным мониторингом и билингом для компаний.

После проведенных тестов показал себя лучше всего, так не вставляет дублирующий код в существующие классы, а корректно анализирует код в файлах

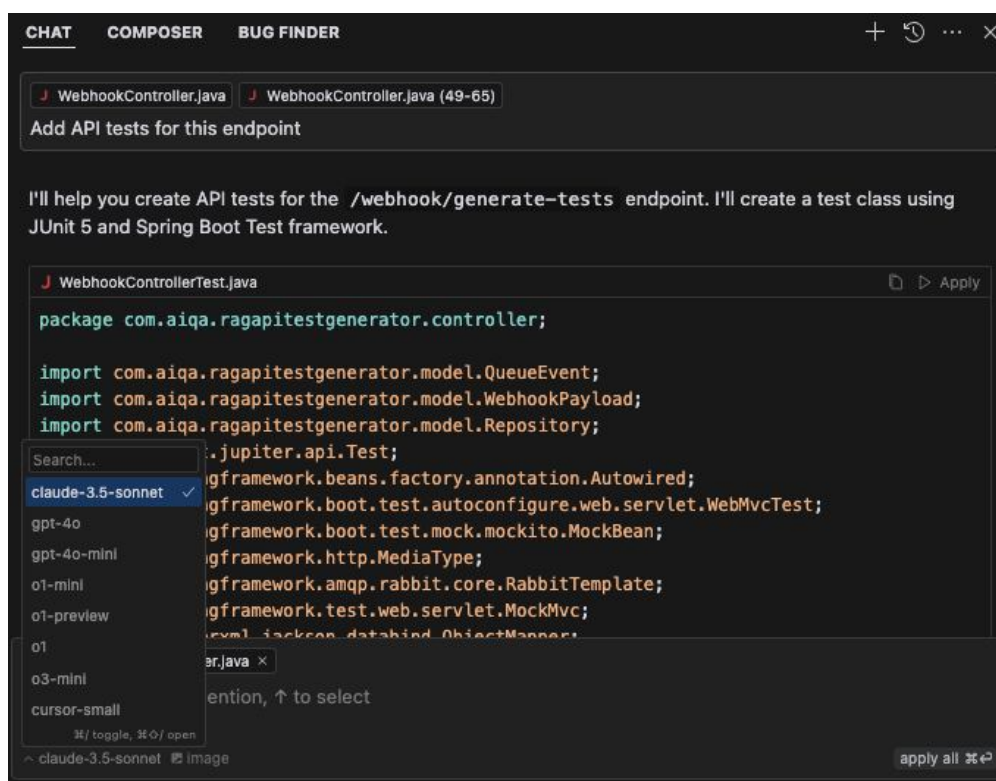


Рис.1.7. Cursor IDE

и вставляет только нужные части кода. Пример использования расположен на рис.1.7. Но из-за нарушения конфиденциальности и отсутствия интеграции с CI проигрывает решению, разработанному в рамках выпускной квалификационной работы.

1.4.6. Сравнение инструментов

После рассмотрения инструментов автоматизации тестирования были проанализированы их преимущества и недостатки, связанные с их использованием. На основе проведенного анализа были выделены ключевые параметры, которые позволяют объективно сравнить данные инструменты, такие как генерация тестов, интеграция с CI, использование LLMs, возможности работать по концепции RAG, удобство использования, стоимость лицензии и сохранение конфиденциальности обрабатываемых данных, рассмотренные в табл.1.3). Они были выбраны как оптимальные, поскольку отражают как функциональные возможности инструментов, так и их практическую применимость в условиях реальных задач автоматизации тестирования. Все рассмотренные инструменты являются платными, что является барьером для их использования в условиях санкций. Помимо этого при использовании рассмотренных инструментов возникает риск нарушения конфиденциальности данных, так как запрос обрабатывает на сторонних серверах компаний, представ-

ляющих данные инструменты генерации кода. Также они имеют ограничения при работе с внешними приватными базами данных для расширенного контекста, так как ограничены только тем, что импортируется на момент формирования запроса.

В свою очередь разработанная система представляет собой проект с открытым исходным кодом, что делает ее доступной для любой организации, желающей повысить скорость процесса тестирования. Также все аналоги, использующие LLM, не предусматривают получение дополнительных данных из других приватных баз данных компании, что снижает их адаптивность к изменениям в ПО, для тестирования которого они применяются. Помимо этого, рассмотренные инструменты, использующие LLM для генерации тестов, предполагают отправку данных на сторонние серверы, что может быть нежелательным для организаций с высокими требованиями к конфиденциальности. Разработанная система, развертываемая на серверах компании, в таком случае обеспечивает соответствие этим требованиям.

Таблица 1.3

Сравнительная таблица существующих инструментов

Критерий	GitHub Copilot [39]	IntelliJ IDEA + AI Assistant [43]	Postman + PostBot [46]	Zencoder [65]	Cursor [62]	Разрабатываемая система
Генерация тестов	Функциональные и интеграционные тесты и функция автодополнения фрагментов тестов.	Функциональные и интеграционные тесты и функция автодополнения фрагментов тестов.	Автотесты API.	Функциональные и регрессионные тесты	Функциональные и интеграционные тесты и функция автодополнения фрагментов тестов.	Функциональные и интеграционные тесты и функция автодополнения фрагментов тестов.
Интеграция с CI	Нет.	Да: автоматическая интеграция с CI.	Да, вручную настраивается.	Нет.	Нет.	Да.
Использование LLMs	Да.	Да.	Да: для генерации запросов и тестов.	Да.	Да.	Да: генерация тестов и анализ через LLM.
Использование концепции RAG	Да: ограничена кодовой базой репозитория.	Да: ограничена кодовой базой репозитория.	Нет.	Да: ограничена кодовой базой репозитория.	Да: ограничена кодовой базой репозитория.	Да: автоматическое обновление контекста.
Удобство использования	Да: простота использования для автокода.	Да, интуитивно понятен.	Да: знакомый интерфейс.	Да: простота использования для автокода.	Да: ограничено использование конкретного IDE.	Да: автоматизация и интеграция с CI.

Продолжение табл. 1.3

Лицензия	Платная (от \$10/мес).	Платная (от \$12.5/мес, требуется IntelliJ IDEA Ultimate).	Бесплатная(для индивидуальных клиентов с урезанным функционалом). Платная (от \$28/мес)	Бесплатная(для индивидуальных клиентов с сильно урезанным функционалом). Платная (от \$19/мес)	Бесплатная(для индивидуальных клиентов с сильно урезанным функционалом). Платная (от \$20/мес)	Открытый исходный код, с возможностью адаптации под проекты.
Конфиденциальность	Риск утечки данных: код отправляется в облако OpenAI.	Риск утечки данных: анализ кода в облаке JetBrains	Риск утечки данных: тесты генерируются через облако Postman.	Риск утечки данных: код отправляется во внешние ресурсы.	Риск утечки данных: код отправляется во внешние ресурсы.	Обработывает данные локально.

1.5. Выводы по главе 1

LLMs, обученные на программном коде, демонстрируют возможности генерации тестов и автоматизации рутинных задач в процессе их написания. Однако LLMs ограничены устареванием знаний и неспособностью обрабатывать динамические данные, что снижает их применимость в условиях постоянно изменяющегося кода. Концепция RAG компенсирует эти недостатки, предоставляя доступ к актуальной информации из внешних БД, что позволяет моделям генерировать тесты, релевантные текущему состоянию системы. Более того, использование RAG обеспечивает контекстуальность тестов, адаптированных к специфике проекта.

Практики CI служат основой для автоматизации и ускорения разработки. Интеграция тестов в пайплайн CI позволяет автоматизировать их выполнение, анализ результатов, что может способствовать сокращению времени цикла разработки.

Анализ рассмотренных технологий и инструментов для автоматизации тестирования, а также существующих решений, таких как GitHub Copilot, IntelliJ IDEA с AI Assistant, Postman с PostBot, Zencoder и Cursor, позволил выявить их сильные и слабые стороны, а также невозможность их использования в условиях санкций. В ходе анализа объект исследования ограничен API-функционалом тестируемых программных средств, поскольку этот уровень системы предоставляет стабильные интерфейсы взаимодействия и является важным элементом при разработке и интеграции сервисов. Такой подход позволяет сфокусироваться на обеспечении надежности и функциональной целостности API, что актуально в условиях динамически изменяющихся программных продуктов.

Таким образом, сочетание LLMs, концепции RAG и практик CI создает фундамент для разработки системы автоматизированного тестирования API. Эти технологии позволяют не только автоматизировать рутинные процессы, но и повысить стабильность системы, обеспечивая более точное и адаптивное тестирование. Обзор продемонстрировал, что ни одно из существующих решений не удовлетворяет всем критериям, рассмотренным в 1.4.6, необходимых для обеспечения требований гибкой разработки ПО. Это подтверждает актуальность создания системы, которая объединит преимущества рассмотренных технологий с возможностью преодоления их ограничений, ориентируясь на автоматизацию тестирования API как ключевого компонента современных программных систем.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ СРЕДСТВА ДЛЯ АВТОМАТИЗИРОВАННОГО ТЕСТИРОВАНИЯ API

Глава посвящена анализу требований к системе автоматизации тестирования, проектированию ее структуры и определению ключевых принципов ее эксплуатации в рамках процессов непрерывной интеграции.

В параграфе 2.1 проводится анализ основных функциональные и нефункциональные требований к средству автоматизированного тестирования. В параграфе 2.2 описываются участники и основные сценарии использования системы. В параграфе 2.3 рассмотрена диаграмма состояний, которая отражает этапы жизненного цикла системы. Параграф 2.4 посвящен функциональному моделированию системы с использованием методологии IDEF0.

В параграфе 2.5 рассматриваются ключевые элементы проектирования архитектуры системы. Параграф 2.5.1 описывает процесс генерации тестов. В параграфе 2.5.2 анализируется процесс актуализации БЗ для обеспечения актуальности и надежности системы. Параграф 2.5.3 посвящен особенностям проектирования системы, включая адаптацию архитектуры к специфическим требованиям эксплуатации.

2.1. Анализ основных требований

Ключевым этапом проектирования является анализ требований для определения ключевых функциональных и нефункциональных характеристик разрабатываемой системы автоматизированного тестирования, обеспечивающих ее соответствие целям выпускной квалификационной работе. Эти требования сформулированы на основе анализа существующих инструментов с аналогичным функционалом, что позволило выявить их сильные и слабые стороны (см. подраздел 1.4) и учесть их при проектировании разрабатываемой системы.

А. Функциональные требования.

1. Генерация автоматических тестов:

- а) автоматическое создание с использованием LLMs на основе изменений в тестируемом ПО;
- б) формирование тестов тестирования API-функциональности.

2. Работа с контекстом:

- a) использование концепции RAG для формирования расширенного контекста тестов;
- b) извлечение релевантной информации из векторной базы данных (далее — БД) для генерации более точных и контекстно релевантных тестов.

3. Интеграция с CI:

- a) запуск тестов после изменений в коде;
- b) автоматическое создание pull request (далее — PR) при успешном выполнении тестов.

4. Актуализация БЗ:

- a) обновление данных в векторной БД после создания PR тестов, чтобы учитывать изменения и улучшать контекст для будущих итераций.

5. Анализ результатов тестирования:

- a) предоставление отчетов о выполнении тестов с идентификацией успешных и проваленных сценариев.

В. Нефункциональные требования.

1. Производительность:

- a) время генерации тестов должно быть минимальным, чтобы не замедлять процесс разработки;
- b) обработка запросов к векторной БД должна происходить с минимальными задержками.

2. Масштабируемость:

- a) возможность обработки больших объемов данных (например, документации, тестов).

3. Надежность:

- a) обеспечение стабильной работы системы даже при высоких нагрузках;
- b) предотвращение потери данных при сбоях.

4. Безопасность:

- a) поддержка работы с приватными данными и конфиденциальной информацией.

5. Модульность:

- а) архитектура системы должна быть модульной, чтобы обеспечить легкость обновления, замены или добавления новых компонентов.

2.2. Использование системы

На основании проведенного анализа требований, разрабатываемая система автоматизации тестирования должна обеспечивать взаимодействие двух основных участников: разработчика и QA-инженера. Для визуализации этих взаимодействий и демонстрации функциональных возможностей системы построена диаграмма случаев использования, которая описывает основные сценарии применения системы в процессе непрерывной интеграции. Диаграмма случаев использования становится фундаментом для дальнейшей разработки системы, отражая ее целевое назначение и области применения.

2.2.1. Основные участники

Система взаимодействует с двумя ключевыми участниками:

- А. Разработчик — инициирует изменения в коде, создает PR и анализирует результаты тестов.
- В. QA-инженер — проверяет сгенерированные тесты, анализирует отчеты о выполнении тестов и управляет настройками тестирования.

Участники взаимодействуют с разработанной системой через CI-инструмент, который оркеструет процесс тестирования, включая работу с такими компонентами, как CI-пайплайн, векторная БД и LLM. На рис.2.1 ниже представлена диаграмма случаев использования, которая визуализирует взаимодействие разработчика и QA-инженера с системой:

2.2.2. Основные случаи использования

Система автоматизированного тестирования предоставляет пользователям следующие функциональные возможности:

- А. Генерация автотестов: после создания PR разработчиком система извлекает изменения из репозитория, формирует контекст с использованием RAG и создает расширенный промт, который передается LLM для созда-

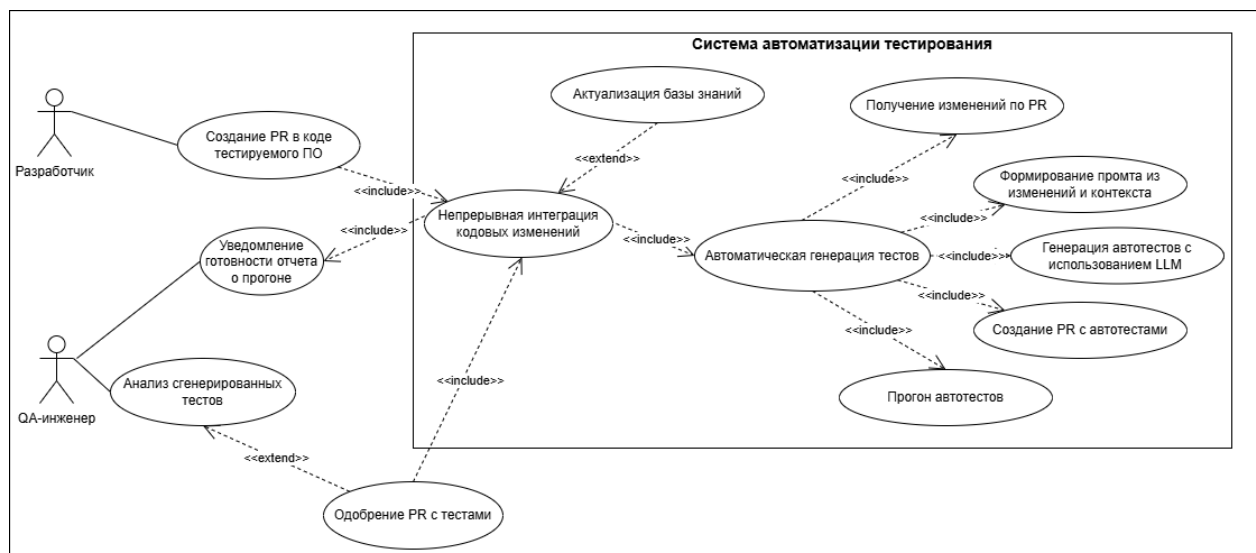


Рис.2.1. Диаграмма случаев использования

ния тестов. После успешной генерации система автоматически создает PR с тестами.

- В. Запуск тестов и анализ результатов: система выполняет автотесты, сохраняет результаты их выполнения и анализирует данные для формирования отчетов. Отчеты предоставляются разработчику и QA-инженеру для проверки и анализа.
- С. Актуализация БЗ: после успешного завершения тестов и слияния PR система обновляет контекст в векторной БД, добавляя новые данные и удаляя устаревшую информацию.

2.3. Диаграмма состояний

Для более глубокого понимания работы разрабатываемой системы автоматизированного тестирования построена диаграмма состояний. Она демонстрирует внутренние переходы системы между состояниями в зависимости от внешних событий, таких как создание PR, выполнение тестов или обновление БЗ. Диаграмма состояний позволяет отразить, как система управляет своими процессами на каждом этапе выполнения задач.

2.3.1. Основные состояния системы

- ожидание изменений: система находится в состоянии ожидания уведомления о PR, поступающего от CI/CD пайплайна;

- получение изменений: после получения уведомления система извлекает информацию об изменениях в коде из репозитория;
- генерация автотестов: сформированный промт передается в LLM для создания тестов;
- запуск тестов: система запускает сгенерированные тесты и сохраняет результаты их выполнения;
- анализ результатов: на основании логов система определяет успешность выполнения тестов;
- актуализация контекста: если тесты успешны, система обновляет контекст в векторной БД;
- ошибка или возврат: в случае неудачи система либо уведомляет пользователей о проблемах, либо возвращается в состояние ожидания изменений.

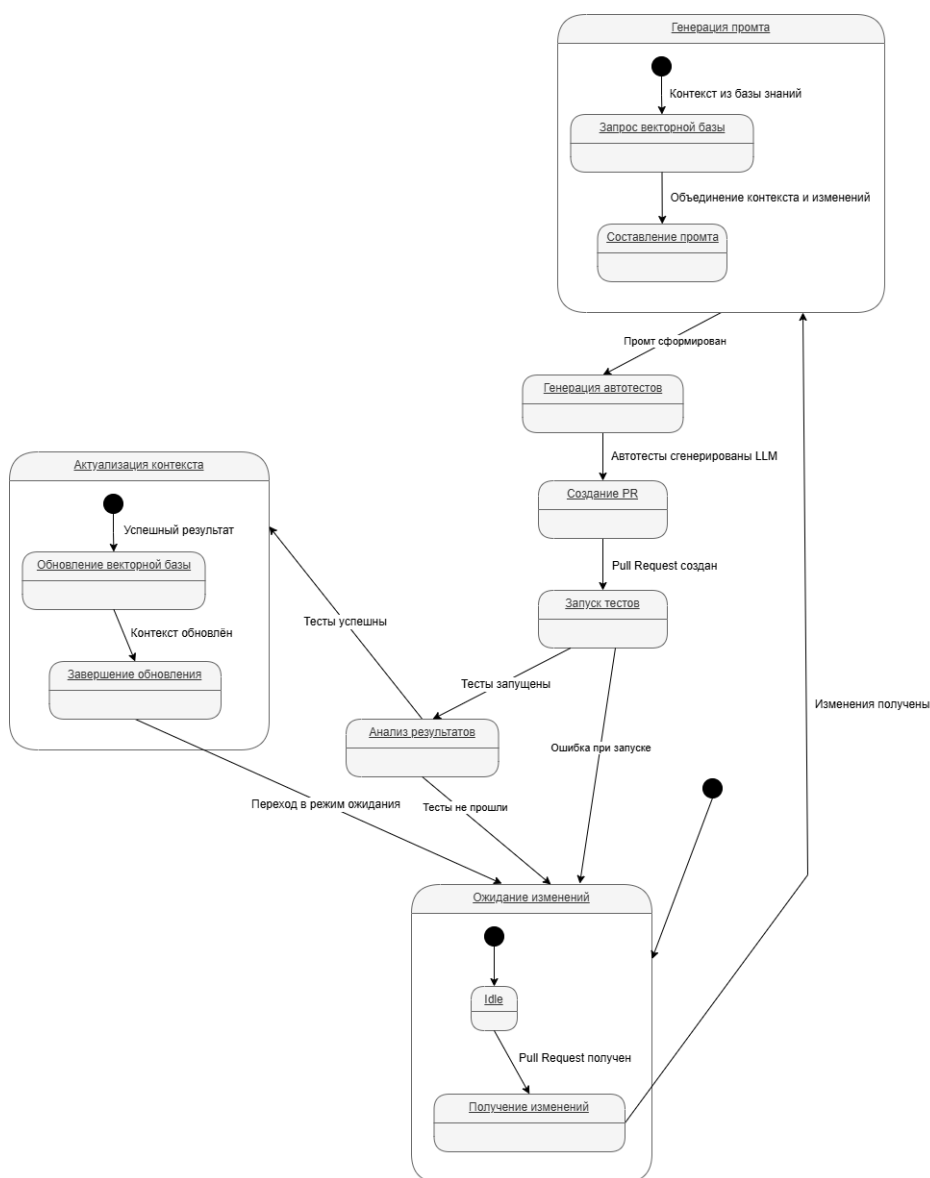


Рис.2.2. Диаграмма состояний

На рис.2.2 представлена диаграмма состояний.

2.4. Функциональное моделирование системы

Функциональное моделирование системы представляет собой ключевой этап проектирования, направленный на описание основных процессов и взаимодействий в системе автоматизации тестирования. Для этого используются IDEF0-диаграммы, которые позволяют наглядно представить декомпозицию функциональности системы. В рамках данного раздела представлены IDEF0-диаграммы, описывающие ключевые процессы системы.

2.4.1. IDEF0-диаграмма уровня A-0

Для демонстрации общей структуры разработанной системы автоматизированного тестирования с использованием LLM в процессе непрерывной интеграции на рис.2.3 ниже представлена IDEF0-диаграммы уровня A-0.

Входы:

- вебхук события от системы контроля версий (Version Control System, далее — VCS)
- изменения в репозитории тестируемого приложения;
- контекст (техническая документация, существующие тесты);
- типы и интерфейсы, релевантный код;
- предварительный промт.

Управляющие воздействия:

- правила и политики тестирования: общие требования, критерии качества, необходимые форматы тестов и т. д.;
- настройки системы RAG: регламенты по использованию модели, лимиты токенов, конфигурации подсказок;
- политики CI: триггеры запуска, правила слития PR'ов и т. д..

Механизмы:

- LLM — модель, к которой отправляются запросы;
- векторная БД — контекст, хранимый и дополняемый при успехе тестов;
- инструменты сборки и запуска тестов — система CI, тестовые окружения, Docker-контейнеры и т. д.;

- инструменты анализа кода — средства для выявления и отслеживания изменений в тестируемом ПО;
- CI-система — система для оркестрации процессов;
- VCS;
- разработчик.

Выходы:

- PR с автотестами — при успешном прохождении тестов система создает новый PR в репозиторий;
- обновленный контекст векторной БД — итоги запуска тестов могут актуализировать хранимое знание;
- отчеты о результатах тестирования — журнал выполнения тестов с указанием их статуса (успех/неуспех).

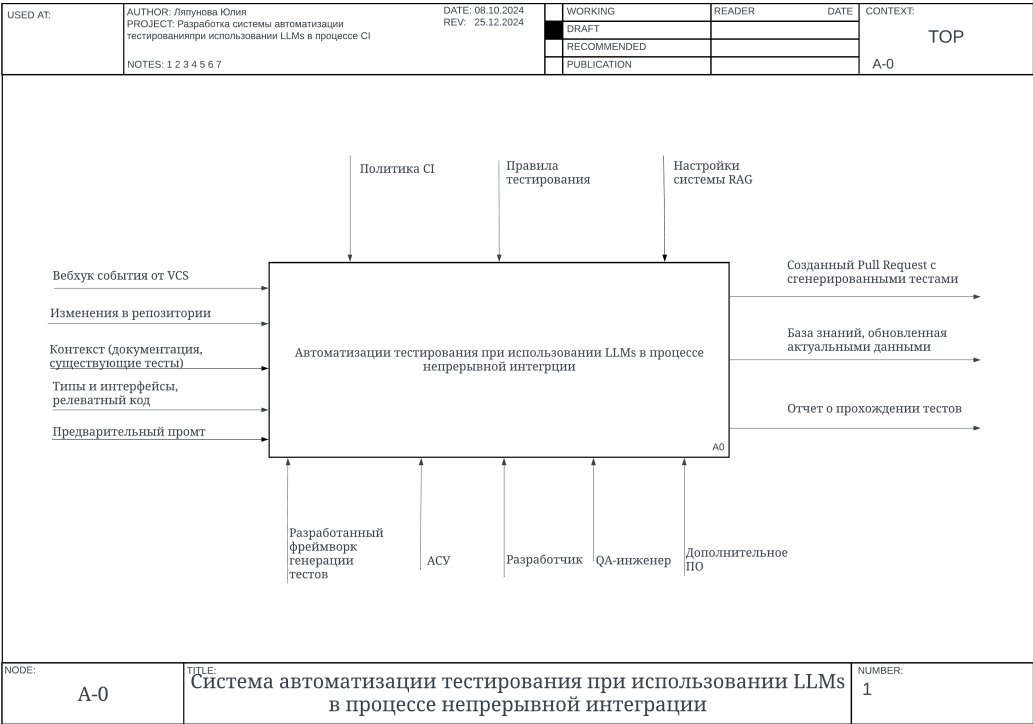


Рис.2.3. Основная диаграмма IDEF0

2.4.2. IDEF0-диаграмма уровня A0

Анализ изменений в коде (A1) — это блок, отвечающий за определение того, какие именно изменения внесены в репозиторий. В процессе выполняется взаимодействие с системой контроля версий для извлечения данных о PR и изменениях в ветке. Затем проводится идентификация изменений, включая добавление, удаление или модификацию API-маршруты, а также выбор областей кода, требующих

тестирования. Этот выбор основывается на настройках и политиках, таких как фильтрация по используемым фреймворкам и модулям. Результатом работы блока является список модулей, функций и файлов, затронутых изменениями, который используется для дальнейшего формирования автотестов.

Формирование расширенного промта (A2), представляющий собой подробную подсказку, осуществляется на основе сведений об изменениях, выявленных на блоке A1, а также контекста. Запрашиваются релевантные данные на основе ключевых изменений из векторной БД для создания семантически богатого промта. Он строится на политик системы, включая лимиты токенов, формат автотестов и степень детализации. Итоговый расширенный промт служит основой для последующей генерации.

Генерация автотестов (A3) осуществляется с использованием LLM на основе расширенного промта, сформированного на A2. В процессе создаются тесты для тестируемых API-маршрутов, включая edge-case сценарии, функциональное покрытие, проверку безопасности и обработку ошибок. Для каждого API-маршрута генерируется отдельный класс, содержащий все тесты, относящиеся к нему, в соответствии с установленной структурой. Сгенерированные тесты проходят проверку на соответствие заданным стандартам, включая используемый ЯП, структуру классов и применяемые фреймворки, что обеспечивает их читаемость и соответствие требованиям проекта.

Запуск тестов и анализ результатов (A4) выполняются в два этапа для оптимизации процесса тестирования. На первом этапе запускается ограниченный тестовый прогон, включающий только созданные или измененные классы. Если первый этап завершается успешно, инициируется второй этап — полный тестовый прогон, охватывающий все тестовые классы, содержащиеся в тестовом репозитории. В результате выполнения тестов результаты сохраняются, включая информацию об успехах, ошибках и сбоях, что позволяет проводить анализ прохождения тестов. Итогом работы блока является генерация отчетов с визуализацией результатов и анализом покрытия кода, что служит основой для принятия решения об успешности прохождения тестов.

Интеграция тестов (A5) осуществляется после успешного прохождения всех необходимых проверок. На этом этапе формируется PR в основной репозиторий и соответствующую ветку, содержащий новый набор сгенерированных автотестов. Перед созданием PR выполняется проверка, чтобы убедиться, что тесты соответствуют установленным критериям, включая покрытие кода и отсутствие ошибок

линтинга. После формирования PR разработчики уведомляются о его готовности для оценки и последующего слития, обеспечивая возможность интеграции новых тестов в проект.

Актуализация контекста в векторной БД (A6) (см. подраздел 2.4.3).

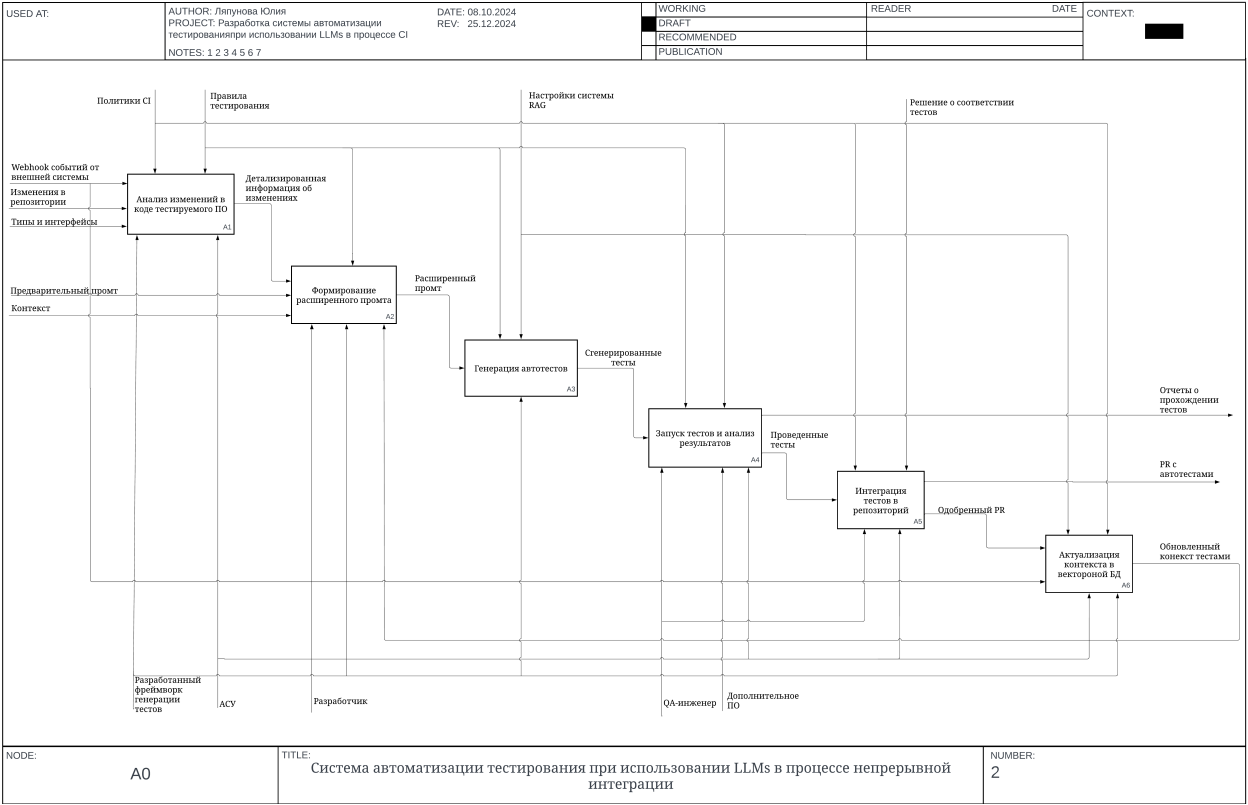


Рис.2.4. Детализированная IDEF0 уровня A0

Детализация управляющего действия «Правила CI», которое содержит регламенты, правила и политики, для конкретных функциональных блоков:

- 1. A1:
 - определен триггер для запуска CI процесса (автоматический запуск при создании или обновлении PR в ветку main), когда именно запускать анализ кода.
- 2. A4:
 - определен триггер для запуска тестового прогона (автоматический запуск при создании или обновлении PR в ветку, которая соответствует шаблону <ветка_разработчика>_tests);
 - правила для оценки состояния тестового окружения (необходимые зависимости, системные ресурсы, его конфигурация и подключение к внешним сервисам);

- критерии допустимого ограничение длительности выполнения тестового прогона и требование завершения всех тестов;
- регламент для обработки результатов тестового прогона (генерация отчетов в формате HTML, XML или других стандартизированных форматов, визуализация результатов во встроенных плагинах в CI-систему, отправка уведомлений о результатах выполнения тестов на email).

3. A5:

- определен триггер запуска процесса интеграции (создание и обновление PR с веткой, содержащей тесты, и успешное прохождение всех тестов в ветке);
- требования, которые должны быть выполнены для успешной интеграции (Покрытие кода не ниже установленного порога. Проверка, что изменения не снижают общее покрытие кода или качество. Сравнение с предыдущими результатами тестирования. Отсутствие ошибок линтинга и статического анализа. Обязательная проверка кода определенным количеством участников перед его одобрением.);
- политики автоматизации процесса слияния (автоматическое слияние PR при выполнении критериев и закрытие PR в случае невыполнения условий).

Детализация управляющего действия «Правила тестирования» (тест-политики, критерии, стандарты) для конкретных функциональных блоков:

1. A1:

- критерии выбора участков кода для тестирования, которые содержатся в правилах организации и определяют, что затронутые изменения касаются функционала API-маршрутов, реализованных с использованием конкретного фреймворка.

2. A2:

- руководство по использованию инструментов для написания автотестов.

3. A3:

- на основе установленных стандартов кодирования определяются правила написания тестов, обеспечивая их читаемость, поддерживаемость и согласованность (например, правила отступов,

- стандарты именования тестовых методов, классов и переменных, организации файлов и папок, шаблоны для написания тестов);
- согласно установленной политике покрытия кода определяются требования к уровню тестового покрытия, включая минимальные процентные показатели и виды покрытия (линейное и ветвлений). Для тестирования API-маршрутов выделяются целевые области: функциональное покрытие, безопасность, обработка ошибок, валидность данных, а также механизмы авторизации и аутентификации;
 - согласно руководству по уровню детализации определяются требования, насколько подробно должны быть написаны автотесты, включая количество проверок и глубину тестирования функционала;
 - на основе политики безопасности и конфиденциальности определяются правила, что автотесты не нарушают безопасность и конфиденциальность данных, особенно при работе с чувствительной информацией (ограничения на использование данных, меры по их защите и требования к их доступу).

Детализация управляющего действия «Правила системы RAG» для конкретных функциональных блоков:

1. А3:

- политика содержит регламенты по использованию LLM (так как она является частью системы RAG) с указанием лимитов токенов и настройки генерации (см. подраздел 2.4.4).

2. А6:

- регламент определяет процесс и условия актуализации данных в векторной БД, обеспечивая: релевантность данных для последующего использования, автоматизацию обновлений с минимальным участием человека, снижение шума и избыточности в базе.

2.4.3. IDEF0-диаграмма для функционального блока А6

Актуализация контекста в векторной БД является важным этапом работы системы автоматизированного тестирования и реализуется функциональным блоком А6. Этот процесс выполняется после успешного прохождения тестов и их слияния

в основной репозиторий. На основе результатов выполнения тестов извлекаются изменения из репозитория, которые разбиваются на семантически значимые фрагменты. Затем эти фрагменты преобразуются в векторные представления, добавляются в векторную БД. Этот процесс гарантирует, что векторная БД данных остается актуальной и релевантной, что играет ключевую роль в поддержании высокого качества и точности тестов, генерируемых на следующих этапах. Таким образом, актуализация контекста обеспечивает систему необходимыми данными для формирования точных промтов и успешной автоматизации тестирования.

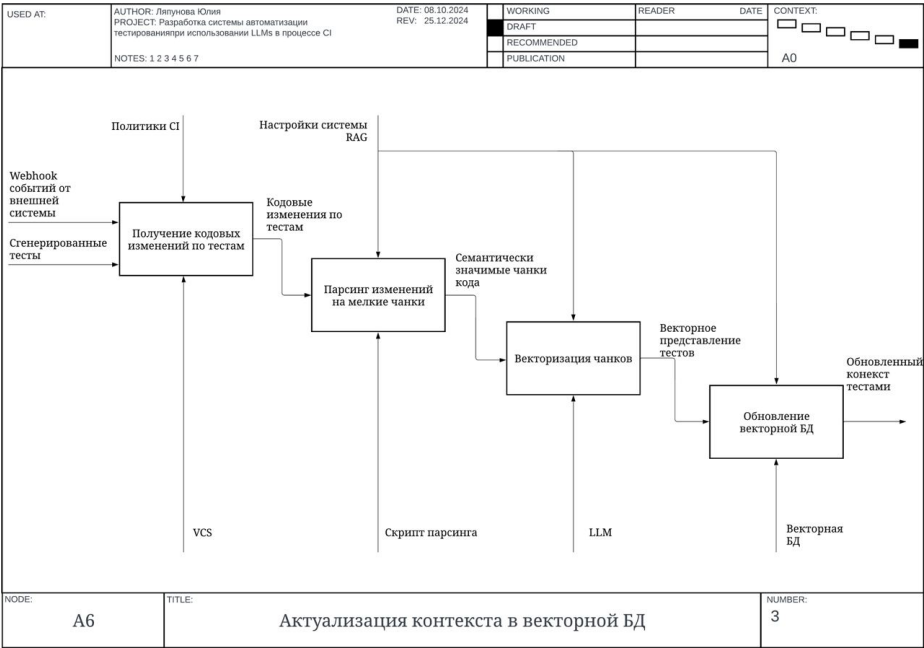


Рис.2.5. Детализированная IDEF0 уровня А6

- Процесс работы блока А6, представленного на рис.2.5:
- получение кодовых изменений по тестам (система через API VCS извлекает данные о кодовых изменениях, связанных с успешно выполненными тестами или новым функционалом);
 - парсинг изменений на мелкие чанки (извлеченные изменения разбиваются на семантически значимые фрагменты, такие как отдельные тесты, что упрощает их обработку);
 - векторизация чанков (полученные чанки передаются в LLM, которая преобразует их в векторные представления, пригодные для хранения и поиска в БД);
 - обновление векторной БД (новые векторные представления добавляются в БД и текущий контекст оптимизируется).

2.4.4. Регламенты использования LLM

В рамках функционального блока АЗ используются модели LLM для выполнения функций генерации тестов. Успешное выполнение задачи требует учета лимита токенов [40] и настраиваемых параметров генерации [55]. Эти элементы определяют, как модель взаимодействует с вводимыми данными и какой результат она выдает. Правильная настройка этих параметров не только улучшает производительность модели, но и позволяет достичь баланса между качеством, экономичностью и релевантностью сгенерированных тестов.

2.4.4.1. Лимиты токенов

Лимиты токенов — это ограничения на количество токенов (символов, слов или их частей), которые модель может обработать за один запрос. Каждый запрос к модели включает входной промт и генерируемый ответ. Основные причины для установки лимита токенов:

1. Управление ресурсами:

- ограничение вычислительных ресурсов — каждая операция с LLM использует вычислительные мощности, поэтому чем больше токенов в запросе или ответе, тем выше нагрузка на процессор, память и энергию. В этом случае установка лимитов помогает оптимизировать использование ресурсов, особенно при массовой обработке данных;
- снижение времени отклика — в силу того, что генерация ответа пропорциональна числу токенов, поэтому чем их больше, тем дольше модель обрабатывает запрос. В этом случае установка лимитов токенов уменьшают время генерации.

2. Управление длиной контекста для соблюдения предела модели — по причине того, что у каждой LLM есть ограничение на длину контекста, так как превышение этого лимита приведет к обрезке данных или ошибкам, лимиты токенов помогают оставаться в рамках возможностей модели.
3. Устранение избыточности информации — без лимитов запросы могут включать избыточные или нерелевантные данные, что снижает точность ответа, поэтому их установка способствует тщательному выбору и структурированию информации.

4. Поддержание качества генерации — LLM могут терять фокус или генерировать нерелевантный текст, если запрос слишком длинный или содержит избыточные детали, поэтому установка лимитов помогает модели концентрироваться на ключевой информации, улучшая качество ответов.
5. Стабильность и предсказуемость — лимиты токенов делают поведение модели более предсказуемым.

При установке лимитов токенов необходимо определить максимальный лимит токенов, разделить лимит между запросом и ответом, использовать фильтры и/или сжатие текста для формирования промта, а также при превышении лимита разбивать данные на последовательные части и обрабатывать их поэтапно.

В результате, установка лимитов токенов для LLM в автоматизированной системе генерации тестов является инструментом для управления ресурсами и обеспечения стабильной работы системы из-за контроля затрат, оптимизации производительности, сохранения качества и точности и предотвращения ошибок при работе с моделью.

2.4.4.2. Параметры генерации

Параметры генерации — это настройки, которые контролируют, как языковая модель генерирует текстовые ответы. Эти параметры помогают настроить поведение модели для достижения баланса между разнообразием, предсказуемостью и качеством выходного текста. Рассмотрим основные параметры генерации [55]:

1. Температура — контролирует степень случайности в выборе следующего слова или токена, значение которого варьируется от 0 до 1. Низкие значения (0.1–0.3) гарантируют более детерминированный и предсказуемый текст, а также подходят для задач, требующих точности (например, генерация кода или автотестов). Высокие значения (0.7–1.0) способствуют генерации более творческого и разнообразного текста, поэтому используются для создания рассказов или креативного контента.
2. Тор-К — ограничивает выбор следующего слова к k наиболее вероятным токенам, а также убирает из рассмотрения менее вероятные варианты, чтобы модель не генерировала слишком случайный текст. При низких значениях (например, $K=5$) модель выбирает из небольшого набора самых

вероятных слов, что делает текст более строгим и целенаправленным. При высоких значениях (например, $K=50$) модель расширяет набор слов, добавляя разнообразие.

3. Тор–Р — модель выбирает слова из набора, где сумма вероятностей токенов достигает заданного порога p . Более гибкий параметр, чем Тор–К, так как не фиксирует размер набора токенов. При низких значениях (например, $P=0.1$) модель фокусируется только на высоко вероятных токенах. При высоких значениях (например, $P=0.9$) модель добавляет разнообразие, выбирая из широкой вероятностной области.

2.5. Проектирование архитектуры системы

Верхнеуровневая архитектура разработанной системы генерации автотестов представлена на рис.2.6. На вход система получает код из репозитория, который содержит программные файлы проекта. Сама система включает три компонента: разработанный фреймворк генерации тестов, включающий базу знаний и LLM, Jenkins и GitHub. В результате работы системы формируются сгенерированные тесты, которые могут быть использованы для автоматизированного тестирования кода.

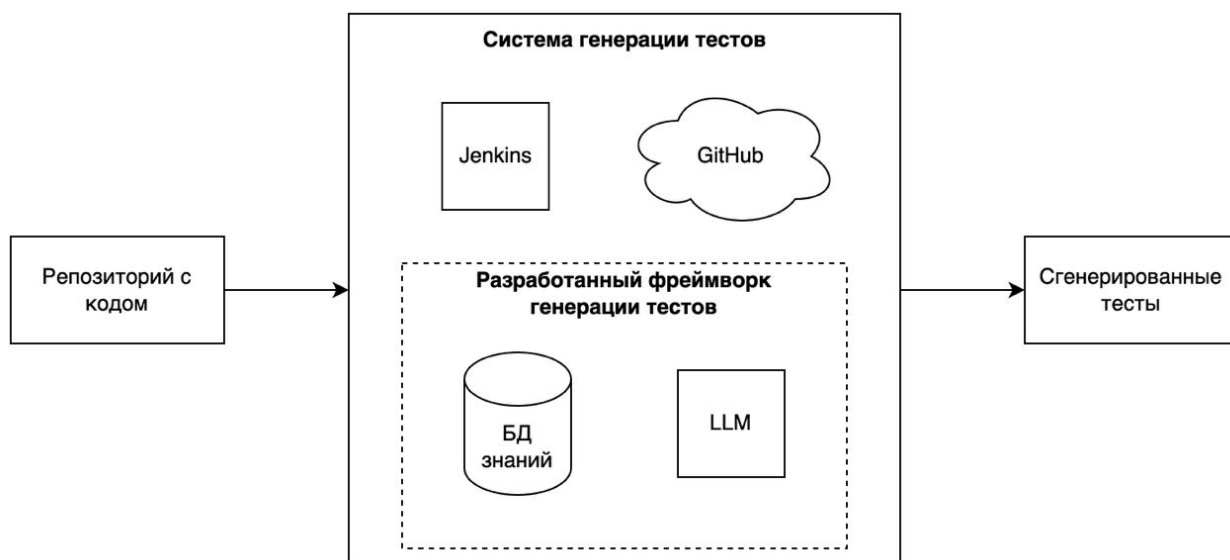


Рис.2.6. Общая архитектура приложения

В система поддерживает два действия:

- генерация тестов (рис.2.7);
- пополнение векторной БД (рис.2.8).

2.5.1. Проектирование процесса генерации тестов

В данном разделе воспользуемся UML диаграммой взаимодействия для спецификации процесса генерации тестов.

Первое действие системы инициируется с помощью вебхука от внешней системы, который срабатывает при создании нового PR в репозитории основного ПО. Этот подход позволяет оперативно реагировать на события, минимизируя задержки между изменениями в коде и началом тестирования. Вебхук отправляет HTTP-запрос на проектируемую систему автоматизированного тестирования. В системе создан отдельный API-маршрут для получения информации о PR.

После получения данных о PR, система отправляет событие в брокер сообщений. Использование брокера обусловлено необходимостью асинхронной обработки событий. Это решение необходимо, так как время обработки запроса (включая генерацию автотестов) превышает лимиты HTTP-соединения. Кроме того, брокер сообщений обеспечивает надежность, так как каждое событие сохраняется в очереди до момента обработки. Данный подход также упрощает масштабирование системы, позволяя добавлять новых worker-узлов для повышения производительности.

Далее worker, подключенный к очереди сообщений, обрабатывает событие. Worker-архитектура обеспечивает распределение нагрузки и возможность обработки нескольких PR одновременно. Это решение позволяет изолировать бизнес-логику от начального этапа получения событий, повышая модульность и устойчивость системы. После чтения очереди worker взаимодействует с GitService, который извлекает изменения, связанные с PR, используя API системы контроля версий. GitService служит промежуточным слоем, абстрагирующим взаимодействие со VCS, что упрощает поддержку и повторное использование кода.

Получив данные о кодовых изменениях, worker обращается к KnowledgeBaseService, состоящий из векторной и объектной БД. Этот сервис предоставляет дополнительный контекст, включая информацию о ранее сгенерированных тестах и документации, хранящихся объектной БД. Этот контекст используется для формирования расширенного промта для LLM. Использование KnowledgeBaseService позволяет исключить необходимость дублирования данных в каждом запросе, упрощая управление и поддержание их актуальности. Векторная БД хранит только векторы и метаданные и используется как высокоэффективное хранилище для семантических векторных представлений,

которая интегрируется с объектной базой для извлечения связанных полных данных. Использование векторной БД обусловлено ее способностью эффективно индексировать и извлекать данные.

Затем worker передает сформированный промт в TestGenerationService. TestGenerationService обеспечивает взаимодействие с LLM, реализуя бизнес-логику генерации тестов и обеспечивая независимость остальных компонентов системы от деталей работы LLM. Это обеспечивает гибкость системы, позволяя обновлять или заменять языковую модель без значительных изменений в архитектуре. В этом сервисе выполняется генерация новых тестов или обновление существующих с использованием языковой модели.

На последнем этапе worker фиксирует полученные тесты в репозиторий, снова используя GitService. Это завершает цикл обработки, обеспечивая автоматическую интеграцию новых тестов в тестовый репозиторий.

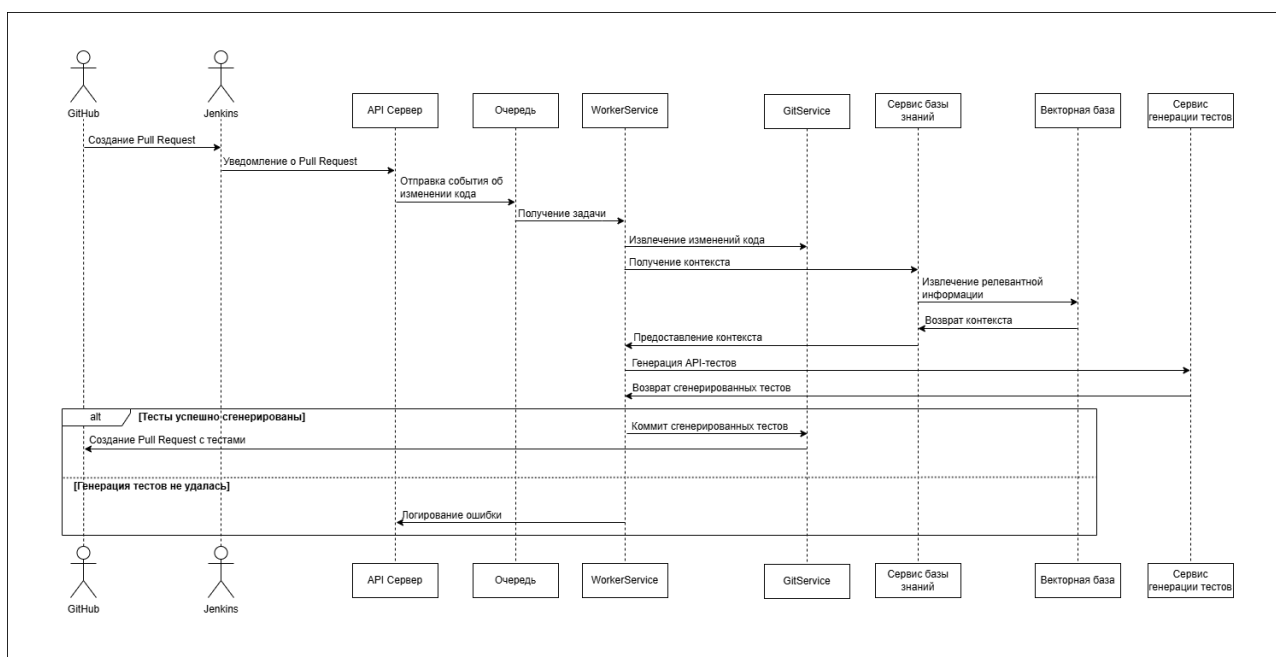


Рис.2.7. Генерация тестов

2.5.2. Проектирование процесса актуализации базы знаний

В данном разделе воспользуемся UML диаграммой взаимодействия для спецификации процесса актуализации БЗ.

После закрытия созданного PR в тестовом репозитории запускается действие 2. Для этого в системе разработан отдельный API-маршрут, который обрабатывает событие закрытия PR. API-маршрут получает информацию о слиянии в тестовом репозитории и отправляет данные в очередь, связанную с обновлением БЗ

(KnowledgeBase). В данном действии использование очереди обусловлено также необходимостью асинхронной обработки событий.

После попадания данных в очередь worker, ответственный за работу с тестовым репозиторием, извлекает событие из очереди и начинает его обработку. Worker взаимодействует с GitService для получения данных о новых тестах, добавленных в результате слияния.

Извлеченные данные затем передаются в KnowledgeBaseService. KnowledgeBase отправляет новые тестовые сценарии в векторную БД (VectorDB).

Выбор события закрытия PR в качестве триггера обусловлен его естественной ролью в процессе интеграции. Этот момент фиксирует, что тесты, добавленные в результате слияния, окончательно готовы к использованию. Таким образом, вся архитектура направлена на обеспечение надежности, гибкости и масштабируемости системы.

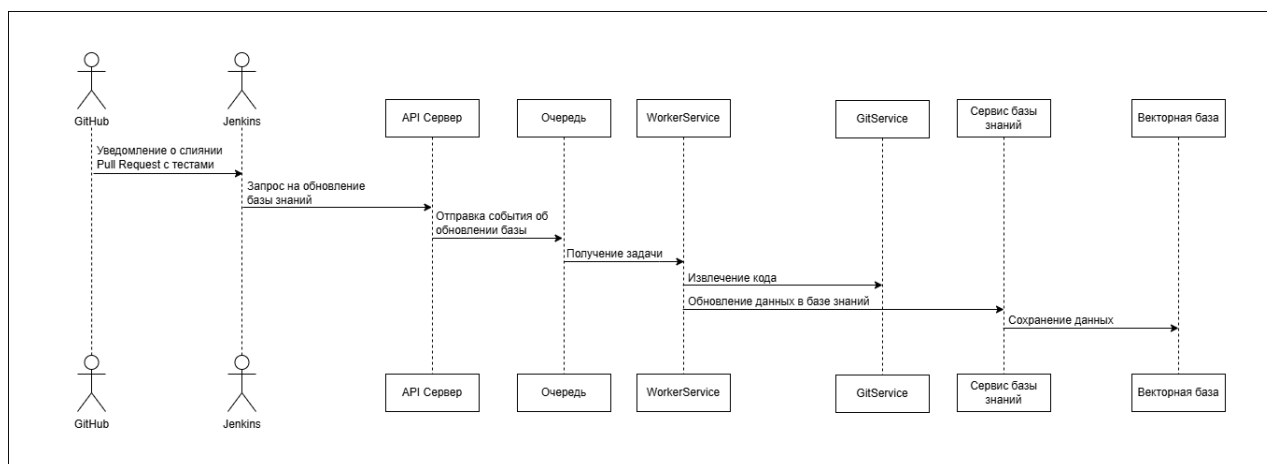


Рис.2.8. Пополнение векторной БД

Описанные два события в разработанной системе автоматизации тестирования построены на основе модульной и асинхронной архитектуры, обеспечивающей надежность, масштабируемость и адаптивность процессов генерации и актуализации тестов. Выбранный подход с использованием брокера сообщений, worker-узлов, KnowledgeBaseService и векторной БД позволяет эффективно управлять большими объемами данных и минимизировать задержки при обработке событий. Такая структура гарантирует актуальность тестов, повышает их качество и способствует автоматизации процессов тестирования в рамках непрерывной интеграции.

2.5.3. Особенности проектирования системы

Проектирование средства с концепцией RAG для генерации автотестов требует учета специфики работы с векторной БД и LLM. Система должна эффективно комбинировать извлечение релевантной информации из БД и генерацию на основе полученных данных. Необходимо учитывать ключевые особенности использования:

А. Пустая векторная БД + LLM обученная на программном коде.

1. Особенности использования:

- а) генерация автотестов не зависит от предварительного наполнения базы, что позволяет быстро начать использование системы;
- б) сниженные требования к вычислительным ресурсам, так как нет затрат на работу с БД;
- в) экономия времени на этапах извлечения данных из векторной БД;
- г) LLM без поддержки векторной БД полагается только на свои внутренние знания, которые могут быть устаревшими или неполными;
- д) возможны пропуски в покрытии специфических сценариев или особенностей кода, так как эта информация отсутствует в базе;
- е) генерация становится менее эффективной при высоких требованиях к покрытию автотестами.

В. Обогащенная векторная БД + LLM необученная на программном коде.

1. Особенности использования:

- а) БД компенсирует недостаток знаний LLM о программном коде, предоставляя релевантную информацию, позволяя системе генерировать автотесты с учетом конкретного контекста проекта;
- б) БД может быть наполнена данными, соответствующими специфике проекта, что делает систему универсальной и адаптируемой под различные домены;
- в) отсутствие необходимости использовать мощные модели LLM, так как большая часть нагрузки по предо-

ставлению информации ложится на векторную БД, что может быть выгодно при ограниченных вычислительных мощностях;

- d) даже при отсутствии глубоких знаний о программировании, LLM способна интерпретировать извлеченные данные и генерировать автотесты на основе них;
- e) способность LLM корректно интерпретировать сложные структуры, синтаксис и логику программ может быть ограничена;
- f) релевантность и полнота автотестов напрямую зависят от качества и объема информации в векторной БД;
- g) требуется создание и поддержание обогащенной векторной БД, включая сбор, обработку и обновление данных.

С. Обогащенная векторная БД + LLM с параметрами генерации, позволяющими вариативность.

1. Особенности использования:

- a) благодаря вариативности генерации, система способна предлагать необычные тестовые сценарии, которые могли бы быть упущены при использовании более строгих параметров;
- b) вариативные параметры позволяют компенсировать возможные пробелы или неясности в данных векторной БД за счет предложений разных интерпретаций;
- c) вариативность позволяет генерировать тесты, покрывающие разные сценарии использования, включая редкие или специфические;
- d) повышенная вариативность может привести к генерации избыточного количества автотестов, многие из которых будут повторяться или мало отличаться друг от друга;
- e) высокая вариативность генерации требует большего времени и вычислительных мощностей;

- f) если данные в векторной БД недостаточно релевантные и полные, вариативность может привести к созданию множества малополезных тестов;
- g) для автоматизации процессов в CI вариативные тесты могут быть трудными для проверки на стабильность и пригодность.

D. Обогащенная векторная БД + LLM с детерминированными параметрами генерации.

1. Особенности использования:

- a) генерация автотестов становится стабильной и предсказуемой, так как результаты одинаковых запросов будут идентичны;
- b) детерминированные параметры способствуют созданию строго логичных и целенаправленных автотестов, минимизируя вероятность ошибок;
- c) стабильность генерации делает этот подход особенно подходящим для использования в CI/CD;
- d) отсутствие необходимости в генерации множества вариантов сокращает вычислительные затраты;
- e) четкая структура сгенерированных тестов облегчает их анализ и проверку;
- f) детерминированные параметры могут ограничивать креативность и вариативность автотестов, что снижает вероятность обнаружения неочевидных ошибок;
- g) если векторная БД содержит неполные или нерелевантные данные, LLM будет генерировать тесты с ограниченной ценностью;
- h) детерминированная генерация затрудняет создание тестов для сценариев, которые выходят за рамки шаблонов, заложенных в векторной БД.

Системы для генерации автотестов должна обеспечивает способность корректно и эффективно генерировать автотесты, учитывая особенности тестируемого кода, структуру и содержание векторной БД, а также параметры генерации. Данные особенности необходимо учесть при тестирование.

2.6. Выводы по главе 2

В ходе главы выполнено проектирование системы автоматизированного тестирования, использующей LLM, в процессе непрерывной интеграции. Для нивелирования ограничений LLM, таких как ограниченность контекста и статичность знаний, выбрана концепция RAG, позволяющая обеспечить доступ к актуальной БЗ и формировать контекстуально релевантные тесты. Основой для реализации RAG послужил подход, описанный в исследовании «Dense Passage Retrieval for Open-Domain Question Answering» [9].

Общие требования к системе включали автоматическую генерацию тестов, актуализацию БЗ, работу с контекстом, интеграцию с CI и поддержку модульной структуры. Эти требования стали основой для проектирования архитектуры, отвечающей современным стандартам качества и адаптивности. Для подтверждения применимости системы описаны случаи использования с точки зрения разработчиков и QA-инженеров.

На основании требований предложена общая архитектура системы, построенная на модульной и асинхронной основе. Она включает брокера сообщений, worker-узлы и векторную БД, что позволяет эффективно управлять большими объемами данных и минимизировать задержки при обработке событий. Интеграция с CI-пайплайном обеспечивает добавление сгенерированных тестов в репозиторий и их автоматическое выполнение.

Далее проведено функциональное проектирование системы, в рамках которого выделены две ключевые функции: генерация тестов и актуализация БЗ. Использование RAG позволяет актуализировать базу знаний после каждого цикла тестирования, что делает систему более гибкой и адаптивной. Применение асинхронной обработки и worker-архитектуры обеспечивает надежность и устойчивость к высоким нагрузкам, а модульная структура упрощает масштабирование и адаптацию под изменяющиеся требования.

Далее перейдем к выбору стека технологий для реализации поставленной цели.

ГЛАВА 3. РЕАЛИЗАЦИЯ СИСТЕМЫ АВТОМАТИЗИРОВАННОГО ТЕСТИРОВАНИЯ API

В данной главе описывается процесс реализации системы автоматизированного тестирования. В параграфе 3.1 подробно рассмотрена оркестрация разрабатываемой системы, включая преднастройку процесса оркестрации и разработку пайплайна. В параграфе 3.2 представлена диаграмма классов, иллюстрирующая архитектурное устройство системы. Параграф 3.3 посвящен функционалу извлечения данных из PR, помимо того в параграфе 3.4 рассматривается парсинг этих данных. В параграфе 3.5 описано формирование расширенного промта для обеспечения корректной работы системы.

3.1. Процесс оркестрации разрабатываемой системы

Оркестрация играет ключевую роль в разрабатываемой системе автоматизации тестирования, обеспечивая согласованное взаимодействие между ее компонентами. Для реализации этой функции выбран Jenkins в качестве CI-инструмента, прежде всего благодаря его ключевому преимуществу — бесплатности и открытому исходному коду. Это открытый проект с большим сообществом разработчиков и обширной базой готовых решений, что значительно упрощает его внедрение и дальнейшее сопровождение.

Выбор Jenkins как инструмента оркестрации позволяет эффективно достичь целей разрабатываемой системы автоматизации тестирования. Эти цели достигаются за счет следующих возможностей Jenkins:

- обработка изменений в репозитории за счет интеграции с VCS через плагины и поддержка вебхуков для автоматического запуска пайплайнов;
- генерация и запуск автотестов за счет возможности выполнения скриптов и внешних API запросов для передачи метаданных в функциональную часть системы автоматизации, а также интеграция с инструментами тестирования (JUnit, TestNG и др.) для выполнения автотестов;
- проверка и подготовка тестового окружения при выполнении скриптов для проверки состояния инфраструктуры, а также возможность настройки развертывание тестового окружения в динамической среде;

- отчетность и уведомления при использовании плагинов для отправки уведомлений на email и генерация отчетов о прохождении тестов с использованием Allure Report;
- масштабируемость и управление нагрузкой в зависимости от объема изменений и количества тестов при поддержке распределенной архитектуры Jenkins (мастер-агенты) для выполнения тестов на нескольких серверах за счет подключения дополнительных агентов или оптимизация использования ресурсов при выделении агентов для выполнения конкретных задач, а также эффективного распределения задач между агентами при одновременной обработке множества вебхуков.

3.1.1. Преднастройка процесса оркестрации

Оркестрация CI процесса реализована с использованием пайплайна в Jenkins. Для автоматического запуска пайплайна настроен вебхук в GitHub для репозитория разрабатываемого ПО. В настройках вебхука указываются следующие параметры:

- payload URL для направления запросов на настроенный сервер Jenkins;
- content type для передачи метаданных, связанные с событием создания PR, в формате JSON (эти данные используются в пайплайне для формирования HTTP-запроса к разработанному API-маршруту системы, предназначенному для автоматической генерации тестов);
- secret для верификации запросов и обеспечения их безопасности (данный ключ также настроен в плагине Jenkins);
- событие типа Pull requests, чтобы вебхук срабатывал только на действия, связанные с созданием PR.

После настройки вебхука развернут экземпляр Jenkins на отдельном сервере. Для корректной работы установлены следующие плагины:

1. Generic Webhook Trigger — для обработки вебхуков, отправленных из GitHub.
2. GitHub Integration Plugin — для интеграции с GitHub и автоматической работы с репозиториями.
3. HTTP Request Plugin — для отправки HTTP-запросов на внешние сервисы.

После установки плагинов в настройках Jenkins добавлен shared secret в разделе GitHub, который соответствует значению секретного ключа, указанного в

вебхуке. Это обеспечивает верификацию запросов, поступающих от вебхука, и гарантирует, что они отправлены именно из настроенного репозитория.

Далее создан Jenkins Pipeline, в настройках которого добавлен триггер Generic Webhook Trigger. Этот триггер предназначен для обработки входящих вебхуков от внешних систем, таких как GitHub, и запуска пайплайна в Jenkins. Generic Webhook Trigger выбран, поскольку он предоставляет гибкие возможности для настройки, включая извлечение данных из JSON-пейлоада вебхука, настройку параметров и фильтров. Это позволяет запускать пайплайн только для определенных событий, например, при создании PR в заданную ветку, что упрощает интеграцию и автоматизацию процессов. В триггере настроены параметры, извлекаемые из данных POST-запроса, которые отправлены вебхуком, таких как:

- BRANCH (JSONPath: \$.pull_request.base.ref) — ветка, в которую происходит merge;
- ACTION (JSONPath: \$.action) — действие с pull request (например, "opened");
- PULL_REQUEST_URL (JSONPath: \$.pull_request.url) — URL PR;
- PULL_REQUEST_ID (JSONPath: \$.number) — уникальный идентификатор PR;
- PULL_REQUEST_TITLE (JSONPath: \$.pull_request.title) — заголовок PR;
- REPO_ID (JSONPath: \$.repository.id) — идентификатор для клонирования репозитория;
- REPO_NAME (JSONPath: \$.repository.full_name) — полное имя для клонирования репозитория;
- REPO_CLONE_URL (JSONPath: \$.repository.clone_url) — URL для клонирования репозитория.

Также настроен дополнительный фильтр с параметрами: `main:opened$ = $BRANCH:$ACTION`. Этот механизм фильтрации, который используется для проверки соответствия значений определенным условиям. В данном случае он позволяет запускать пайплайн только при выполнении условий, когда действие — это создание PR, а целевая ветка — «main». Это позволяет избежать лишних запусков, если событие не соответствует заданным условиям.

3.1.2. Разработка пайплайна

В рамках пайплайна реализован скрипт, состоящий из следующих этапов:

1. Отправка данных вебхука в систему — на этом этапе формируется тело HTTP-запроса в формате JSON. В него включены метаданные, извлеченные из вебхука, описанные ранее. Эти данные передаются в разработанную систему, чтобы она могла использовать их для получения всех изменений, внесенных в рамках созданного PR, и работы с ними на следующих итерациях. Затем выполняется запрос `POST /webhook/generate-tests`. После отправки запроса выполняется проверка статуса ответа. Если статус не равен 200, пайплайн завершает выполнение с сообщением об ошибке, включающим код статуса и тело ответа. В случае успешного выполнения пайплайна переходит на следующий этап.
2. Запуск проверки тестового окружения.
3. Запуск тестового прогона.

3.2. Диаграмма классов

Ниже рис.3.1 представлена диаграмма классов, отражающая ключевые компоненты системы и их взаимосвязи. Общая логика приложения сводится к обработке входящих вебхуков, генерации тестов на основе поступающих изменений и обновлению внутренней базы знаний. Для этого используются очереди событий, сервисы для работы с внешними репозиториями и LLM, а также сервисы, отвечающие за формирование и хранение векторных представлений (далее — эмбеддингов) и управление контекстом.

`WebhookController` и `QueueListener` обрабатывают первичные входящие сигналы (события о необходимости генерации тестов и обновления БД). Далее, в соответствии с типом события, логика распределяется между рабочими компонентами: `TestGenerationWorker` отвечает за формирование тестов, а `KnowledgeUpdatingWorker` — за обновление БД.

Важную роль играет `RAGService`, который формирует промты для запроса к языковой модели, извлекает релевантные данные из внутреннего контекста и векторных хранилищ, а затем генерирует итоговые тесты. Сервис `GithubService` (см. подраздел 3.3) взаимодействует с удаленными репозиториями, получая актуальные изменения из `pull`-запросов и коммитов. Для извлечения детальной информации об измененных фрагментах кода используются `DiffParser` и `PatchParser`, а классы `EndpointMethodLocator` и `ModelCollector` (см. подраздел 3.4) помогают найти и описать соответствующие методы и модели в кодовой базе.

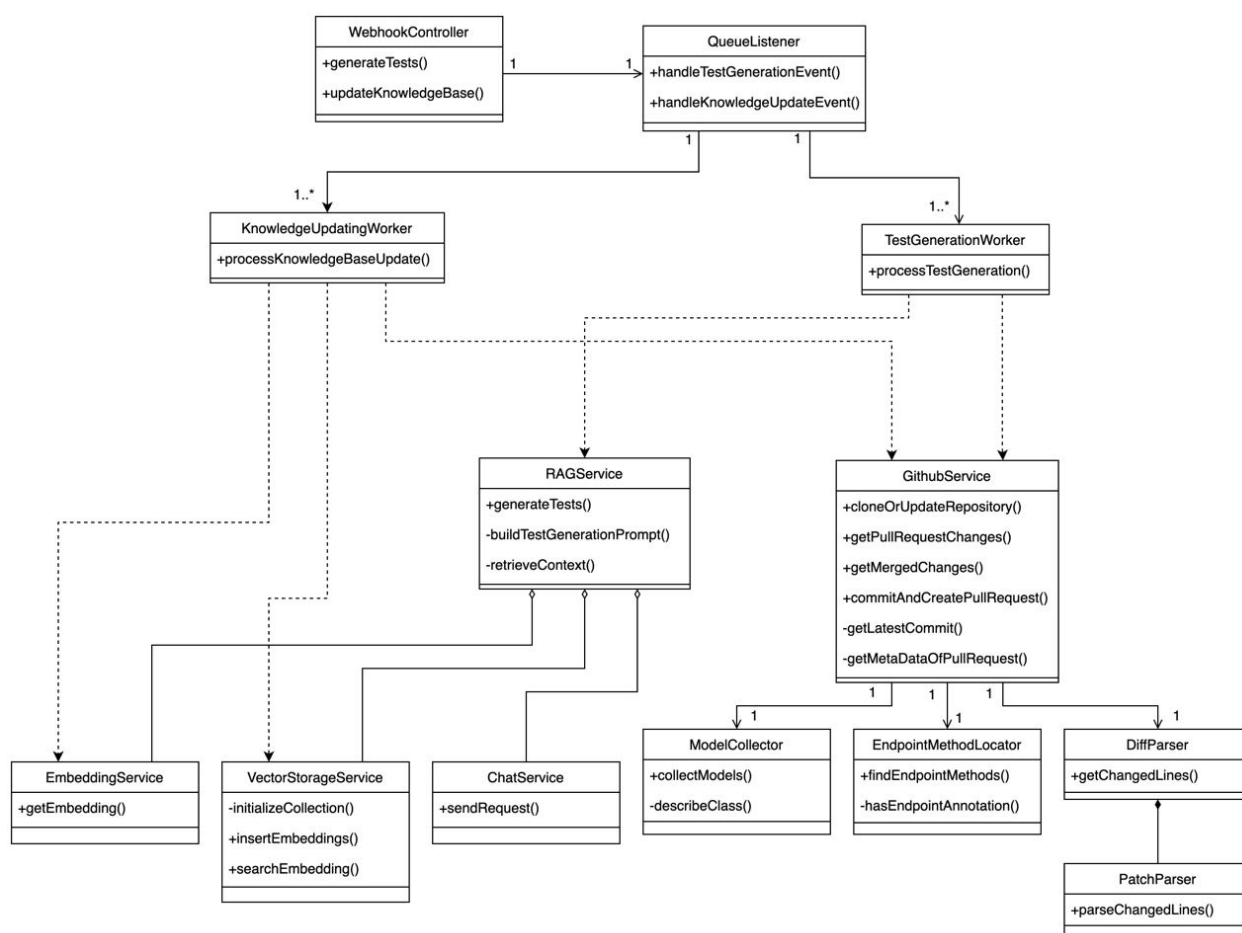


Рис.3.1. Диаграмма классов

Для эффективной работы с эмбедингами предусмотрен `EmbeddingService`, который формирует векторные представления текстовых данных. Затем эти векторы сохраняются или ищутся в `VectorStorageService`, использующем векторное хранилище (см. подраздел 3.5). Наконец, класс `ChatService` обеспечивает взаимодействие с языковой моделью, обрабатывая сформированные промты и получая ответы.

Таким образом, диаграмма иллюстрирует, как различные компоненты — от контроллеров и очередей до сервисов по генерации тестов и работе с эмбедингами — взаимодействуют между собой, образуя единую систему для автоматизации процессов анализа изменений кода, генерации тестов и обновления знаний.

3.2.1. Класс *WebhookController*

Точкой входа для клиентов в разработанной системе является класс `WebhookController`. Данный класс предоставляет два основных публичных метода: `generateTest()` и `uploadKnowledgeFile()`, которые выступают в роли обработчиков маршрутов `POST /generate-tests` и `POST /upload-knowledge-`

file соответственно. Помимо этого, класс содержит свойство `rabbitTemplate`, представляющее собой экземпляр клиента брокера сообщений RabbitMQ, используемого для отправки данных в соответствующие очереди.

Метод `generateTest()` принимает на вход аргумент типа `WebhookPayload`, который содержит информацию о созданном PR. В рамках его реализации осуществляется валидация входных данных для обеспечения их корректности. После успешной проверки аргументы передаются в очередь `test-generation-queue` брокера сообщений RabbitMQ с использованием объекта `rabbitTemplate`. Таким образом, данный метод отвечает за передачу запросов на генерацию тестов в соответствующий компонент системы.

Метод `uploadKnowledgeFile()` принимает три аргумента: загружаемый файл, имя файла и тип файла (возможные значения: `TESTS` или `DOCUMENTATION`). После выполнения валидации входных данных, информация о файле отправляется в очередь `knowledge-base-update-queue` брокера сообщений RabbitMQ с использованием объекта `rabbitTemplate`. Этот метод предназначен для обновления БЗ системы, включая загрузку тестов и документации.

Таким образом, класс реализует интерфейс взаимодействия с системой, обеспечивая обработку входящих запросов и их интеграцию с системой обмена сообщениями — RabbitMQ.

3.2.2. Класс *RabbitMQConfig*

Настройки очередей определяются в классе *RabbitMQConfig*. Этот класс обеспечивает создание четырех очередей: `test-generation-queue` и `knowledge-base-update-queue`, а также `test-generation-dlq` и `knowledge-base-update-dlq`. Очереди `dead letter queue` (далее — DLQ) предназначены для хранения сообщений, обработка которых завершилась с ошибкой в процессе генерации тестов или обновления БЗ, соответственно.

Создание указанных очередей осуществляется в методах `testGenerationQueue()`, `knowledgeBaseUpdateQueue()`, `testGenerationDLQ()` и `knowledgeBaseUpdateDLQ()`. Кроме того, класс содержит метод `deadLetterExchange()`, в котором создается обменник⁵, предназначенный для обработки неудачных сооб-

⁵Представляет собой компонент, который принимает входящие сообщения и направляет их в одну или несколько очередей на основе ключей маршрутизации и настроек привязки. Различают несколько типов обменников, каждый из которых определяет способ доставки сообщений. В данном случае исполь-

щений. Также выполняется настройка привязки очередей DLQ к этому обменнику с использованием определенных ключей маршрутизации.

Для обеспечения корректной работы слушателей сообщений и определения их содержимого, в *RabbitMQConfig* подключен конвертер *Jackson2JsonMessageConverter*. Этот конвертер автоматически преобразует входящие сообщения в формат JSON, что упрощает их обработку в системе.

3.2.3. Класс *QueueListener*

Регистрация слушателей сообщений осуществляется в классе *QueueListener*. В данном классе определено четыре зарегистрированных слушателя, каждый из которых соответствует одной из очередей. Реализация слушателей выполняется в методах: *handleTestGenerationEvent()*, *handleKnowledgeUpdateEvent()*, *handleTestGenerationDLQ()* и *handleKnowledgeBaseUpdateDLQ()*. Кроме того, класс содержит два свойства: *testGenerationWorker* и *knowledgeWorker*, представляющие собой экземпляры классов, содержащих основную бизнес-логику приложения.

Метод *handleTestGenerationEvent()* отвечает за обработку сообщений из очереди *test-generation-queue*. Он выполняет валидацию полученного сообщения и передает его в обработчик *testGenerationWorker* для дальнейшей обработки.

Метод *handleKnowledgeUpdateEvent()* слушает очередь *knowledge-base-update-queue*. В рамках его работы осуществляется валидация полученного сообщения и передача его в обработчик *knowledgeWorker*, где выполняется основная логика обновления данных.

Методы *handleTestGenerationDLQ()* и *handleKnowledgeBaseUpdateDLQ()* предназначены для обработки сообщений, перенаправленных в очереди (*test-generation-dlq* и *knowledge-base-update-dlq*, соответственно, в случае возникновения ошибок в процессе их первоначальной обработки).

Таким образом, класс реализует обработку сообщений из очередей *RabbitMQ*, выполняя их предварительную валидацию и передачу в специализированные компоненты системы для дальнейшей обработки.

зуется обменник типа *dead-letter exchange*, который служит для обработки сообщений, не обработанных основными очередями.

3.2.4. Класс *TestGenerationWorker*

Класс *TestGenerationWorker* выполняет функции обработчика событий, связанных с генерацией тестов. В его составе определены два ключевых свойства: `githubService` и `ragService`.

Свойство `githubService` используется для взаимодействия с API GitHub, включая операции по созданию и управлению PR. Свойство `ragService` отвечает за реализацию логики работы с БД, а также за формирование запросов к модели LLM для генерации тестов.

Основная функциональность класса сосредоточена в методе `processTestGeneration()`. Этот метод анализирует изменения, внесенные в PR, определяет модифицированные API-маршруты и автоматически генерирует тесты для каждого из них. Сгенерированные тесты затем оформляются в виде отдельного PR, который создается в GitHub.

Таким образом, класс выполняет роль в автоматизации процессов генерации тестов, интеграции с GitHub и взаимодействии с БЗ системы.

3.2.5. Класс *KnowledgeWorker*

Класс *KnowledgeWorker* выполняет функции обработчика событий, связанных с обновлением БЗ системы. Он содержит два свойства: `embeddingService` и `vectorStorage`.

Свойство `embeddingService` отвечает за создание эмбеддингов содержимого загружаемых файлов. Эти эмбеддинги используются для представления данных в формате, пригодном для последующего хранения и поиска в векторной БД. Свойство `vectorStorage` обеспечивает взаимодействие с векторной БД, выполняя операции по сохранению и извлечению данных.

Функциональность класса дополнена двумя приватными методами: `processCodeContent()` и `processDocContent()`, которые предназначены для обработки файлов различных типов. Метод `processCodeContent()` выполняет обработку файлов с исходным кодом, в то время как метод `processDocContent()` обрабатывает документацию.

Основной публичный метод класса, `processKnowledgeBaseUpdate()`, вызывает соответствующий метод обработки файла (`processCodeContent()` или `processDocContent()`), в зависимости от типа загружаемого файла. После обра-

ботки содержимого данный метод выполняет вставку записи в векторную БД с использованием свойства `vectorStorage`.

Таким образом, класс играет роль в процессе обновления БЗ, обеспечивая преобразование содержимого файлов в эмбединги, их корректную обработку и последующее хранение в векторной БД.

3.2.6. Класс *RAGService*

Класс *RAGService* реализует функциональность, связанную с генерацией тестов, интегрируя работу с векторной БД и моделью LLM. Он содержит три ключевых свойства: `embeddingService`, `vectorStorage` и `chatService`.

Свойства `embeddingService` и `vectorStorage` выполняют те же функции, что и в других компонентах системы: создание эмбедингов для представления данных и взаимодействие с векторной БД. Свойство `chatService` отвечает за отправку запросов в LLM и получение результатов.

Функциональность класса дополнена двумя приватными методами:

- `retrieveContext()`: извлекает релевантный контекст из векторной БД, обеспечивая выборку данных, необходимых для генерации тестов;
- `buildTestGenerationPrompt()`: формирует запрос (*prompt*) для LLM на основе извлеченного контекста.

Основной публичный метод класса, `generateTests()`, объединяет все этапы процесса генерации тестов. Он вызывает приватный метод `retrieveContext()` для извлечения контекста, использует метод `buildTestGenerationPrompt()` для формирования запроса, а затем отправляет полученный запрос в LLM с помощью свойства `chatService`. После получения ответа от LLM метод возвращает сгенерированные тесты.

Таким образом, класс играет центральную роль в автоматизации процесса генерации тестов, обеспечивая эффективное взаимодействие с векторной БД и использованием возможностей LLM.

3.2.7. Класс *GitHubService*

Класс *GitHubService* реализует взаимодействие с платформой GitHub и обеспечивает анализ изменений в PR. Он содержит свойство `github`, представляющее собой клиент для работы с API GitHub, выполняя операции, связанные с получением информации о репозиториях, PR и их содержимом.

Основная функциональность класса реализована через несколько ключевых методов:

- `analyzePullRequestChanges()`: выполняет последовательность операций, включающую получение PR (`getPullRequest()`), клонирование репозитория (`cloneOrUpdateRepository()`) и анализ изменений, внесенных в PR (`analyzeChanges()`);
- `analyzeChanges()`: вычисляет разницу между основной веткой репозитория и веткой PR, после чего передает эту разницу в метод `processChangedFiles()`;
- `processChangedFiles()`: выполняет сбор моделей, используемых в репозитории, с помощью компонента `ModelCollector`, а также извлекает детали каждого измененного эндпоинта для контекста, используя метод `processControllerFile()`;
- `processControllerFile()`: анализирует измененные строки кода внутри файла, используя `DiffParser`, выполняет разбор структуры кода с помощью `StaticJavaParser`, определяет метод эндпоинта с использованием `EndpointMethodLocator`, а затем добавляет полученные данные в общий контейнер (`Map`) эндпоинтов.

Таким образом, класс используется обработки изменений, происходящих в рамках PR, обеспечивая анализ модификаций кода, идентификацию измененных API-маршрутов и подготовку данных для дальнейшей автоматической генерации тестов.

3.2.8. Класс *ModelCollector*

Класс *ModelCollector* предназначен для сбора информации о моделях, используемых в репозитории. Он содержит два ключевых метода: приватный метод `describeClass()` и публичный метод `collectModels()`.

Приватный метод `describeClass()` выполняет извлечение и формирование основных характеристик класса, необходимых для дальнейшего анализа и использования в контексте системы.

Публичный метод `collectModels()` осуществляет обработку файлов, содержащих модели. Для этого он выполняет разбор каждого файла с помощью библиотеки `StaticJavaParser`, после чего вызывает метод `describeClass()` с

соответствующими аргументами для получения детализированной информации о классе.

Таким образом, класс играет роль в анализе моделей, обеспечивая их структурированное представление для последующего использования в системе.

3.2.9. Класс *EndpointMethodLocator*

Класс *EndpointMethodLocator* предназначен для идентификации методов, обрабатывающих API-маршруты в приложении, разработанном с использованием Spring Boot. Он включает два ключевых метода: приватный метод `hasEndpointAnnotation()` и публичный метод `findEndpointMethods()`.

Приватный метод `hasEndpointAnnotation()` выполняет проверку наличия у метода аннотаций, характерных для обработки HTTP-запросов в *Spring Boot*. Данный метод используется для определения принадлежности метода к обработчику API-маршрутов.

Публичный метод `findEndpointMethods()` анализирует измененные строки кода и выделяет методы, потенциально относящиеся к обработке HTTP-запросов. После их выявления выполняется его проверка с использованием метода `hasEndpointAnnotation()` на предмет наличия соответствующих аннотаций.

Таким образом, класс обеспечивает автоматизированный анализ измененного кода с целью выявления методов, ответственных за обработку запросов в приложении.

3.2.10. Класс *DiffParser*

Класс *DiffParser* предназначен для анализа изменений, внесенных в рамках PR. Он содержит метод `getChangedLines()`, который извлекает информацию о модификациях в файлах и формирует список номеров измененных строк.

Метод `getChangedLines()` обрабатывает данные о разнице между исходной и измененной версиями кода, определяя, какие строки добавлены, удалены или изменены. Полученный список измененных строк впоследствии используется для анализа модифицированных участков кода и их влияния на функциональность системы.

Таким образом, класс играет важную роль в процессе анализа изменений в коде, предоставляя информацию, необходимую для последующей обработки и генерации тестов.

3.2.11. Класс *PatchParser*

Класс *PatchParser* предназначен для обработки данных, полученных в результате анализа изменений. Он содержит публичный метод `parseChangedLines()`.

Метод `parseChangedLines()` анализирует выходные данные `git diff` и извлекает информацию о модифицированных строках, используя регулярное выражение, определенное в `CHUNK_HEADER_PATTERN`. Этот метод вызывается в `getChangedLines()` класса *DiffParser* и служит вспомогательным инструментом для точного выделения измененных строк в файлах.

Таким образом, класс играет роль в процессе детектирования изменений в коде, обеспечивая корректное извлечение информации о модифицированных строках, которая впоследствии используется для дальнейшего анализа.

3.2.12. Класс *EmbeddingService*

Класс *EmbeddingService* отвечает за генерацию эмбеддингов для последующего использования в БЗ. Он содержит свойство `embeddingModel`, представляющее собой экземпляр класса `OllamaEmbeddingModel`, который выступает в качестве клиента для взаимодействия с LLM, предназначенной для создания эмбеддингов.

Основная функциональность класса реализована в методе `getEmbedding()`, который выполняет генерацию векторного представления входных данных. Полученные эмбеддинги используются для хранения и обработки информации в векторной БД, обеспечивая эффективный поиск и анализ семантически схожего контента.

Таким образом, класс используется в процессе преобразования текстовой информации в эмбеддинги, обеспечивая ее дальнейшую интеграцию в систему управления знаниями.

3.2.13. Класс *VectorStorageService*

Класс *VectorStorageService* предназначен для взаимодействия с векторной БД `Milvus`. Он содержит свойство `vectorClient`, представляющее собой экземпляр клиента `Milvus`, обеспечивающего доступ к функционалу хранения и обработки векторных представлений данных.

Основная функциональность класса реализована через следующие методы:

- `initializeCollection()` — выполняет создание новой коллекции в БД Milvus, обеспечивая структуру для хранения векторных данных;
- `createIndex()` — отвечает за создание индекса в БД, позволяя оптимизировать процесс поиска по векторным представлениям;
- `insertEmbeddings()` — осуществляет вставку эмбеддингов в БД, обеспечивая их последующее использование в процессах поиска и анализа;
- `searchEmbedding()` — выполняет поиск записей в БД на основе заданного векторного представления, позволяя находить семантически схожие объекты.

Таким образом, класс играет роль в управлении векторными данными, обеспечивая их эффективное хранение, индексацию и поиск в БД Milvus.

3.2.14. Класс *ChatService*

Класс *ChatService* предназначен для взаимодействия с LLM в рамках обработки запросов в диалоговом режиме. Он содержит свойство `chatModel`, представляющее собой экземпляр класса `OllamaChatModel`, который выполняет функции клиента для работы с LLM, обеспечивающей генерацию текстовых ответов.

Основная функциональность класса реализована в методе `sendRequest()`, который отвечает за формирование и отправку запросов в LLM, а также за получение и обработку ее ответов. Данный метод обеспечивает возможность взаимодействия с моделью в рамках различных сценариев, требующих обработки естественного языка.

Таким образом, класс играет центральную роль в интеграции с языковой моделью, обеспечивая генерацию текстовых ответов и поддержку диалоговых взаимодействий в системе.

3.3. Функционал извлечения данных из PR

Первый этап разработки включал реализацию функционала извлечения кодовых изменений из PR. На первом шаге выполняется подключение к репозиторию и извлечение всех данных из GitHub с использованием специализированных библиотек `GNPullRequest` и `JGit`. Эти библиотеки обеспечивают доступ к данным о PR, включая кодовые изменения, информацию о ветках, метаданные репозитория и другие связанные данные.

Библиотека GitHub [47] представляет собой обертку для работы с GitHub API, предоставляя интерфейсы для получения данных, связанных с PR, включая информацию о ветках, авторах изменений, комментариях и других метаданных. Ее использование обусловлено простой интеграцией с сервисом GitHub, поддержкой протоколов аутентификации OAuth и возможностью эффективно работать с основными объектами репозитория.

Библиотека JGit [44] предоставляет средства для работы с локальным репозиторием. Она поддерживает ключевые команды системы контроля версий Git, такие как клонирование, обновление и синхронизацию с удаленной версией, обеспечивая автоматизацию операций с репозиторием. Использование данной библиотеки обосновано ее совместимостью со стандартными функциями Git, возможностью аутентификации через токены и упрощенной реализацией процедур обновления или создания локальных копий репозитория.

После получения данных от Jenkins о репозитории и идентификаторе PR выполняется подключение к GitHub. Для этого используется метод аутентификации через OAuth, обеспечивающий безопасный доступ к данным репозитория с использованием заранее настроенного токена доступа. В случае успешного подключения, , обработанного с учетом возможных ошибок, с помощью GitHub API извлекаются сведения о базовой ветке (base branch), в которую предполагается слияние, и ветке с изменениями (head branch). Эта информация подготавливается для последующей обработки.

Далее на основе URL репозитория формируется локальная папка для его размещения, чтобы избежать конфликтов между репозиториями и упростить управление данными. Для синхронизации используется библиотека JGit. Если локальная копия репозитория существует, она синхронизируется с удаленной с использованием методов fetch (для загрузки изменений) и pull (для слияния с удаленным аналогом). Если локальная копия отсутствует, репозиторий клонируется с использованием метода cloneRepository, который автоматически настраивает ссылки и загружает данные. В обоих случаях используется аутентификация через токен, что позволяет безопасно работать как с публичными, так и с приватными репозиториями. Также осуществляется обработка возможных ошибок, таких как недоступность репозитория, проблемы с авторизацией или сбой сети. Эти ошибки сохраняются для упрощения диагностики.

Далее подготавливаются локальные ветки для анализа изменений. Выполняется переключение на baseBranch и извлечение ее последнего коммита, после

чего проверяется наличие локальной копии headBranch. Если ветка существует, она удаляется и создается заново, синхронизируясь с удаленной версией. Затем извлекается последний коммит из headBranch.

Для анализа различий между baseBranch и headBranch используется метод diff из библиотеки JGit. Этот метод выявляет все изменения, включая добавление, удаление или модификацию файлов. Результатом работы метода является список изменений в виде объектов DiffEntry, содержащих тип изменения, пути к файлу в обеих ветках и степень сходства (для переименований). Эти данные используются для дальнейшей обработки и интеграции.

3.4. Парсинг извлеченных данных

Парсинг и извлечение структурных данных реализован с использованием библиотеки JavaParser [42]. Эта библиотека предназначена для работы с синтаксическим деревом Java-кода, что позволяет выполнять анализ исходных файлов, извлекать структуры классов, их поля, методы, аннотации и другие элементы. Эта библиотека является открытым и активно поддерживаемым проектом.

Алгоритм извлечения моделей данных представляет собой первый этап парсинга данных, извлеченных из репозитория. Модели данных в данном контексте определяются как типы, используемые в качестве аргументов и возвращаемых значений для API-маршрутов извлеченных на следующем этапе.

Для реализации процесса извлечения данных о моделях разработан класс ModelCollector, который анализирует содержимое директории с моделями. Он извлекает информацию из Java-классов и структурирует ее в определенном структурированном виде.

Перед началом анализа ModelCollector проверяет существование указанной директории. После этого обрабатываются файлы с расширением «.java», расположенные в директории моделей. Для обработки используется библиотека StaticJavaParser, которая создает синтаксическое дерево класса. Из классов извлекаются имя (className), аннотации (annotations) и список полей (fields). Если класс наследуется (через директиву extends), извлекаются также поля родительского класса, что обеспечивает полную информацию о структуре модели.

При возникновении ошибок, таких как синтаксические проблемы или сбои при разборе файлов, они сохраняются, а обработка продолжается для остальных, что повышает устойчивость алгоритма и предотвращает его прерывание.

После определения моделей выполняется анализ затронутых API-маршрутов в PR. В архитектуре приложений на основе Spring Framework контроллеры играют ключевую роль, обрабатывая HTTP-запросы и предоставляя доступ к функциональности системы через API. Именно поэтому функционал извлечения API-маршрутов сосредоточен на анализе контроллеров в репозитории для выявления и обработки добавленных API-маршрутов.

Из списка изменений diffs, полученного ранее, отбираются файлы, относящиеся к контроллерам. Контроллеры идентифицируются по содержащимся в файлах классам с аннотациями `@RestController` или `@Controller`, что позволяет отфильтровать только релевантные файлы, потенциально содержащие API-маршруты.

После идентификации измененных файлов извлекаются затронутые строки кода в каждом из них с использованием реализованного класса `PatchParser`. Этот класс анализирует содержимое объектов `DiffEntry`, представленное в виде патча, например:

```

5 | @@ -10,5 +10,6 @@
   | +    new line added
   |   unchanged line
   | -    removed line

```

`PatchParser` извлекает номера строк, которые изменены или добавлены. Для этого используется регулярное выражение, которое находит заголовки блоков изменений:

```
@@ -\d+,\d+ \+(\d+),(\d+) @@
```

После, на основании синтаксического дерева, построенного для содержимого файла с помощью библиотеки `StaticJavaParser`, и списка затронутых строк выполняется поиск методов, являющихся API-маршрутами. Этот процесс реализован в специально созданном классе `EndpointMethodLocator`. Методы идентифицируются по их привязке к измененным строкам и наличию аннотаций, таких как `@GetMapping`, `@PostMapping`, `@PutMapping` и `@DeleteMapping`. Эти аннотации позволяют однозначно определить, что метод является API-маршрутом, а также извлечь его основные атрибуты, включая HTTP-метод, URL-адрес, типы аргументов и тип возвращаемого значения.

После идентификации API-маршрутов формируется структура, описывающая изменения в контроллере. Создается объект `EndpointCodeDetails`, который включает список API-маршрутов, имя класса контроллера и данные о моделях,

используемых в API-маршрутах. Сформированная структура добавляется в общий список API-маршрутов для последующей обработки.

3.5. Формирование расширенного промта

После того как внесенные изменения по API-маршрутам становятся доступными, необходимо сформировать текстовый промт, обогащенный релевантным контекстом. В данном случае названия классов API-маршрутов выступают в качестве ключей для поиска и анализа информации в векторной БД. Чтобы обеспечить эффективную обработку этих ключей, на их основе генерируются эмбединги, по которым впоследствии производится поиск подходящих записей в векторной БД.

Основная логика формирования векторных представлений реализована в классе `EmbeddingService`. В нем используется модель `NOMIC_EMBED_TEXT` [53], являющаяся одной из открытых моделей для генерации эмбедингов текстов. Данная модель способна преобразовывать входные строки в числовые вектора, отражающие семантические особенности исходного текста.

В результате обращения запроса к эмбединги модели, результаты вычисления возвращаются в объекте `EmbeddingResponse`, содержащем векторное представление исходного текста. Далее полученный вектор может быть использован для поиска сопоставимых записей в векторной БД.

Для хранения данных в векторном представлении используется векторная БД `Milvus` [51] в качестве векторного хранилища. Коллекция `Milvus` организуется со следующими полями:

- `idField` — тип `Int64`, первичный ключ, автоинкремент;
- `embeddingField` — тип `FloatVector`, размерность определяется параметром `VECTOR_DIM`, по данному полю создается индекс для поиска по сходству;
- `docIdField` — тип `Int64`, используется для связи с конкретным документом;
- `chunkIdField` — тип `Int64`, при необходимости уточняет фрагмент документа.

В данной схеме только поле `idField` является автоинкрементируемым первичным ключом. Поскольку поиск осуществляется по полю `embeddingField`, для него в `Milvus` создается соответствующий индекс, обеспечивающий эффективную операцию поиска. По описанной коллекции в хранилище `Milvus` выполняется поисковый запрос, результатом которого является объект типа `SearchResp`, содер-

жащий наиболее релевантные записи, позволяющие обогатить исходный промт дополнительным контекстом.

3.6. Выводы по главе 3

В данной главе подробно рассмотрена реализация системы автоматизированного тестирования, включающая настройку инфраструктуры и разработку ключевых функциональных компонентов.

Оркестрация системы выполнена с использованием Jenkins, что позволило автоматизировать процесс обработки событий, передачу данных и выполнение тестов в рамках CI процесса. Настройка вебхуков и разработка пайплайнов обеспечили эффективную интеграцию с репозиториями, минимизировав ручные операции при запуске тестов.

Структура системы формализована представлена с использованием диаграммы классов, демонстрирующей взаимосвязи между основными компонентами: обработкой входящих событий, извлечение измененного программного кода, генерацией тестов и обновлением БЗ.

Одним из ключевых элементов разработки стала реализация функционала извлечения изменений из PR. С использованием GitHub API и JGit обеспечено автоматическое получение данных о модификациях в коде, что стало основой для дальнейшего анализа. Для структурирования полученных данных и выделения значимых изменений разработан механизм парсинга, реализованный с помощью JavaParser. Они позволили идентифицировать измененные модели данных и API-маршруты в кодовой базе, что важно для корректного формирования тестов.

Для повышения релеванности сгенерированных тестов в рамках концепции RAG, позволяющий учитывать контекст изменений, использовалась векторная БД Milvus. Это позволило организовать поиск релевантных данных для формирования расширенного промта для генерацию тестов. Это решение позволило частично нивелировать ограничения LLM, связанные с ограниченностью контекста и устареванием знаний.

ГЛАВА 4. ОЦЕНКА КАЧЕСТВА РАЗРАБОТАННОЙ СИСТЕМЫ АВТОМАТИЗИРОВАННОГО ТЕСТИРОВАНИЯ API

Глава 4 посвящена тестированию реализованной системы автоматизированного тестирования, основной целью которой является проверка качества средства различными методами тестирования. В параграфе 4.1 рассматриваются ключевые аспекты выбора подхода к тестированию. Параграф 4.2 описывает функциональное тестирование, а параграф 4.3 посвящен нагрузочному тестированию. Тестирование при различных настройках RAG изложено в параграфе 4.4, где также представлены варианты функционирования системы при разной степени обогащения данными векторной БД, а также с моделями LLM, обученными на различных наборах данных программного кода.

4.1. Выбор подхода к тестированию

4.1.1. Виды тестирования

Используемые виды тестирования:

1. *Модульное тестирование* направлено на проверку отдельных модулей разработанной системы в изоляции от других частей приложения. Этот подход позволяет выявлять ошибки на ранних этапах разработки, обеспечивая высокую степень надежности и корректности программного кода. При этом каждый модуль тестируется отдельно, что помогает определить, работает ли он в соответствии с заявленными требованиями и спецификациями. Для оценки качества модульного тестирования будет использоваться метрика *покрытия кода*, которая оценивается с помощью ПО JaCoCo [41].
2. *Функциональное тестирование* предназначено для проверки того, что разработанная система выполняет свои функции в соответствии с заявленными требованиями. Такой подход является подходящим, так как он позволяет удостовериться, что сгенерированные тесты корректно проверяют программный код, соответствуют ожидаемым сценариям использования и охватывают ключевые функциональные элементы приложения. Выбор функционального тестирования также обоснован тем, что разработанная система направлена на автоматизацию создания тестов, которые должны

обеспечивать высокий уровень надежности и корректности тестируемых программ. Проверка функционала сгенерированных автотестов является важной для оценки эффективности системы и ее применимости в реальных условиях. Для оценки качества функционального тестирования будут использоваться следующие критерии:

- покрытие кода (JaCoCo);
- устойчивость тестов к мутациям (оценивается с помощью ПО Pitest [35]).

3. *Нагрузочное тестирование* для выявления сбоев в работе разработанной системы, которые могут возникнуть при росте числа пользователей или объема операций. Для его проведения выбран инструмент Apache JMeter [25] с открытым исходным кодом, который предоставляет функции для анализа производительности системы. JMeter позволяет моделировать различные сценарии нагрузки и генерировать детализированные отчеты для анализа. Ключевые показатели, которые позволяет оценить инструмент и которые использованы для анализа:

- Throughput (пропускная способность): количество обработанных запросов в секунду;
- Average Response Time (среднее время ответа): среднее время обработки запроса системой;
- Min/Max Response Time (минимальное и максимальное время ответа): диапазон времени отклика;
- Percentiles (перцентили): 90-й, 95-й и 99-й перцентили помогают анализировать распределение времени отклика;
- Error Rate (уровень ошибок): процент запросов, завершившихся с ошибкой;
- Processing Time (время обработки): полное время обработки запроса системой;
- Active Threads (активные потоки): количество потоков, одновременно работающих в рамках теста;
- Throughput per Thread Group (пропускная способность группы потоков): количество запросов, обработанных отдельной группой потоков;
- Transactions per Second (транзакции в секунду): частота выполнения транзакций.

4.1.2. Проекты для выполнения тестирования

В качестве *первого проекта* для тестирования функциональности разработанного средства выбран открытый проект Spring PetClinic [59] по следующим причинам:

1. *Структура и сложность.* Проект демонстрирует типичную архитектуру к построению приложений на базе Spring Boot, включая использование различных слоев (REST-контроллеры, сервисы и т. д.). Он включает базовые API-маршруты для управления сущностями и работу с БД. Благодаря такой структуре проект дает хорошее представление о реальной организации кода и предоставляет базу для тестирования разработанной системы, позволяя проверить работу автогенерации тестов на различных сценариях.
2. *Широкая известность и популярность.* Проект представляет собой известный демо-проект сообщества Spring (8 тысяч звезд, более 24 тысяч созданий ответвлений).
3. *Социальная значимость для образования и ветеринарии.* Благодаря высокой популярности Spring Boot и широкому распространению решений на его основе, проект часто используется для обучения разработчиков и демонстрации возможностей Spring Framework, что способствует повышению уровня квалификации программистов и популяризации современных технологий разработки. Благодаря этому проекту, студенты и начинающие разработчики получают доступ к лучшим практикам, что помогает улучшить качество ПО. Также открытая лицензия и доступность способствуют широкой вовлеченности сообщества.

В качестве *второго проекта* для тестирования функциональности выбран сам сервис автоматической генерации тестов, потому что содержит реализацию двух API-маршрутов. Главная особенность использования данного проекта для тестирования перед другими выбранными проектами заключается в том, что проект является новым и *ни одна из сравниваемых больших языковых моделей* при тестировании не обучена на коде этого проекта.

Выбор *третьего проекта* SpringBootAdmin [58] для тестирования разработанной функциональности обоснован несколькими ключевыми факторами. Прежде всего, данный проект обладает значительно большим объемом кода и более сложной функциональностью по сравнению с ранее выбранными проектами. Это делает его

подходящим для проверки устойчивости и корректности разработанного решения в условиях высокой сложности. Кроме того, репозиторий имеет большое количество звезд (более 12 тысяч) и ответвлений (более 3 тысяч), что свидетельствует о его популярности и активном использовании сообществом разработчиков.

4.2. Модульное тестирование

Для модульного тестирования выбраны основные функциональные модули проекта, за исключением классов с методами, содержащие API-маршруты. Классы с API-маршрутами протестированы на этапе функционального тестирования, где они будут автоматически сгенерированы самим инструментом, так как разработанное приложение ориентированно для тестирования классов с API-маршрутами (см. подраздел 4.4.3.2).

Тесты для модульного тестирования распределены по следующим группам:

- *service-классы*: проверяется бизнес-логика и корректная обработка исключений. Включены классы `GitHubService` (см. подраздел П2.5), `VectorStorageService` (см. подраздел П2.7), `RAGService` (см. подраздел П2.6) и `EmbeddingService` (см. подраздел П2.4);
- *utility-классы*: оценивается корректность вспомогательных функций, включая обработку строк и форматирования. Включены классы `ModelCollector` (см. подраздел П2.11), `EndpointMethodLocator` (см. подраздел П2.13), `PatchParser` (см. подраздел П2.12);
- *worker-классы*: тесты охватывают обработку задач, связанных с генерацией тестов и обновлением БЗ. Включены классы `TestGenerationWorker` (см. подраздел П2.9), `KnowledgeUpdatingWorker` (см. подраздел П2.8);
- *queue-классы*: тесты охватывают обработку сообщений в брокере сообщений `RabbitMQ`, включая их валидацию и передачу в соответствующие обработчики. Включены классы `QueueListener` (см. подраздел П2.3), `RabbitMQConfig` (см. подраздел П2.2).

Класс с API-маршрутами (`WebhookController` (см. подраздел П2.1) исключен из модульного тестирования, так как он покрывается тестами на этапе функционального тестирования.

В ходе модульного тестирования разработано 156 автоматизированных тестов («П2») с использованием Java и фреймворка JUnit [45]. По его результатам получены следующие показатели покрытия кода на рис.4.1:

RAG API Test Generator

RAG API Test Generator

Element	Missed Instructions	Cov	Missed Branches	Cov	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.aiga.ragapitestgenerator.service		54%		70%	44	81	161	363	36	57	5	10
com.aiga.ragapitestgenerator.controller		0%		0%	10	10	34	34	6	6	1	1
com.aiga.ragapitestgenerator.model		23%		0%	6	10	14	18	5	9	3	6
com.aiga.ragapitestgenerator.util		95%		100%	1	58	8	145	1	24	1	8
com.aiga.ragapitestgenerator		0%		n/a	2	2	3	3	2	2	1	1
com.aiga.ragapitestgenerator.exception		90%		n/a	1	3	2	10	1	3	1	2
com.aiga.ragapitestgenerator.worker		100%		100%	0	20	0	80	0	9	0	3
com.aiga.ragapitestgenerator.queue		100%		100%	0	20	0	50	0	17	0	2
com.aiga.ragapitestgenerator.dto		100%		n/a	0	1	0	1	0	1	0	1
Total	867 of 2,860	69%	24 of 154	84%	64	205	222	704	51	128	12	34

Рис.4.1. Результаты модульного тестирования

- общее покрытие инструкций: 73% (737 из 2,730 инструкций пропущено, исключая 130 из классов, содержащих API-маршруты);
- покрытие ветвлений: 89% (16 из 146 ветвлений пропущено, исключая 8 из классов, содержащих API-маршруты);
- пропущенные методы: 45 из 122 (исключая 6 методов из классов, содержащих API-маршруты);
- пропущенные классы: 11 из 33 (исключая 1 класс, содержащий API-маршруты).

Текущие показатели модульного тестирования демонстрируют общее покрытие инструкций на уровне 73%. Этот результат находится в приемлемом диапазоне, так как свидетельствует о том, что основная часть функциональности протестирована. Оставшиеся 27% не протестированных инструкций в значительной степени приходится на классы, для которых тестирование представляет технические сложности или является необязательным. Например, класс `GitHubService` содержит сложные зависимости, связанные с мокированием взаимодействий с API GitHub и авторизацией.

Дополнительно, часть кода, исключенного из тестового покрытия, представлена вспомогательными структурами данных, такими как классы `EndpointCodeDetails` (см. подраздел П2.13) и `KnowledgeChunk` (см. подраздел П2.14). Эти классы выполняют функции контейнеров данных и содержат минимальную бизнес-логику.

Уровень покрытия ветвлений составляет 89%, что является хорошим показателем. Это свидетельствует о том, что большинство возможных логических путей выполнения проверено, что значительно снижает вероятность возникновения ошибок в условиях ветвления или обработки исключений. Таким образом, текущее покрытие модульного тестирования обеспечивает достаточный уровень стабильности и корректности системы, а непокрытые участки кода либо не яв-

ляются критичными, либо связаны с техническими ограничениями модульного тестирования.

4.3. Функциональное тестирование

Детальное и поэтапное функциональное тестирование для проекта Spring PetClinic рассмотрено ниже. Аналогичные шаги выполнены и для других проектов, подробнее см. подраздел 4.4.3.

4.3.1. Оптимизация проекта Spring PetClinic

Прежде чем приступить к тестированию, проведен детальный анализ исходного кода выбранного проекта. В ходе анализа определены ключевые API-маршруты, реализованные в контроллерах, которые обеспечивают выполнение основных операций над сущностями «Владелец» (Owner), «Питомец» (Pet) и «Ветеринар» (Vet). Помимо этого, в контроллере PetController выявлены API-маршруты, предоставляющие функциональность создания и редактирования сущности «Питомец».

Однако, в ходе анализа выявлено отсутствие механизма удаления записей. Удаление является стандартной операцией, позволяющей корректировать данные в случае ошибок или удалять устаревшую информацию.

Кроме того, внедрение операции удаления способствует оптимизации хранения данных в системе, что помогает уменьшить объем хранимых записей, повысить производительность БД и сократить время выполнения запросов, что актуально для масштабируемых приложений с большими объемами информации.

В связи с этим принято решение разработать новый API-маршрут, обеспечивающий удаление записи о питомце. Эта доработка расширяет функциональность приложения и повышает его гибкость в управлении данными. Новая функциональность будет протестирована с использованием разработанного средства автоматизации тестирования, что позволит оценить корректность и надежность реализованного решения.

В рамках доработки в контроллере реализован POST-метод для удаления соответствующей записи о питомце из БД:

```
@PostMapping("/pets/{petId}/delete")
public ResponseEntity<String> deletePet(@RequestHeader(value
    = "Authorization", required = false) String authHeader,
```

```

5      @PathVariable("ownerId") int ownerId, @PathVariable("petId")
        int petId) {

        if (!isValidToken(authHeader)) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).
                body("Unauthorized: Invalid or missing token");
        }

10      Owner owner = this.findOwner(ownerId);

        Pet optionalPet = this.findPet(ownerId, petId);
        if (optionalPet == null) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).body
                ("Pet with ID " + petId + " not found");
15      }
        owner.deletePet(optionalPet);

        return ResponseEntity.ok("Pet with ID " + petId + " was
            successfully deleted");
    }

```

4.3.2. Настройки пайплайна

Для тестирования предложенных изменений с использованием разработанного средства проект предварительно скопирован [48]. После этого настроен соответствующий вебхук рис.4.2, обеспечивающий автоматическую обработку изменений в коде.

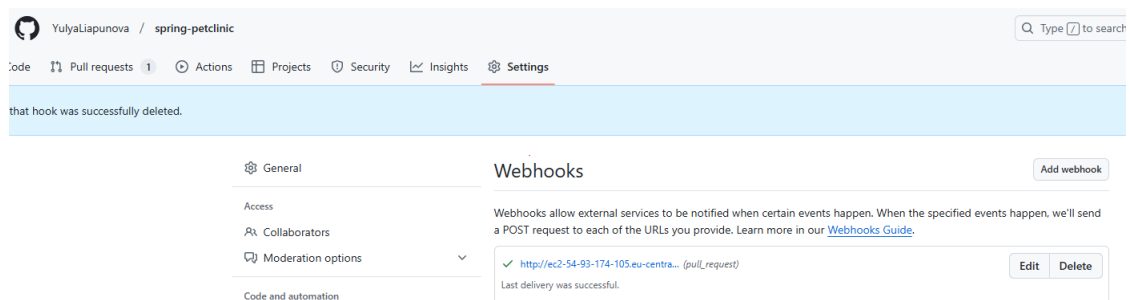


Рис.4.2. Настройка вебхука

После создания PR с внесенными изменениями в ветку main, автоматически срабатывает сконфигурированный пайплайн. В процессе выполнения пайплайна через его консоль можно детально отслеживать шаги на рис.4.3 процесса автогенерации тестов.

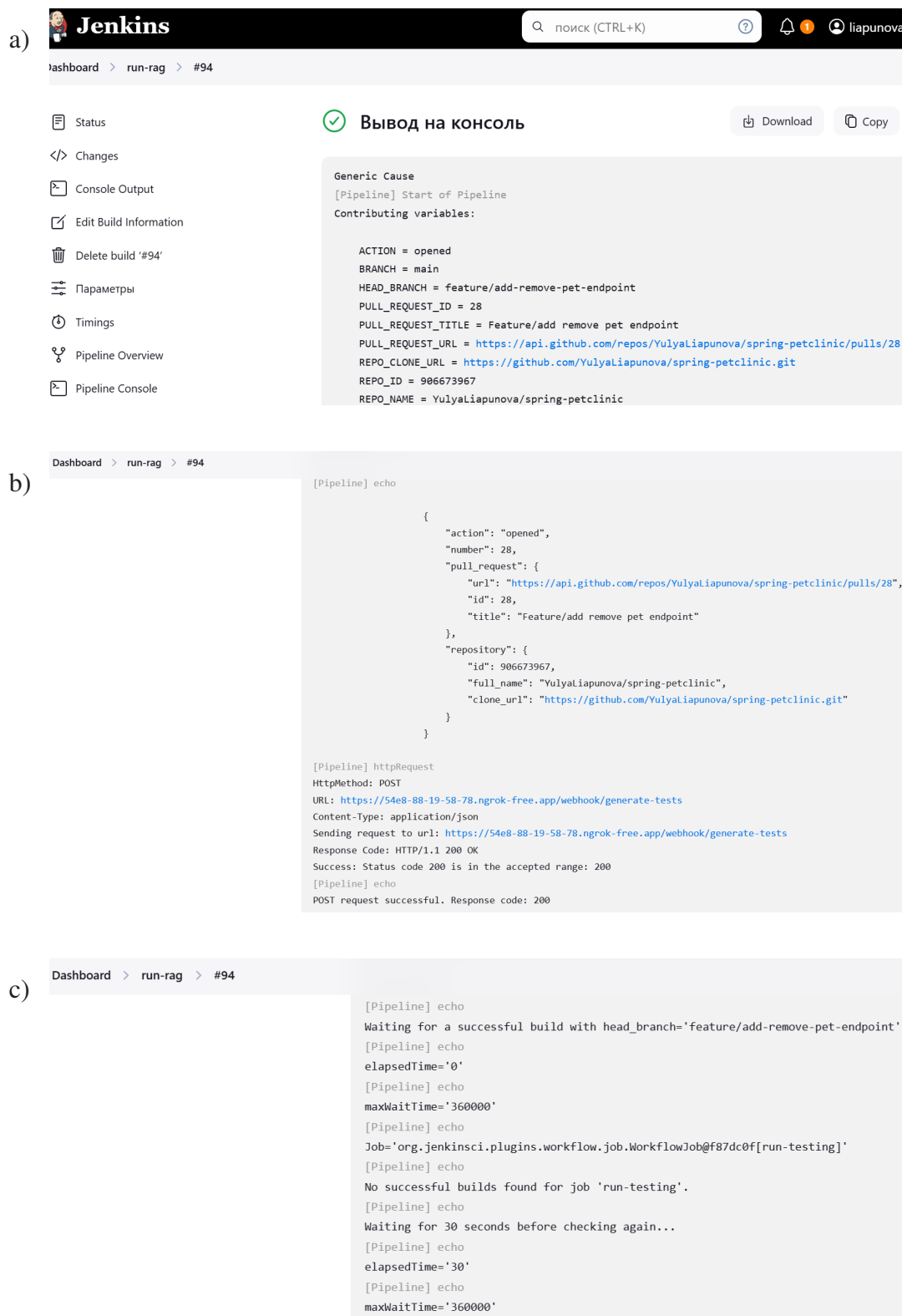


Рис.4.3. Скриншот работы пайплайна (часть 1), включая: *a* — получение данных вебхука; *b* — вызов системы автоматизированного тестирования; *c* — ожидание PR с сгенерированными тестами

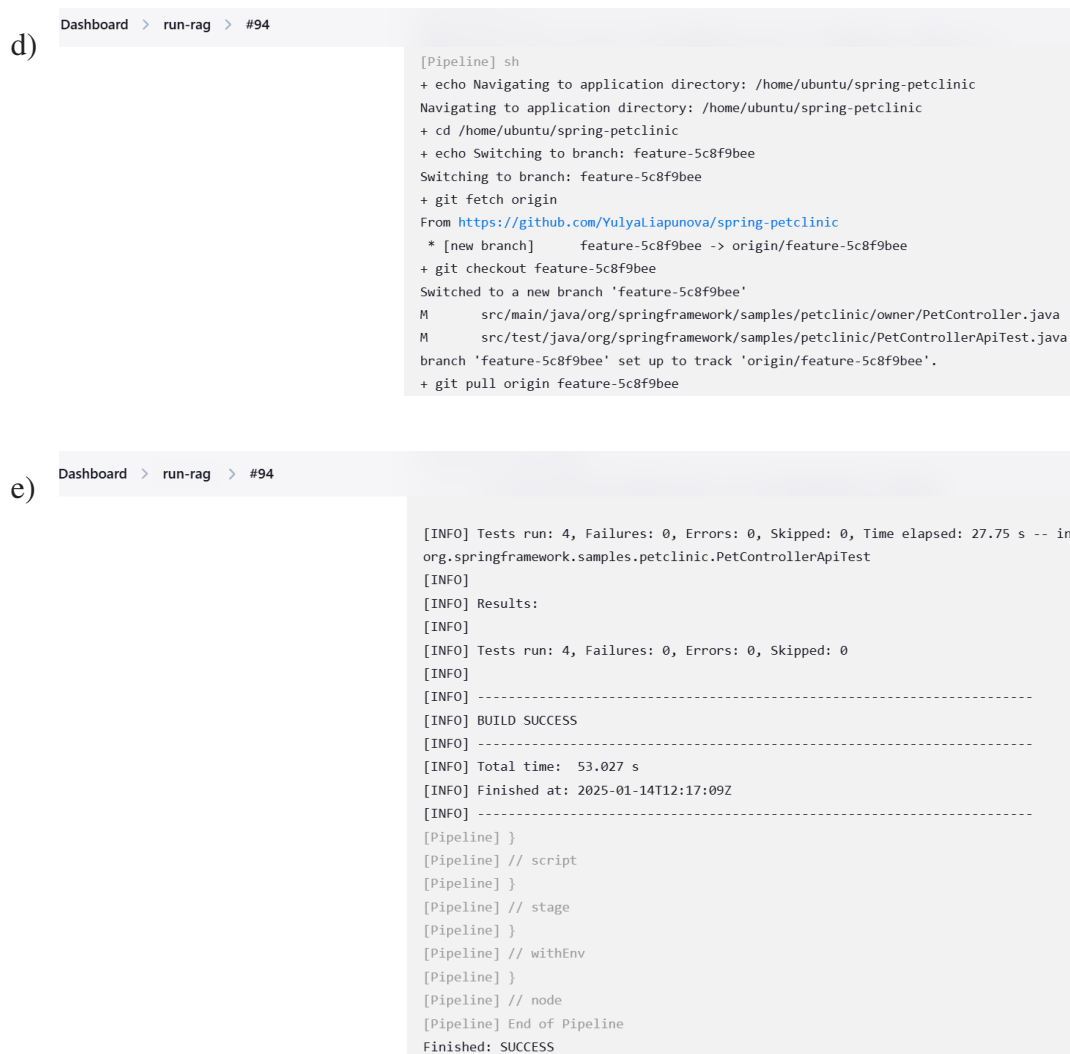


Рис.4.3. Скриншот работы пайплайна (часть 2), включая: *d* — подготовка окружения для тестирования; *e* — результаты тестирования

4.3.3. Результаты генерации тестов и покрытие исходного кода

В результате работы средства автоматизации тестирования сгенерировано 4 автотеста моделью codestral. На основе этих тестов автоматически создан новый PR в ветку разработки (на рис.4.4), содержащий сгенерированные тесты. Автотесты охватывают позитивный кейс удаления записи о питомце у владельца, а также негативные кейсы удаления несуществующего питомца, удаление без авторизации и удаления с некорректным идентификатором владельца.

Общее время выполнения пайплайна составило 3 минуты 24 секунды, из которых 53 секунды затрачены на прогон тестов. Скриншот с результатами приведен на рис.4.3е.

На основе анализа с использованием инструмента JaCoCo получены результаты покрытия кода для реализованного метода удаления питомца. Анализ показал,

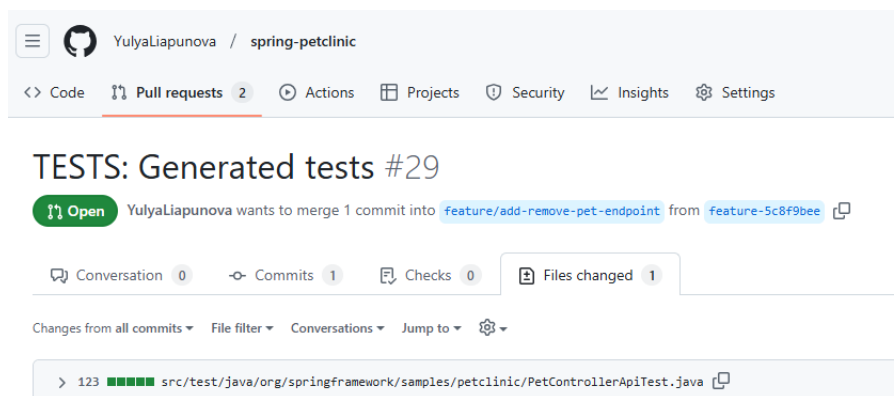


Рис.4.4. Автоматически созданный PR с тестами

что покрытие охватывает весь метод, включая все строки и ветвления. Фрагмент покрытия кода представлен на рис.4.5.

```

157. @PostMapping("/pets/{petId}/delete")
158. public ResponseEntity<String> deletePet(@RequestHeader(value = "Authorization", required = false) String authHeader,
159. @PathVariable("ownerId") int ownerId, @PathVariable("petId") int petId) {
160.
161.     if (!isValidToken(authHeader)) {
162.         return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Unauthorized: Invalid or missing token");
163.     }
164.
165.     Owner owner = this.findOwner(ownerId);
166.
167.     Pet optionalPet = this.findPet(ownerId, petId);
168.     if (optionalPet == null) {
169.         return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Pet with ID " + petId + " not found");
170.     }
171.     owner.deletePet(optionalPet);
172.
173.     return ResponseEntity.ok("Pet with ID " + petId + " was successfully deleted");
174. }
175.
176. private boolean isValidToken(String token) {
177.     return "valid-token".equals(token);
178. }

```

Рис.4.5. Анализ кода Ясосо

4.3.4. Проверка устойчивости тестов к мутациям исходного кода

Анализ с использованием инструмента Pitest показал, что мутационным изменениям подвержены: функционал удаления питомца из данных владельца и процесс формирования успешного ответа после выполнения удаления (171 и 173 строки на рис.4.6, соответственно).

Перейдем далее к анализу двух ситуаций, когда в результате мутации исходного кода проходил соответствующий автоматически сгенерированный тест.

4.3.5. Анализ первого выжившего мутанта

Первая мутация обнаружена при исключении вызова метода удаления питомца из БД после успешного выполнения всех валидаций (на рис.4.7). Это могло свидетельствовать, что сгенерированный тест не содержит проверки на то, что объект питомца действительно удален из списка питомцев владельца.


```

157     @PostMapping("/pets/{petId}/delete")
158     public ResponseEntity<String> deletePet(@RequestHeader(value = "Authorization", required = false) String authHeader,
159     @PathVariable("ownerId") int ownerId, @PathVariable("petId") int petId) {
160
161         if (!isValidToken(authHeader)) {
162             return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Unauthorized: Invalid or missing token");
163         }
164
165         Owner owner = this.findOwner(ownerId);
166
167         Pet optionalPet = this.findPet(ownerId, petId);
168         if (optionalPet == null) {
169             return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Pet with ID " + petId + " not found");
170         }
171         owner.deletePet(optionalPet);
172
173         return ResponseEntity.ok("Pet with ID " + petId + " was successfully deleted");
174     }
175
176     private boolean isValidToken(String token) {
177         return "valid-token".equals(token);
178     }
179 }

```

Рис.4.6. Анализ кода Pitest

```

1. removed call to org.springframework.samples.petclinic.model.Owner::deletePet → SURVIVED Covering tests
Covered by tests:
171 • org.springframework.samples.petclinic.PetControllerApiTest.[engine:junit-
jupiter]/[class:org.springframework.samples.petclinic.PetControllerApiTest]/[method:testDeletePetSuccess()]

```

Рис.4.7. Первый выживший мутант

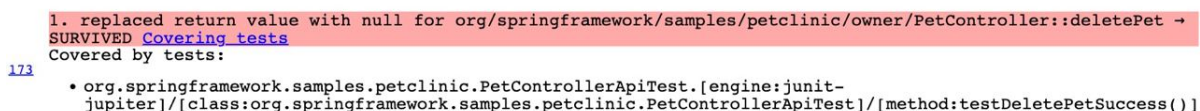
Однако, в ходе самостоятельного ручного анализа сгенерированного теста `testDeletePetSuccess` выявлено, что проверка удаления питомца из списка владельца присутствует на последнем шаге теста. Для более точного понимания проблемы сгенерированный тест дополнительно проанализирован в режиме отладки. В результате установлено, что оценка изменения состояния данных владельца реализована некорректно. Проверка существующих питомцев у владельца осуществлялась с использованием метода, который возвращал устаревшие данные и не отражал актуального состояния списка питомцев.

Данное несоответствие используемого метода при генерации теста могло возникнуть из-за ограничений в выборе методов, предназначенных для проверки актуального состояния сущностей системы `PetClinic`. Система генерации тестов, не имея полного представления обо всех доступных методах в системе, могла выбрать некорректный метод для оценки актуального состояния сущности «Владелец».

Этот случай подчеркивает необходимость улучшения процесса формирования промта при генерации тестов. Для обеспечения точного использования существующих методов требуется выполнение дополнительного анализа существующего кода разработанной системой с целью выявления доступных методов и их последующего включения в формируемый промт. Такие улучшения позволят повысить точность и надежность автоматически сгенерированных тестов.

4.3.6. Анализ второго выжившего мутанта

Вторая мутация обнаружена при формировании успешного HTTP-ответа после выполнения операции удаления питомца (на рис.4.8). Данная мутация создана путем замены возвращаемого значения на `null`, что может свидетельствовать о том, что сгенерированные автотесты не проверяют корректность возвращаемого значения в случае успешного удаления питомца.



```
1. replaced return value with null for org.springframework.samples.petclinic.owner.PetController::deletePet →
SURVIVED Covering tests
173 Covered by tests:
    • org.springframework.samples.petclinic.PetControllerApiTest.[engine:junit-
      jupiter]/[class:org.springframework.samples.petclinic.PetControllerApiTest]/[method:testDeletePetSuccess()]
```

Рис.4.8. Второй выживший мутант

В результате анализа теста `testDeletePetSuccess` установлено, что данный тест проверяет исключительно статус HTTP-ответа (`HttpStatus.OK`), но не оценивает содержимое тела ответа. Это ограничение в проверке привело к тому, что некорректное поведение API осталось незамеченным.

Для повышения качества автотестирования в БЗ разработанной системы рекомендуется добавить кейсы, связанные с проверкой содержимого тела ответа при выполнении операций удаления, в качестве лучших практик. На текущий момент используемые практики ограничиваются проверкой только статуса HTTP-ответа, что упрощает тестирование, но не позволяет в полной мере оценить корректность работы API. Расширение практик проверки, включающее анализ содержимого тела ответа метода `DELETE`, позволит более точно выявлять возможные ошибки в работе API и повысит общий уровень надежности системы.

4.4. Тестирование особенностей RAG

Использование концепции RAG в разработанной системе требует специфического подхода к тестированию, учитывающего влияние комбинации полноты векторной БД и языковой модели на поведение системы. Приведенные комбинации векторной БД и LLM (см. подраздел 2.5.3) позволяют оценить работу системы в различных условиях. Анализ этих случаев позволяет определить влияние полноты векторной БД и специфичности обучения языковой модели на качество и точность работы системы.

4.4.1. Пустая векторная БД + LLM обученная на программном коде

Для выполнения тестирования этого кейса использована пустая векторная БД. В результате промт не обогатился лучшими практиками тестирования и документацией по тестируемому API-маршруту, в которой содержатся его описание, входные параметры, коды ответов, примеры тела ответа и дополнительные замечания. После выполнения генерации автотестов выявлены следующие результаты:

1. Отсутствие проверки на соответствие бизнес-требованиям: тест на успешное создание не проверял наличие удаленного элемента в базе (данная проверка описана в технической документации в дополнительных замечаниях к API-маршруту).
2. Неполное тестовое покрытие: в результирующем классе отсутствовал метод проверки сценария удаления животного у несуществующего владельца. Это может быть обусловлено тем, что промт не обогатился входными параметрами из технической документации.
3. Отсутствие проверок содержащейся информации в теле ответа: отсутствуют проверки сообщений об ошибках. Данная информация могла быть получена из примеров ответа, содержащихся в технической документации.

Полученные результаты демонстрируют влияние отсутствия обогащенной БЗ на полноту и качество сгенерированных автотестов. Отсутствие доступа к документации и лучшим практикам привело к снижению тестового покрытия и неполной проверке функциональности. В связи с этим дальнейшее тестирование будет проводиться исключительно на основе обогащенной БЗ, что позволит повысить точность и качество сгенерированных тестов.

4.4.2. Обогащенная векторная БД + LLM необученная на программном коде

После определения влияние обогащенной векторной БД (см. подраздел 4.4.1) для корректной генерации тестов возникает вопрос о том, насколько критично использование модели, не обученной на целевом ЯП. Необходимо оценить влияние выбора модели на корректность и полноту сгенерированных автотестов.

Для выполнения тестирования системы использована модель Llama 3.2 [50], которая по характеристикам не обучена на ЯП Java. После выполнения генерации автотестов выявлены следующие результаты:

1. Неправильные исключения: модель указала некорректные исключения. А именно, вместо требуемого класса исключений указан слишком общий

класс исключения (`HttpServerErrorException.InternalServerError.class`) для кейса с несуществующим владельцем (`RuntimeException.class`).

2. Неоптимальное использование аннотаций: при использовании модели, не обученной на программном коде, она не использовала аннотацию `@BeforeClass` для создания в ней экземпляра класса `RestTemplate` для выполнения HTTP-запроса. В результате создает этот экземпляр в каждом классе.

Результаты данного тестирования свидетельствуют о том, что выбор модели, не обученной на целевом ЯП, оказывает негативное влияние на качество и корректность сгенерированных тестов. Несмотря на наличие обогащенной векторной БД, модель допускает ошибки в ключевых аспектах тестирования, таких как обработка исключений и применение аннотаций. Это указывает на необходимость использования моделей, специально обученных на программном коде, для достижения более точных и корректных результатов в процессе автоматической генерации тестов.

4.4.3. Обогащенная векторная БД + LLM обученная на программном коде

После определения влияния обогащенной векторной БД и модели, не обученной на целевом ЯП, на корректность генерации тестов необходимо оценить, в какой степени использование моделей, имеющей специализированные знания о Java, но обученные на разных наборах данных, влияет на полноту и точность сгенерированных автотестов.

Для выполнения функционального тестирования в такой комбинации необходимо рассмотреть несколько LLMs, обученных на программном коде. Для анализа выбраны три популярные языковые модели, находящиеся в свободном доступе и поддерживающих генерацию кода на ЯП Java:

- `codellama` [32];
- `codegemma` [30];
- `codestral` [33].

Генерация тестов с использованием данных моделей проведены на трех репозиториях. Результаты генерации рассмотрены ниже.

Тестирование генерации тестов проведено на машине со следующими характеристиками:

- процессор: M3 Max;

- оперативная память: 36Гб;
- кол-во ядер: 14.

4.4.3.1. Функциональное тестирование PetClinic

Для генерации тестов для API-маршрута удаления использован промт размером 1661 токенов.

Таблица 4.1

Сравнение генерации автотестов для PetClinic

Критерий	codellama	codegemma	codestral
Покрытие	75%	75%	100%
Устойчивость тестов к мутациям	80%	80%	92%
Скорость генерации (мс)	17741	14415	69180
Кол-во сгенерированных токенов	733	564	1094

После оценки покрытия сгенерированных тестов выявлено, что codellama и codegemma сгенерировали на один кейс меньше, чем требовало 100% покрытие этого API-маршрута. Эти две модели не учли кейс удаления питомца несуществующего владельца.

Результат оценки сгенерированных тестов к мутациям показал, что каждая модель допустила мутации определенного вида:

- codestral допустила две мутации, одна из которых связана с проверкой тела ответа при успешном удалении, а также некорректное использование функции для получения текущего состояния после удаления;
- codegemma допустила аналогичные мутации, как и модель codestral, а также мутация на непокрытый кейс по удалению питомца у несуществующего владельца;
- codellama допустила аналогичные мутации, как и модель codegemma.

Время затраченное на генерацию тестов модели codellama и codegemma в 4-5 раз меньше, чем у codestral. Данный результат можно объяснить количеством сгенерированных токенов, которые в 1,5-2 раза меньше, чем у codestral.

4.4.3.2. Функциональное тестирование разработанной системы

Для генерации тестов по покрытию двух API-маршрутов использован промт размером 2184 токенов. Первый маршрут отвечает за отправку данных о

созданном PR в очередь. Второй маршрут отвечает за отправку файлов для пополнения БЗ.

Таблица 4.2

Сравнение генерации автотестов для разработанной системы

Критерий	codellama	codegemma	codestrall
Покрытие	60%	80%	100%
Устойчивость тестов к мутациям	54%	83%	95%
Скорость генерации (мс)	19699	32935	85332
Кол-во сгенерированных токенов	627	1448	1681

Оценка покрытия сгенерированных тестов показала следующие результаты:

- codestral сгенерировала тесты, покрывающие все кейсы использования API-маршрутов, как позитивные, так и негативные;
- codegemma сгенерировала тесты, которые покрывают основной функционал, но не учтены негативные кейсы на получение ошибок отправки сообщения в очередь и чтения файла, необходимого для пополнения БЗ;
- codellama сгенерировала позитивные кейсы использования API-маршрутов, но не учтены разные негативные кейсы на обновления БЗ, связанные с отсутствием вариативности параметров запроса.

Результат оценки устойчивости сгенерированных тестов в рамках созданных двух API-маршрутов в разработанной системе к мутациям показал, что каждая модель допустила мутации определенного вида:

- codestral допустила мутацию при проверке тела ответа для кейса с невалидными параметрами запроса для обновления БЗ (данная аномалия может быть связана с особенностями работы языковой модели, так как в промте не задано явной проверки тела запроса в каждом кейсе);
- codegemma допустила мутации, связанные с отсутствием покрытия кейсов по обработке ошибок с файлом и очередью;
- codellama допустила аналогичные мутации, как и модель codegemma, а также отсутствие проверок тела ответа для всей кейсов.

4.4.3.3. Функциональное тестирование SpringBootAdmin

Для генерации тестов по покрытию нескольких API-маршрутов использован промт размерностью 4372 токенов. Особенность данного функционала для

тестирования заключается в проверке покрытия тестами нескольких маршрутов, имеющих один и тот же путь, но отличающийся HTTP-методом, потому что тесты должны четко разделять проверки этих маршрутов и учитывать возможные конфликты.

Таблица 4.3

Сравнение генерации автотестов для SpringBootAdmin

Критерий	codellama	codegemma	codestrall
Покрытие	50%	74%	85%
Устойчивость тестов к мутациям	43%	65%	80%
Скорость генерации (мс)	25917	30567	124079
Кол-во сгенерированных токенов	1573	1668	1995

Оценка покрытия сгенерированных тестов показала следующие результаты:

- codestral показал хорошие результаты по покрытию позитивных и негативных кейсов, за исключением проверки исключительного случая (NoSuchElementException), а также отсутствие покрытия реактивного потока(Flux), который генерирует события с заданным интервалом времени;
- codegemma показала отсутствие покрытия реактивного потока, аналогично codestral, а также граничные случаи для пустого списка и отсутствующих данных;
- codellama показала отсутствие покрытия кейсов, аналогично codegemma, а также исключительные случаи при фильтрации данных (Stream.filter).

Мутации следующего вида допущены моделями:

- codestral сгенерировала тесты не подверженные мутациям при различных сценариях обработки данных, что демонстрирует высокую вариативность кейсов. Однако, содержатся мутации из-за отсутствия покрытия реактивного потока, так как данная функциональность является специфичной для java-проектов;
- codegemma содержит аналогичные мутации, как и codestral, а также мутации из-за отсутствия покрытия граничных значений на пустые объекты;
- codellama не устойчива к аналогичным мутациям, как и codegemma, а также мутациям в фильтрации.

С увеличением сложности проекта наблюдается снижение процента покрытия и устойчивости к мутациям, а также увеличение времени генерации тестов. Но,

в сравнении, модель codestral имеет лучшие показатели относительно других рассмотренных моделей, за исключением времени, что делает ее более подходящей для системы автогенерации тестов. Так как при ее использовании будет сокращен ручной труд QA-инженеров для высокого покрытия разработанного функционала тестами. Среднее время создания одного автотеста для автоматизированного QA-инженера составляет несколько минут (около 5-30 минут в зависимости от сложности теста на основе самостоятельных замеров при написании автотестов), в то время как за 2 минуты модель codestral генерирует несколько тестов, которые составляют 85% покрытия функционала.

4.4.3.4. Генерация тестов с помощью аналогов Zenkoder и Cursor

Воспользуемся двумя близкими по функционалу аналогами Zencoder и Cursor (см. подраздел 1.4).

Для проверки функционала генерации тестов использовались разработанная системы, а именно два новых разработанных API-маршрута. В результате работы обоих аналогов были сгенерированные тесты (см. подраздел П2.17), в которых отсутствовали необходимые импорты пакетов. После ручного дополнения необходимых импортов тесты были успешно пройдены. Помимо этого, отсутствует разнообразие негативных кейсов для значений входных параметров тестируемых API-маршрутов.

Также выявлено, что Zencoder не вносит изменения в существующий файл, а вставляет новый сгенерированный код в начало файла, что свидетельствует о том, что инструмент плохо анализирует существующие части кода тестов. Пример такого поведения продемонстрирован на рис.4.9.

Рассмотренные инструменты показали хорошую генерацию кода автотестов. Однако, их основное различие с полученными результатами от разработанной системы в наличие ограниченности негативных кейсов. Данный факт может свидетельствовать о том, что актуальная база знаний помогает расширить генерацию и сделать ее более широкой.

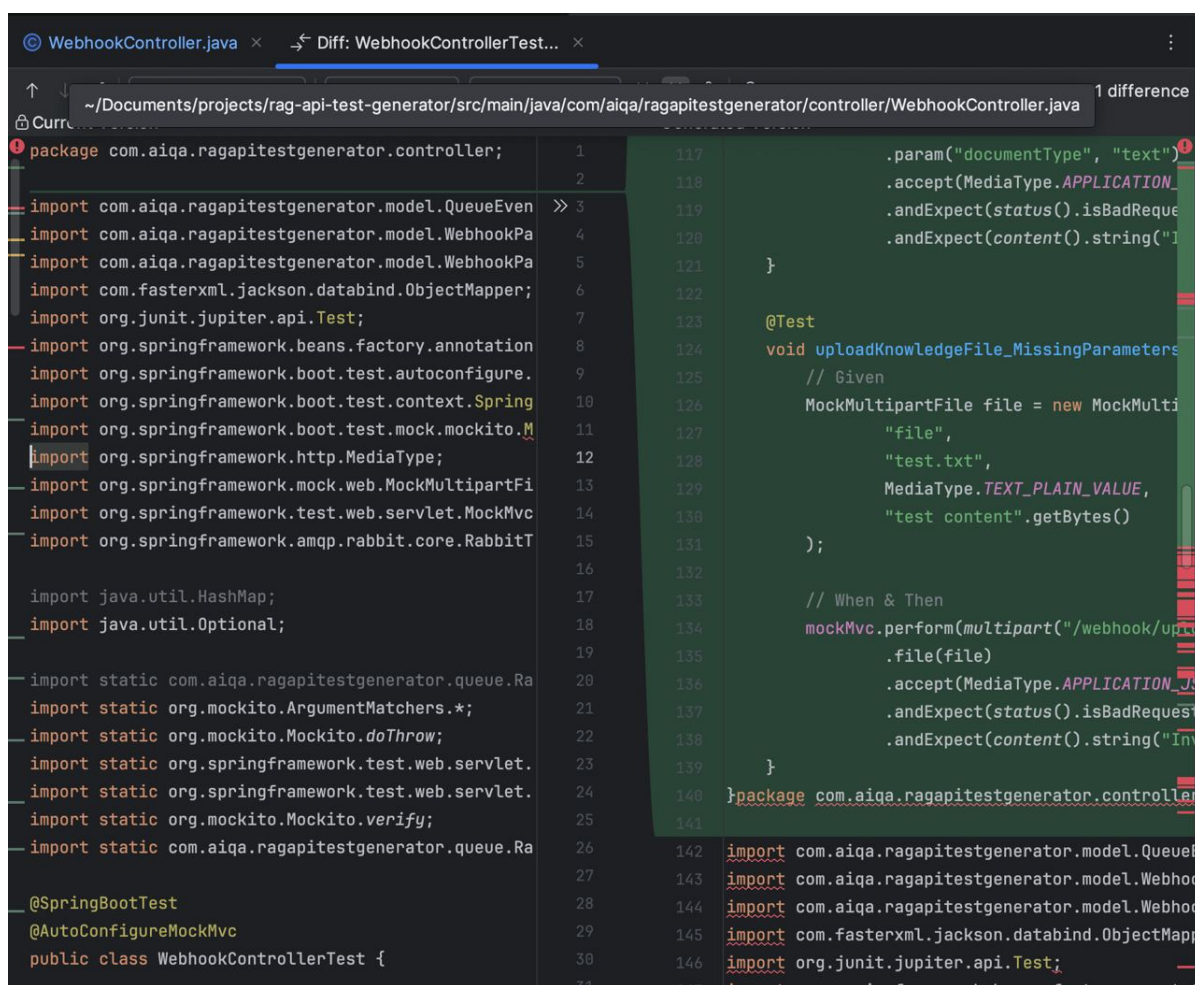


Рис.4.9. Работа инструмента Zencoder

4.5. Нагрузочное тестирование

Следующим этапом тестирования проведено нагрузочное тестирование для проверки стабильности работы системы под следующими уровнями интенсивности пользовательских операций:

1. Легкая нагрузка — моделирует минимальную нагрузку на систему и позволяет оценить ее производительность в стандартных условиях:
 - Number of Threads (количество потоков): 10;
 - Loop Count (количество повторений): 10;
 - Ramp-up Period (время разгона): 10 секунд.
2. Средняя нагрузка — имитирует условия, близкие к рабочим, с увеличением числа пользователей за более длительный период времени:
 - Number of Threads: 100;
 - Loop Count: 10;
 - Ramp-up Period: 100 секунд.

3. Всплеск нагрузки — проверяет способность системы справляться с резким увеличением числа пользователей:

- Number of Threads: 100;
- Loop Count: 10;
- Ramp-up Period: 10 секунд.

При легкой нагрузке на рис.4.10а система показывает стабильные результаты. Среднее время ответа составляет 181 мс, а медиана — 160 мс, что свидетельствует о быстрой обработке запросов. Пропускная способность составила 9.3 запроса в секунду, что соответствует ожидаемым характеристикам при данной нагрузке. Процент ошибок равен 0%, все запросы обработаны корректно. Это говорит о том, что система уверенно справляется с базовыми нагрузками.

При средней нагрузке на рис.4.10b время ответа увеличивается: среднее составляет 199 мс, медиана — 159 мс. Максимальное время ответа существенно возросло до 2344 мс, что может быть обосновано влиянием технических ограничений арендованного сервера. Стандартное отклонение времени ответа значительно увеличилось до 185.56 мс, что указывает на меньшую стабильность системы при этой нагрузке. Пропускная способность составила 9.9 запросов в секунду, что лишь немного превышает показатели при легкой нагрузке. Процент ошибок остается на уровне 0%, а все запросы обработаны благодаря использованию очереди.

При всплеске нагрузки на рис.4.10с система демонстрирует дальнейшее увеличение времени ответа: среднее — 218 мс, медиана — 196 мс, максимальное — 766 мс. При этом стандартное отклонение снизилось до 94.79 мс, что свидетельствует о более стабильной работе системы в этом режиме. Процент ошибок остается равным 0%, все запросы надежно обработаны. Это подтверждает, что использование очереди эффективно справляется с нагрузкой.

Все параметры, замеренные при нагрузке, продемонстрированы в табл.4.4.

Результаты нагрузочного тестирования показывают, что система демонстрирует удовлетворительную устойчивость и корректную обработку запросов при разных уровнях нагрузки. Пропускная способность постепенно увеличивается, достигая своего максимума в условиях всплеска нагрузки (82.4 запроса в секунду). При этом наблюдается закономерное увеличение времени ответа, что связано с техническими ограничениями текущего арендованного сервера. Эти ограничения особенно заметны при средней нагрузке, где наблюдается значительное увеличение максимального времени ответа.

Таблица 4.4

Результаты нагрузочных тестов

Критерий	Легкая нагрузка	Средняя нагрузка	Всплеск нагрузки
Количество запросов	100	1000	1000
Среднее время ответа (мс)	181	199	218
Медиана времени ответа (мс)	160	159	196
Стандартное отклонение (мс)	89.78	185.56	94.79
90-й перцентиль (мс)	279	298	343
95-й перцентиль (мс)	329	413	414
99-й перцентиль (мс)	413	1046	579
Минимальное время ответа (мс)	89	88	86
Максимальное время ответа (мс)	692	2344	766
Процент ошибок	0	0	0
Пропускная способность (запросов/секунду)	9.3	9.9	82.4
Принятые КБ/сек	1.51	1.61	13.36
Отправленные КБ/сек	5.87	6.25	51.84

Однако во всех случаях процент ошибок остается равным 0%. Таким образом, использование очереди обработки сообщений, продемонстрированы на рис.4.11, обеспечивает надежность системы, исключая потерю запросов и укладываясь в установленные таймауты. Даже при высокой нагрузке все запросы поступают в очередь и обрабатываются с течением времени.

Помимо работы основной очереди, также протестирована очередь DLQ, использование которой дополнительно повышает устойчивость обработки сообщений. В кейсах возникновения ошибок при генерации тестов или при пополнении БЗ, такие сообщения перенаправляются в отдельную очередь рис.4.12. Это позволяет исключить их потерю и сохранить их для дальнейшего анализа. Сообщения, помещенные в DLQ, могут быть повторно отправлены в основную очередь после исправления причин сбоя, что обеспечивает дополнительную надежность системы. Такой подход делает систему более устойчивой и гибкой, снижая риск потери данных даже в самых сложных условиях эксплуатации.

Текущие результаты свидетельствуют о том, что система готова к эксплуатации при увеличении нагрузки, но требует дополнительных технических ресурсов для обеспечения максимальной производительности.

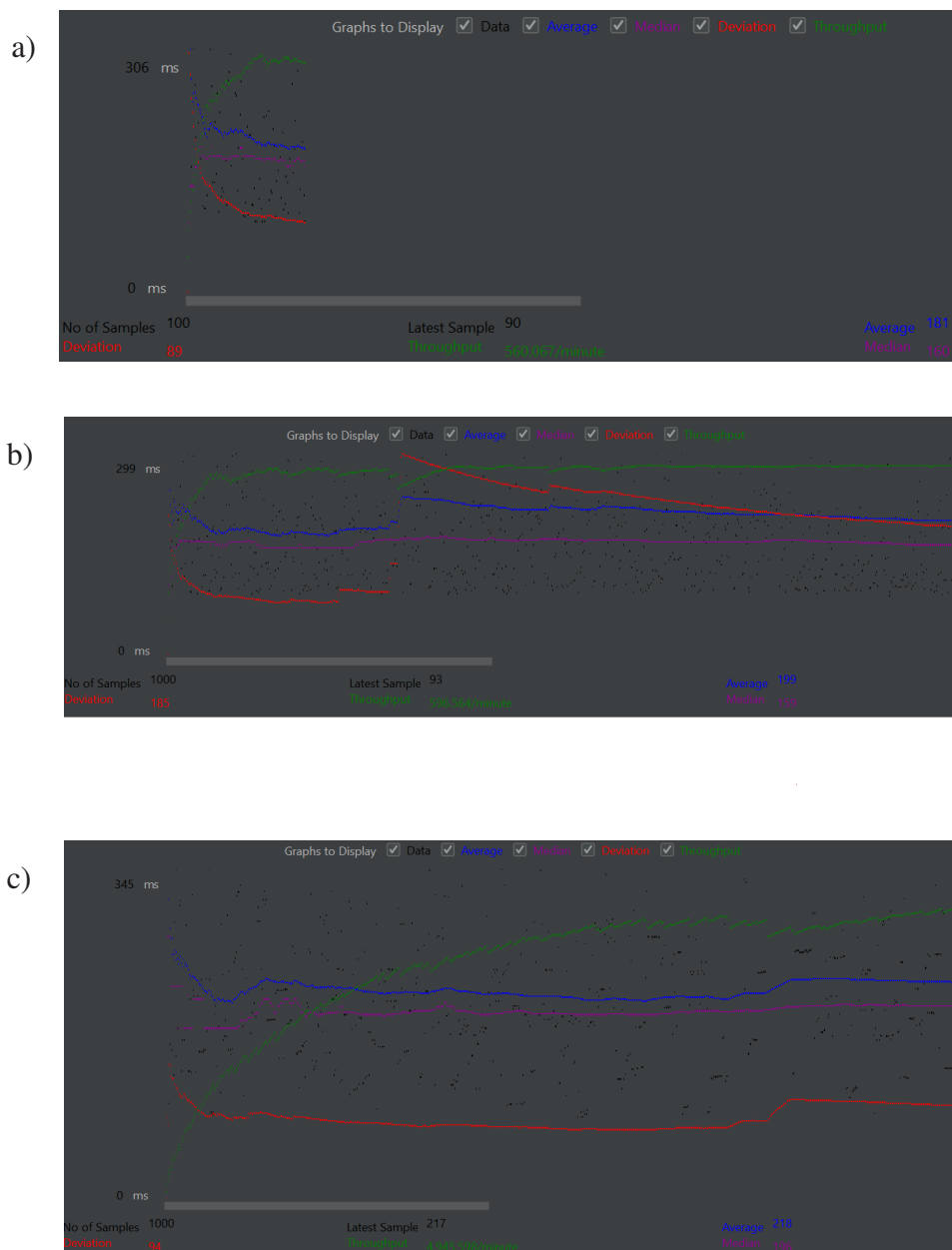


Рис.4.10. Графики производительности, в том числе: *a* — для легкой нагрузки; *b* — для средней нагрузки; *c* — для всплеска нагрузки

4.6. Выводы по главе 4

В данной главе проведено тестирование разработанной системы, включающее модульное, функциональное и нагрузочное тестирование, а также анализ особенностей использования концепции RAG.

Проведенное модульное тестирование позволило подтвердить корректность работы основных компонентов системы, обеспечив покрытие инструкций на уровне 73% и ветвлений на уровне 89%. Выявленные пропуски обусловлены техническими ограничениями при мокировании внешних зависимостей, а также отсутствием необходимости тестирования структур данных.

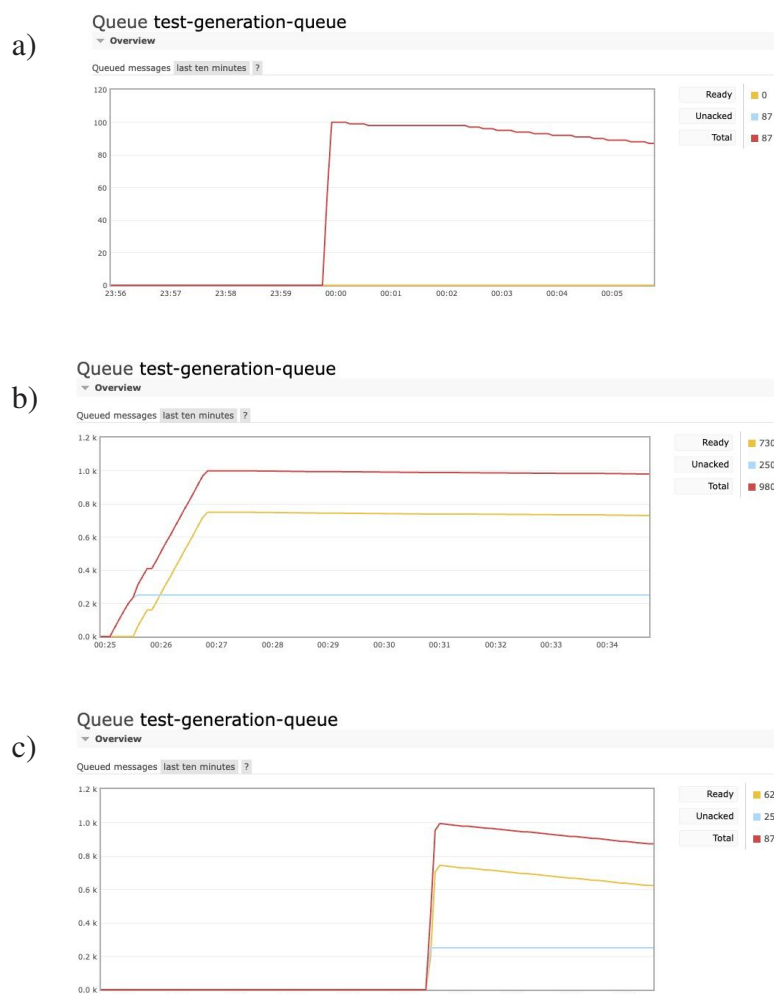


Рис.4.11. Состояние очереди, в том числе: *a* — для легкой нагрузки; *b* — для средней нагрузки; *c* — для всплеска нагрузки

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	knowledge-base-update-dlq	classic	D Args	running	0	0	0				
/	knowledge-base-update-queue	classic	D DLX DLK Args	running	0	0	0	0.00/s	0.00/s	0.00/s	
/	test-generation-dlq	classic	D Args	running	1	0	1				
/	test-generation-queue	classic	D DLX DLK Args	running	0	0	0	0.00/s	0.00/s	0.00/s	

Рис.4.12. Пополнение DQL

Функциональное тестирование, проведенное на трех проектах, продемонстрировало зависимость полноты и качества сгенерированных тестов от выбора используемой языковой модели. Анализ работы различных моделей выявил, что использование Codestral позволило достичь лучшего уровня покрытия функциональности, в то время как модели CodeLlama и CodeGemma не обеспечивали достаточного охвата негативных сценариев и демонстрировали недостаточную устойчивость к мутациям. Проведенный анализ особенностей работы системы в зависимости от наличия обогащенной векторной базы знаний и специфики языковой модели показал, что отсутствие БЗ приводит к значимым пропускам в

сгенерированных тестах. Таким образом, максимальная полнота и корректность автотестов достигается при сочетании специализированной языковой модели и обогащенной базы знаний, что подтверждает целесообразность выбранного подхода к реализации системы.

Результаты нагрузочного тестирования подтвердили устойчивость системы к различным уровням нагрузки, включая стандартные и стрессовые сценарии эксплуатации. При увеличении нагрузки наблюдался рост времени ответа, однако система продемонстрировала стабильную работу без возникновения ошибок, что подтверждает эффективность использования брокера сообщений RabbitMQ и механизма DLQ для надежной обработки входящих запросов. Максимальная пропускная способность системы, составившая 82.4 запроса в секунду, подтверждает ее способность эффективно обрабатывать входящие запросы в условиях интенсивной нагрузки. Такой показатель обеспечивает достаточную производительность для компаний с различным масштабом штата разработчиков, включая как небольшие команды разработки, так и крупные организации с высокой частотой внесения изменений в кодовую базу.

ЗАКЛЮЧЕНИЕ

Выпускная квалификационная работа посвящена разработке системы автоматизированного тестирования, использующей LLM в процессе непрерывной интеграции. В ходе исследования проведен анализ существующих инструментов автоматизации тестирования. В результате анализа выявлены ряд недостатков, включая ограничение на использование данных инструментов на территории РФ ввиду санкционных ограничений, а также отсутствие возможности работы с актуальной базой знаний для генерации тестовых данных. На основе выявленных ограничений предложен подход, основанный на концепции RAG, включающий использование открытой LLM, развернутой локально, а также дополнительной приватной базы знаний регулярно обновляющейся.

В ходе проектирования и реализации системы сформулированы основные требования, учитывающие необходимость автоматической генерации тестов, актуализации базы знаний, обработки контекстной информации, интеграции с CI-процессом и поддержки масштабируемой архитектуры. Разработанная архитектура системы включает брокер сообщений, worker-узлы и векторную базу данных, что позволяет минимизировать задержки при обработке событий, эффективно управлять данными и обеспечивать выполнение стабильного процесса тестирования.

Проведенное тестирование системы включало модульное, функциональное и нагрузочное тестирование, а также исследование особенностей работы RAG в разных конфигурациях. Модульное тестирование подтвердило корректность работы системы, обеспечив покрытие инструкций на уровне 73% и покрытие ветвлений на уровне 89%. Функциональное тестирование продемонстрировало, что полнота и точность сгенерированных тестов зависят от выбора языковой модели. Среди протестированных моделей наилучшие результаты показала Codestral, обеспечившая максимальное покрытие и устойчивость к мутациям. Нагрузочное тестирование подтвердило способность системы выдерживать интенсивные нагрузки без потери запросов и критических отказов, что свидетельствует о надежности разработанного решения. Максимальная пропускная способность составила 82.4 запроса в секунду, что позволяет использовать систему в компаниях любого масштаба, включая крупные организации с высокой частотой коммитов в кодовую базу.

Дальнейшее развитие данной системы возможно в двух направлениях: *оптимизация процесса генерации тестов и развитие механизмов самообучения системы.*

Для оптимизации процесса генерации тестов можно:

- внедрение механизма, позволяющего не только генерировать тесты для новой функциональности, но и актуализировать уже существующие тесты при внесении изменений в код. Это позволит избежать устаревания тестов и повысить их релевантность;
- расширить функциональность системы, включив поддержку генерации unit-тестов, тестов производительности и тестов безопасности. Это позволит охватить более широкий спектр;
- определить механизм выявления доступных методов в коде, которые корректнее выявлять релевантные проверки, для формирования более точного расширенного промта.

Для развитие механизмов самообучения системы можно:

- реализовать механизм, позволяющего анализировать ошибки в сгенерированных тестах и учитывать их при последующих генерациях;
- добавление интеграции с системами статического анализа кода для выявления потенциальных ошибок еще на этапе генерации тестов.

Разработанная система представляет собой решение для автоматизации тестирования в процессе непрерывной интеграции. Использование LLM и концепции RAG позволило создать инструмент, способный генерировать тесты с учетом изменений в коде, что повышает точность и актуальность тестирования. Проведенное тестирование подтвердило надежность системы и ее применимость в условиях существующих проектов.

Дальнейшее развитие системы в направлении автоматического обновления тестов, расширения типов тестирования и внедрения механизмов самообучения позволит повысить эффективность автоматизированного тестирования и обеспечить надежный процесс контроля качества программного обеспечения.

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

LLMs	Large Language Models.
RAG	Retrieval-Augmented Generation.
CI	Continuous Integration.
RNN	Рекуррентные нейронные сети.
CNN	Конволюционные нейронные сети.
GPU	Graphics Processing Unit.
TPU	Tensor Processing Unit.
ИИ	Искусственный интеллект.
ПО	Программное обеспечение.
ЯП	Язык программирования.
БД	База данных.
БЗ	База знаний.
PR	Pull Request.
DLQ	Dead Letter Queue.
АСУ	Автоматизированная система управления.

СЛОВАРЬ ТЕРМИНОВ

Вебхук — это механизм, который позволяет одной системе автоматически отправлять уведомления или данные другой системе в реальном времени, как только происходит определенное событие.

Брокер сообщений — это программный компонент, который управляет обменом сообщениями между различными сервисами или компонентами в распределенной системе. Он действует как посредник, принимая сообщения от отправителей и доставляя их получателям.

Worker-узел — это компонент распределенной системы, который выполняет заданные задачи, предоставляя вычислительные ресурсы для обработки данных или выполнения операций.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. 74% ИТ-специалистов обеспокоены устареванием своих навыков из-за ИИ. — URL: <https://www.mira-comp.ru/blog/it-specialistov-obespokoeny-ustarevaniem/> (дата обращения: 25.12.2024).
2. Шевченко А. Как адаптировать большие языковые модели под задачи бизнеса. — URL: https://habr.com/ru/companies/ru_mts/articles/837860/ (дата обращения: 08.01.2025).
3. A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions / Y. Chen [и др.] // arXiv preprint. — 2024. — arXiv: 2410.12837 [cs.CL]. — URL: <https://arxiv.org/abs/2410.12837>.
4. A Roadmap for Using Continuous Integration Environments / L. Yu [и др.] // Communications of the ACM. — 2024. — Т. 67, № 6. — С. 82—90. — DOI 10.1145/3631519. — URL: <https://dl.acm.org/doi/10.1145/3631519>.
5. Almutawa M., Ghabrah Q., Canini M. Towards LLM-Assisted System Testing for Microservices // 2024 IEEE 44th International Conference on Distributed Computing Systems Workshops (ICDCSW). — 2024. — С. 29—34. — DOI 10.1109/ICDCSW63686.2024.00011.
6. Are We Testing or Being Tested? Exploring the Practical Applications of Large Language Models in Software Testing / R. Santos [и др.] // 2024 IEEE Conference on Software Testing, Verification and Validation (ICST). — 2024. — С. 353—360. — DOI 10.1109/ICST60714.2024.00039.
7. Automated Unit Test Improvement using Large Language Models at Meta / G. Neglia [и др.] // arXiv preprint. — 2024. — arXiv: 2402.09171 [cs.SE]. — URL: <https://arxiv.org/pdf/2402.09171>.
8. Borg M. Trust Calibration in IDEs: Paving the Way for Widespread Adoption of AI // arXiv preprint. — 2024.
9. Dense Passage Retrieval for Open-Domain Question Answering / V. Karpukhin [и др.] // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. — Association for Computational Linguistics, 2020. — С. 6769—6781. — DOI 10.18653/v1/2020.emnlp-main.550. — URL: <https://doi.org/10.18653/v1/2020.emnlp-main.550> (дата обращения: 25.12.2024).
10. Evaluating Large Language Models Trained on Code / M. Chen [и др.] // arXiv preprint. — 2021. — URL: <https://arxiv.org/abs/2107.03374> (дата обращения: 25.12.2024).

11. From Prompt Injections to SQL Injection Attacks: How Protected is Your LLM-Integrated Web Application? / R. Pedro [и др.] // arXiv preprint. — 2023.
12. Language Models are Few-Shot Learners / T. B. Brown [и др.] // arXiv preprint. — 2020. — arXiv: 2005.14165 [cs.CL]. — URL: <https://arxiv.org/abs/2005.14165>.
13. LLM for Test Script Generation and Migration: Challenges, Capabilities, and Opportunities / S. Yu [и др.] // 2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS). — 2023. — С. 206—217. — DOI 10.1109/QRS60937.2023.00029.
14. Model Editing at Scale Leads to Gradual and Catastrophic Forgetting / T. Mitchell [и др.] // arXiv preprint. — 2024. — arXiv: 2401.07453 [cs.CL]. — URL: <https://arxiv.org/abs/2401.07453>.
15. On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? / E. M. Bender [и др.] // Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT). — New York, NY, USA: Association for Computing Machinery, 2021. — С. 610—623. — DOI 10.1145/3442188.3445922. — URL: <https://doi.org/10.1145/3442188.3445922> (дата обращения: 25.12.2024).
16. Plug and Play Language Models: A Simple Approach to Controlled Text Generation / S. Dathathri [и др.] // arXiv preprint. — 2020. — arXiv: 2005.11401 [cs.CL]. — URL: <https://arxiv.org/abs/2005.11401>.
17. Procko T. T., Ochoa O. Graph Retrieval-Augmented Generation for Large Language Models: A Survey // 2024 Conference on AI, Science, Engineering, and Technology (AIxSET). — 2024. — С. 166—169. — DOI 10.1109/AIxSET62544.2024.00030.
18. Research S. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Model for Programming // arXiv preprint. — 2021.
19. Smith J., Doe J., Brown M. Adaptive Control of Retrieval-Augmented Generation for Large Language Models Through Reflective Tags // Electronics. — 2023. — Т. 13, № 23. — С. 4643. — DOI 10.3390/electronics13234643. — URL: <https://www.mdpi.com/2079-9292/13/23/4643>.
20. Strubell E., Ganesh A., McCallum A. Energy and Policy Considerations for Deep Learning in NLP // Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL). — Florence, Italy: Association for Computational Linguistics, 2019. — С. 3645—3650. — DOI 10.18653/v1/P19-1355. — URL: <https://aclanthology.org/P19-1355> (дата обращения: 25.12.2024).

21. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents / N. Mündler [и др.] // arXiv preprint. — 2024. — arXiv: 2406.12952 [cs.SE]. — URL: <https://arxiv.org/pdf/2406.12952> (дата обращения: 08.01.2025).
22. Towards Continual Knowledge Learning of Language Models / S. Wang [и др.] // arXiv preprint. — 2021. — arXiv: 2110.03215 [cs.CL]. — URL: <https://arxiv.org/abs/2110.03215>.
23. Использование искусственного интеллекта для автоматизации тестирования: пер. с рус. / В. П. Часовских [и др.] // Управление образованием: теория и практика. — 2024. — Т. 14, вып. 6—1. — С. 173—180. — DOI 10.25726/m9928-2647-5498-h. — URL: <https://emreview.ru/index.php/emr/article/view/1646/1328> (дата обращения: 25.12.2024).
24. Куркин А. Практики DevOps и CI/CD в разработке. — Москва: Издательство Питер, 2020. — С. 65—75, 80—100.
25. Apache JMeter: Load Testing Tool. — URL: <https://jmeter.apache.org/> (visited on 22.01.2025).
26. Attention is All You Need / A. Vaswani [et al.] // Proceedings of the 31st International Conference on Neural Information Processing Systems / ed. by I. Guyon [et al.]. — 2017. — Vol. 12. — URL: https://papers.nips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf (visited on 25.12.2024).
27. BLOOM: BigScience Large Open-Science Open-Access Multilingual Language Model / T. Le Scao [et al.]. — URL: <https://huggingface.co/bigscience/bloom> (visited on 08.01.2025).
28. Brittain B. OpenAI, Microsoft want court to toss lawsuit accusing them of abusing open-source code. — URL: <https://www.reuters.com/legal/litigation/openai-microsoft-want-court-toss-lawsuit-accusing-them-abusing-open-source-code-2023-01-27/> (visited on 25.12.2024).
29. CodeGeeX4: Large Code Generation Model / Q. Zheng [et al.]. — URL: <https://github.com/THUDM/CodeGeeX4> (visited on 08.01.2025).
30. CodeGemma: Lightweight Models for Programming Tasks / H. Zhao [et al.]. — URL: <https://ollama.com/library/codegemma> (visited on 22.01.2025).
31. CodeGen2 3.7B P: Pretrained Code Generation Model / E. Nijkamp [et al.]. — URL: https://huggingface.co/Salesforce/codegen2-3_7B_P (visited on 08.01.2025).
32. CodeLlama-13B-HF: Pretrained Code Generation Model / B. Rozière [et al.]. — URL: <https://huggingface.co/codellama/CodeLlama-13b-hf> (visited on 08.01.2025).

33. Codestral: Advanced Model for Code Generation and Understanding / A. Jiang [et al.]. — URL: <https://huggingface.co/mistralai/Mamba-Codestral-7B-v0.1> (visited on 22.01.2025).
34. CodeT5: Open Code Large Language Models / J. Li [et al.]. — URL: <https://www.salesforce.com/blog/codet5-open-code-large-language-models/> (visited on 08.01.2025).
35. *Coles H.* Pitest: Mutation Testing for Java. — URL: <https://pitest.org/> (visited on 22.01.2025).
36. DeciCoder-6B: Pretrained Code Generation Model. — URL: <https://huggingface.co/Deci/DeciCoder-6B> (visited on 08.01.2025).
37. Falcon 180B: Large Language Model / E. Almazrouei [et al.]. — URL: <https://huggingface.co/tiiuae/falcon-180B> (visited on 08.01.2025).
38. *Fowler M.* Continuous Integration. — URL: <https://martinfowler.com/articles/continuousIntegration.html> (visited on 25.12.2024).
39. GitHub Copilot: Your AI Pair Programmer. — URL: <https://github.com/features/copilot> (visited on 08.01.2025).
40. *Haywood S., Warren G.* Understanding Tokens. — URL: <https://learn.microsoft.com/en-us/dotnet/ai/conceptual/understanding-tokens> (visited on 08.01.2025).
41. *Hoffmann M. R., Janiczak B., Mandrikov E.* JaCoCo: Java Code Coverage Library. — URL: <https://www.eclemma.org/jacoco/> (visited on 22.01.2025).
42. JavaParser: Parse, Analyze and Transform Java Code / J. V. Gesser [et al.]. — URL: <https://github.com/javaparser/javaparser> (visited on 08.01.2025).
43. JetBrains AI Assistant. — URL: <https://www.jetbrains.com/ai/> (visited on 22.01.2025).
44. JGit: Java Implementation of Git / S. Pearce [et al.]. — URL: <https://www.eclipse.org/jgit/> (visited on 08.01.2025).
45. JUnit 5 / S. Bechtold [et al.]. — URL: <https://junit.org/junit5/> (visited on 22.01.2025).
46. *Kane A.* Introducing Postbot: Postman's New AI Assistant. — URL: <https://blog.postman.com/introducing-postbot-postmans-new-ai-assistant/> (visited on 08.01.2025).
47. *Kawaguchi K., Collignon A., Verdelhan M. de.* GitHub API for Java: A Library to Interact with GitHub. — 2025. — URL: <https://github-api.kohsuke.org/> (visited on 08.01.2025).

48. *Liapunova Y.* Spring PetClinic. — 2025. — URL: <https://github.com/YulyaLiapunova/spring-petclinic> (visited on 22.01.2025).
49. *Liapunova Y., Parkhomenko V.* RAG API Test Generator. — 2025. — URL: <https://github.com/YulyaLiapunova/rag-api-test-generator> (visited on 22.01.2025).
50. Llama 3.2 Documentation. — URL: <https://ollama.com/library/llama3.2> (visited on 22.01.2025).
51. Milvus: The Open-Source Vector Database. — 2025. — URL: <https://milvus.io/> (visited on 08.01.2025).
52. MultiPL-E: A Scalable Benchmark for Multilingual Programming Language Understanding / F. Cassano [et al.]. — URL: <https://nuprl.github.io/MultiPL-E> (visited on 08.01.2025).
53. Nomic Embed Text: Advanced Text Embedding Library. — URL: <https://ollama.com/library/nomic-embed-text> (visited on 08.01.2025).
54. OpenAI / E. Musk [et al.]. — URL: <https://openai.com/> (visited on 25.12.2024).
55. *Puente Viejo D.* The Science of Control: How Temperature, Top_p, and Top_k Shape Large Language Models. — URL: <https://medium.com/%40daniel.puenteviejo/the-science-of-control-how-temperature-top-p-and-top-k-shape-large-language-models-853cb0480dae> (visited on 10.02.2025).
56. Replit Code v1.5 - CodeGPT Model. — URL: https://ollama.com/codegpt/replit-code-v1_5-3b (visited on 08.01.2025).
57. SantaCoder: Pretrained Model for Code Generation / L. Ben Allal [et al.]. — URL: <https://huggingface.co/bigcode/santacoder> (visited on 08.01.2025).
58. Spring Boot Admin: Admin UI for managing Spring Boot applications. — URL: <https://github.com/codecentric/spring-boot-admin/> (visited on 08.01.2025).
59. Spring PetClinic: Sample Application to Demonstrate the Spring Framework. — URL: <https://github.com/spring-projects/spring-petclinic> (visited on 08.01.2025).
60. Stability AI Official Website. — URL: <https://stability.ai/> (visited on 08.01.2025).
61. StarCoder 2 - Library / A. Lozhkov [et al.]. — URL: <https://ollama.com/library/starcoder2> (visited on 08.01.2025).
62. *Truell M., Asif S., Lunnemark A.* Cursor: The AI Code Editor. — URL: <https://www.cursor.com/> (visited on 10.02.2025).

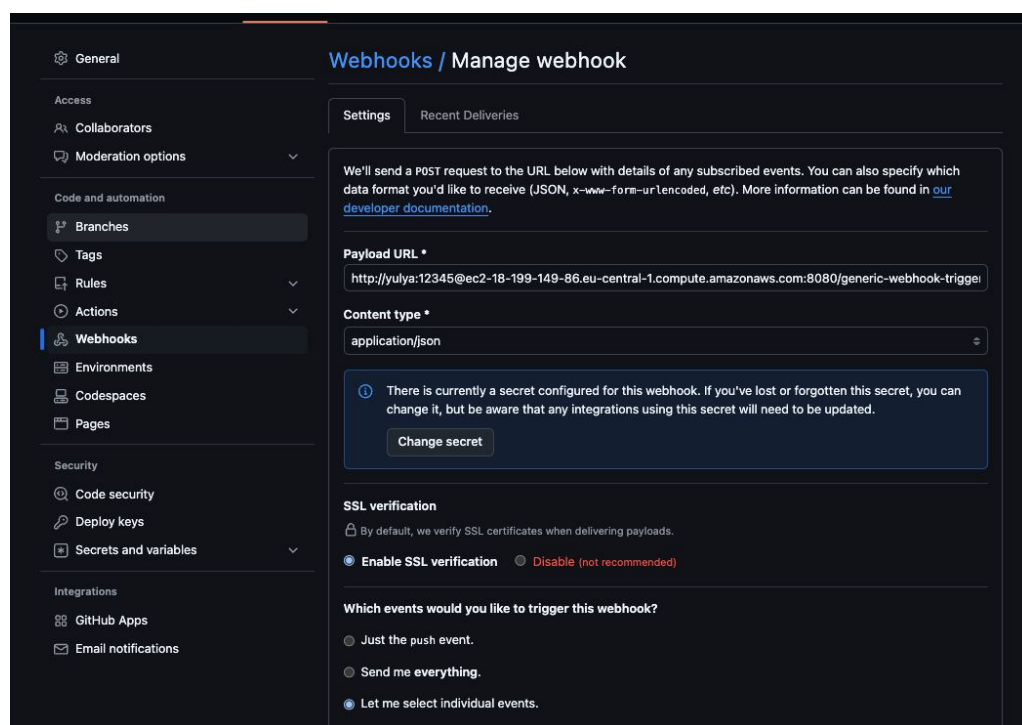
63. Unleashing Developer Productivity with Generative AI / B. Karaci Deniz [et al.]. — URL: <https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/unleashing-developer-productivity-with-generative-ai> (visited on 08.01.2025).
64. YaLM 100B: Large Language Model by Yandex. — URL: <https://github.com/yandex/YaLM-100B> (visited on 08.01.2025).
65. Zencoder: The Most Intuitive AI Coding Agent. — URL: <https://zencoder.ai/> (visited on 10.02.2025).

Приложение 1

Руководство по настройке и запуску разработанной системы

Для использования разработанной системы в своем проекте, расположенном в GitHub [49], необходимо иметь сервер с настроенным CI, например Jenkins. Рекомендуемые требования для сервера: 2 CPU, 2 Gb RAM, 50Gb памяти.

a)



b)

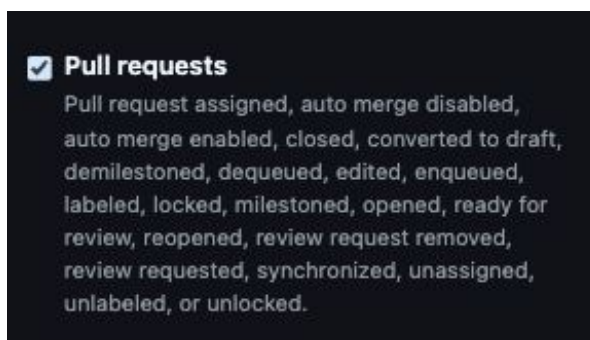


Рис.П1.1. Настройка вебхука GitHub (часть 1), включая: *a* — настройка действия; *b* — тип события

Далее настраивается отправка событий о создании MR в CI сервер на стороне GitHub с помощью вэб-хуков рис.П1.1. Настройка должна включать адрес сервера CI с маршрутом и название события, на котором запускается вэб-хук. Для Jenkins маршрутом является «/generic-webhook-trigger/invoke».

На стороне CI инструмента необходимо создать два пайплайна. Первый пайплайн (см. подраздел П2.15) принимает событие о созданном MR в GitHub с кодом, который нужно покрыть тестами. В конфигурационном файле необходимо изменить URL репозитория GitHub, который отправляет событие, и добавить адрес развернутой системы генерации тестов на базе RAG. Второй пайплайн (см. подраздел П2.15) получает событие о созданном MR в GitHub со сгенерированными тестами и оповещает первый пайплайн о том, что можно запускать прогон тестов для репозитория.

Система генерации тестов на базе RAG может быть запущена на том же сервере, где находится CI инструмент, либо отдельно. Разворачивание инфраструктуры для разработанной системы возможно с помощью docker-compose файла (см. подраздел П2.16). Дополнительно на сервер необходимо загрузить Ollama клиент и скачать языковые модели, которые будут использоваться для генерации тестов. По умолчанию в коде используются языковые модели «nomic-embed-text», необходимая для векторизации контента, который сохраняется в базе данных знаний, и по которой ищутся релевантные данные в базе данных знаний, и языковая модель «codestral». При отличных от языковых моделей по умолчанию необходимо внести изменения в классах LlmConfig и EmbeddingService AI внутри кода для корректного использования во время работы разработанной системы.

Дополнительно, на стороне GitHub нужно выпустить токен взаимодействия с аккаунтом по API с правами на внесение изменений в репозитории аккаунта. Это необходимо для создания MR самой системой генерации тестов на базе RAG. И после этого добавить выпущенный токен в переменные среды системой генерации тестов.

После выполнения всех выше перечисленных действий возможно использования разработанной системы для своих репозиториев. Для того, чтобы наполнить базу знаний, можно использовать API-маршрут POST /upload-knowledge-file, который положит данные файла в базу данных. Для генерации тестов и создания MR со сгенерированными тестами в репозиторий будет использовать эндпоинт POST /generate-tests, который имеется в конфигурационном файле Jenkins. Для корректной нарезки файла на самостоятельные части, необходимые для контекста, нужно вставить разделительные символы: #START_SECTION и #END_SECTION.

Помимо требований к инфраструктуре, необходимо соблюдать ограничения для проектов, для которых разработанная система генерации тестов будет

использоваться. Требования к проектам: использовать SpringBoot версии 3+ и Java 17.

Таблица П1.1

Сравнительная таблица моделей для генерации программного кода

Модель	Кол-во параметров	Поддерживаемые ЯП	Особенности	Лицензия	MultiPL-E Performance (pass@1; pass@10; pass@100)
StableCode [60] от Stability AI	3B	C, C++, Java, JavaScript, CSS, Go, HTML, Ruby, Rust, Markdown, Shell, Php, Sql, R, Typescript, Python, Jupyter-Clean, RestructuredText	Модель ориентирована на генерацию кода. Имеется версия для автодополнения и объяснения кода.	Бесплатное членство для физических лиц, исследователей и компаний с доходом менее \$1 млн в рамках некоммерческого и ограниченного коммерческого использования. Профессиональное членство для стартапов и компаний с доходом менее \$1 млн в рамках коммерческого использования — \$20 в месяц. Корпоративное членство для крупных компаний с доходом более \$1 млн в рамках масштабного коммерческого использования — стоимость индивидуальная по согласованию.	32.1%; —; —
StarCoder2 [61] от BigCode Project	3B, 7B, 15B	Более чем 80 ЯП, включая те, которые покрываются StableCode	Поддерживает адаптацию через fine-tuning, показывает отличные результаты на HumanEval.	Бесплатное некоммерческое использование для личных или исследовательских целей без ограничений, а также коммерческое использование при условии соблюдения ограничений, указанных в лицензировании.	20.5%; —; —

Продолжение табл.П1.1

SantaCoder [57] от BigCode Project	1,1B	Python, JavaScript, Java,	Легковесная версия на основе GPT-2, сужена для работы с популярными ЯП.	Бесплатное некоммерческое использование для личных или исследовательских целей, а также коммерческое использование при соблюдении всех ограничений, перечисленных в лицензировании.	15%; 26%; 41%
CodeGeeX4 [29] от Университета Фудань	13B	Python, C++, Java, JavaScript, Go, C, C#, PHP, Ruby, Swift, TypeScript, Kotlin, Rust, Objective-C, Scala, Perl, Haskell, Lua, R, MATLAB, SQL, Shell, HTML/CSS		Бесплатное некоммерческое использование для академических исследований или образовательных целей с указанием авторства и включение лицензии в любые копии или существенные части программного обеспечения. Коммерческое использование по согласованию при соблюдении всех условий лицензии, включая запрет на военное и незаконное использование.	58.42%; —; —
Replit Code [56] от Replit	3B	Java, JavaScript, C, PHP, Python, C++, C#, TypeScript, Go, CSS, HTML, Rust, Ruby, Swift, Scala, Shell, Lua, Perl, Haskell, JSX, Julia, Common Lisp, OCaml, Solidity, Scheme, R, Zig, SQL, Racket, D	Модель для автодополнения кода, интегрирована с Replit IDE.	Бесплатное некоммерческое и коммерческое использование при указании авторства и соблюдения условий лицензии.	23.6%; —; —

Продолжение табл.П1.1

CodeT5 [34] от Salesforce Research	220M, 770M, 2B, 6B, 16B	Ruby, JavaScript, Python, Java, PHP, C, C++, C#, Go	Способна на генерацию кода, исправление ошибок, рефакторинг. Показывает отличные результаты в задачах перевода между ЯП.	Бесплатное некоммерческое использование для образовательных, исследовательских и личных целей без ограничений, а также коммерческое использование с соблюдением лицензионного соглашения, включая сохранение уведомления об авторских правах и запрет на использование имени Salesforce для продвижения.	28.0%; 47.2%; 69.8%
CodeGen2 [31] от Salesforce Research	1B, 3,7B, 7B, 16B	C, C++, C#, Dart, Go, Java, Javascript, Kotlin, Lua, PHP, Python, Ruby, Rust, Scala, Shell, Sql, Swift, Typescript, Vue	Расширенное семейство моделей, адаптированных для профессиональной генерации кода, интеграции в IDE и работы в масштабируемых приложениях.	Бесплатное некоммерческое использование для образовательных, исследовательских, личных целей и разработки открытого ПО с соблюдением лицензионного соглашения, а также коммерческое использование с аналогичными условиями.	18.83%; 31.78%; 50.41
DeciCoder [36]	7B, 13B, 34B	Python, Java, Javascript, Rust, C++, C, C#	Легковесная модель для завершения кода, оптимизирована по производительности и памяти. Показывает повышение производительности в задачах HumanEval на 3,5 раза.	Открытая. Можно использовать для коммерческих продуктов без ограничений.	16.7%; —; —
Code LLaMA [32]	7B, 13B, 34B, 70B	Python, C++, Java, PHP, C#, TypeScript, Bash	Дообучена для работы с кодом на основе LLaMA 2. Показывает результаты на задачах перевода и генерации кода на популярных ЯП.	Бесплатно для использования, как коммерческого, так и некоммерческого, при условии соблюдения политики приемлемого использования.	12.2%; 25.2%; 44.4%

Продолжение табл.П1.1

CodeGemma от Google [30]	2B, 7B	C++, C#, Go, Java, JavaScript, Kotlin, Python, Rust	Легковесная модель для выполнения различных задач, связанных с программированием. Помимо обычного автодополнения, имеет функция заполнения середины (генерировать код в пропущенной части между заданным началом и концом).	Бесплатно для любого вида использования. Для некоммерческого использования нет ограничений, кроме запретов на запрещенные области применения. Для коммерческого использования можно применять внутри компании или в составе продуктов, но нельзя распространять саму модель или предлагать API-доступ.	41%; 48.4%; 54.7%
Codestral Mamba от Mistral AI[33]	7B	Более чем 80 ЯП, включая те, которые покрываются CodeGemma и Code LLaMA	Генеративная AI-модель для задач, связанных с программированием. Аналогично CodeGemma имеет функцию заполнения середины. Интегрирована в такие инструменты, как LlamaIndex и LangChain, а также доступна через плагины для VSCode и JetBrains.	Бесплатное некоммерческое и коммерческое использование при указании авторства и соблюдения условий лицензии, так как выпускается под лицензией Apache 2.0.	44.6%; 58.4%; 63.3%

Код разработанной системы

П2.1. Класс WebhookController.java

```
5  @RestController
   @RequestMapping("/webhook")
   public class WebhookController {
       private static final Logger logger = LoggerFactory.getLogger
           (WebhookController.class);
       private final RabbitTemplate rabbitTemplate;

       @Autowired
       public WebhookController(RabbitTemplate rabbitTemplate) {
           this.rabbitTemplate = rabbitTemplate;
       }

       private void validatePayload(WebhookPayload payload) {
           if (payload == null || payload.getRepository() == null ||
               payload.getAction() == null) {
15          throw new IllegalArgumentException("Invalid payload
               received");
           }
           logger.info("Processing PR {} from repository URL {}",
               payload.getNumber(), payload.getRepository().getCloneUrl()
               );
       }

20      private QueueEvent buildQueueEvent(WebhookPayload payload) {
           QueueEvent queueEvent = new QueueEvent();
           queueEvent.setType(payload.getAction());
           queueEvent.setPullRequestId(payload.getNumber());
25          queueEvent.setRepositoryUrl(payload.getRepository().
               getCloneUrl());
           queueEvent.setRepositoryName(payload.getRepository().
               getFullName());
           return queueEvent;
       }

30      @PostMapping("/generate-tests")
```

```

public ResponseEntity<String> generateTests (@RequestBody
    WebhookPayload payload) {
    try {
        validatePayload(payload);
        QueueEvent queueEvent = buildQueueEvent(payload);
35         rabbitTemplate.convertAndSend(TEST_GENERATION_QUEUE,
            queueEvent);

        return ResponseEntity.ok("Event added to queue");
    } catch (Exception e) {
        logger.error("Failed to process webhook payload", e);
40         if (e instanceof IllegalArgumentException) {
            return ResponseEntity.badRequest().body("Invalid
                payload");
        }
        return ResponseEntity.status(HttpStatus.
            INTERNAL_SERVER_ERROR)
            .body("An error occurred while processing the request");
45     }
}

@PostMapping("/upload-knowledge-file")
public ResponseEntity<String> uploadKnowledgeFile(
50     @RequestParam("file") MultipartFile file ,
    @RequestParam("fileName") String fileName ,
    @RequestParam("documentType") String documentType
) {
    try {
55         String content = new String(file.getBytes());
        Map<String , Object> message = new HashMap<>();
        message.put("documentType", documentType);
        message.put("fileContent", content);
        message.put("fileName", fileName);
60         rabbitTemplate.convertAndSend(
            KNOWLEDGE_BASE_UPDATE_QUEUE, message);

        return ResponseEntity.ok("File uploaded and processing
            started.");
    } catch (IOException e) {
        return ResponseEntity.status(500).body("Error processing
            file: " + e.getMessage());
65     }
}

```

```
| }
```

П2.2. Класс RabbitMQConfig.java

```

public class RabbitMQConfig {
    public static final String TEST_GENERATION_QUEUE = "test -
        generation -queue";
    public static final String KNOWLEDGE_BASE_UPDATE_QUEUE = "
        knowledge -base -update -queue";
5    public static final String TEST_GENERATION_DLQ = "test -
        generation -dlq";
    public static final String KNOWLEDGE_BASE_UPDATE_DLQ = "
        knowledge -base -update -dlq";
    public static final String DEAD_LETTER_EXCHANGE = "dead -
        letter -exchange";

    @Bean
10    public Jackson2JsonMessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }

    @Bean
15    public RabbitTemplate rabbitTemplate(ConnectionFactory
        connectionFactory, Jackson2JsonMessageConverter
        jsonMessageConverter) {
        RabbitTemplate template = new RabbitTemplate(
            connectionFactory);
        template.setMessageConverter(jsonMessageConverter);
        return template;
    }

20    @Bean
    public DirectExchange deadLetterExchange() {
        return new DirectExchange(DEAD_LETTER_EXCHANGE);
    }

25    @Bean
    public Queue testGenerationDLQ() {
        return QueueBuilder.durable(TEST_GENERATION_DLQ).build();
    }

30    @Bean

```

```

public Queue knowledgeBaseUpdateDLQ () {
    return QueueBuilder.durable (KNOWLEDGE_BASE_UPDATE_DLQ) .
        build ();
}

35
@Bean
public Queue testGenerationQueue () {
    return QueueBuilder.durable (TEST_GENERATION_QUEUE)
        .withArgument ("x-dead-letter-exchange",
            DEAD_LETTER_EXCHANGE)
40
        .withArgument ("x-dead-letter-routing-key",
            TEST_GENERATION_DLQ)
        .build ();
}

@Bean
45
public Queue knowledgeBaseUpdateQueue () {
    return QueueBuilder.durable (KNOWLEDGE_BASE_UPDATE_QUEUE)
        .withArgument ("x-dead-letter-exchange",
            DEAD_LETTER_EXCHANGE)
        .withArgument ("x-dead-letter-routing-key",
            KNOWLEDGE_BASE_UPDATE_DLQ)
        .build ();
50
}

@Bean
public Binding testGenerationDLQBinding (Queue
    testGenerationDLQ, DirectExchange deadLetterExchange) {
    return BindingBuilder.bind (testGenerationDLQ) .to (
        deadLetterExchange) .with (TEST_GENERATION_DLQ);
55
}

@Bean
public Binding knowledgeBaseUpdateDLQBinding (Queue
    knowledgeBaseUpdateDLQ, DirectExchange deadLetterExchange
    ) {
    return BindingBuilder.bind (knowledgeBaseUpdateDLQ) .to (
        deadLetterExchange) .with (KNOWLEDGE_BASE_UPDATE_DLQ);
60
}

}

@ExtendWith (MockitoExtension.class)
class RabbitMQConfigTest {

```



```

5  @InjectMocks
   private RabbitMQConfig rabbitMQConfig;

   @Mock
   private ConnectionFactory connectionFactory;

10  @Test
   void testJsonMessageConverterBean() {
       Jackson2JsonMessageConverter converter = rabbitMQConfig.
           jsonMessageConverter();
       assertNotNull(converter, "Jackson2JsonMessageConverter bean
           should not be null");
15  }

   @Test
   void testRabbitTemplateBean() {
       Jackson2JsonMessageConverter converter = new
           Jackson2JsonMessageConverter();

20       RabbitTemplate template = rabbitMQConfig.rabbitTemplate(
           connectionFactory, converter);

       assertNotNull(template, "RabbitTemplate bean should not be
           null");
       assertEquals(converter, template.getMessageConverter(), "
           MessageConverter should match");
25  }

   @Test
   void testDeadLetterExchangeBean() {
       DirectExchange exchange = rabbitMQConfig.deadLetterExchange
           ();
30       assertNotNull(exchange, "DeadLetterExchange bean should not
           be null");
       assertEquals(RabbitMQConfig.DEAD_LETTER_EXCHANGE, exchange.
           getName(), "Exchange name should match");
   }

   @Test
35  void testQueuesCreation() {
       Queue testGenerationQueue = rabbitMQConfig.
           testGenerationQueue();

```

```

Queue knowledgeBaseUpdateQueue = rabbitMQConfig.
    knowledgeBaseUpdateQueue();
Queue testGenerationDLQ = rabbitMQConfig.testGenerationDLQ();
;
Queue knowledgeBaseUpdateDLQ = rabbitMQConfig.
    knowledgeBaseUpdateDLQ();
40
assertNotNull(testGenerationQueue, "TestGenerationQueue
    should not be null");
assertNotNull(knowledgeBaseUpdateQueue, "
    KnowledgeBaseUpdateQueue should not be null");
assertNotNull(testGenerationDLQ, "TestGenerationDLQ should
    not be null");
assertNotNull(knowledgeBaseUpdateDLQ, "
    KnowledgeBaseUpdateDLQ should not be null");
45 }

@Test
void testBindings() {
    Queue testGenerationDLQ = new Queue(RabbitMQConfig.
        TEST_GENERATION_DLQ);
50 Queue knowledgeBaseUpdateDLQ = new Queue(RabbitMQConfig.
        KNOWLEDGE_BASE_UPDATE_DLQ);
    DirectExchange deadLetterExchange = new DirectExchange(
        RabbitMQConfig.DEAD_LETTER_EXCHANGE);

    Binding testGenDLQBinding = rabbitMQConfig.
        testGenerationDLQBinding(testGenerationDLQ,
            deadLetterExchange);
    Binding knowledgeBaseUpdateDLQBinding = rabbitMQConfig.
        knowledgeBaseUpdateDLQBinding(knowledgeBaseUpdateDLQ,
            deadLetterExchange);
55
assertNotNull(testGenDLQBinding, "TestGenerationDLQBinding
    should not be null");
assertNotNull(knowledgeBaseUpdateDLQBinding, "
    KnowledgeBaseUpdateDLQBinding should not be null");
assertEquals(RabbitMQConfig.TEST_GENERATION_DLQ,
    testGenDLQBinding.getRoutingKey(), "Routing key should
    match for TestGenerationDLQ");
assertEquals(RabbitMQConfig.KNOWLEDGE_BASE_UPDATE_DLQ,
    knowledgeBaseUpdateDLQBinding.getRoutingKey(), "Routing
    key should match for KnowledgeBaseUpdateDLQ");

```

```

60 | }
    | }

```

П2.3. Класс QueueListener.java

```

public class QueueListener {
    private static final Logger logger = LoggerFactory.getLogger
        (QueueListener.class);
    private final TestGenerationWorker testGenerationWorker;
5   private final KnowledgeUpdatingWorker knowledgeWorker;

    public QueueListener(TestGenerationWorker
        testGenerationWorker, KnowledgeUpdatingWorker
        knowledgeWorker) {
        this.testGenerationWorker = testGenerationWorker;
        this.knowledgeWorker = knowledgeWorker;
10  }

    @RabbitListener(queues = TEST_GENERATION_QUEUE)
    public void handleTestGenerationEvent(QueueEvent event) {
        logger.info("Received Test Generation Event: {}", event.
            toString());
15    try {
        validateEvent(event);
        testGenerationWorker.processTestGeneration(event);
    } catch (Exception e) {
        logger.error("Error processing Test Generation Event:
            {}", e.getMessage(), e);
20    throw new AmqpRejectAndDontRequeueException(e);
    }
}

    @RabbitListener(queues = KNOWLEDGE_BASE_UPDATE_QUEUE)
25 public void handleKnowledgeUpdateEvent(Map<String, Object>
    event) {
        logger.info("Received Knowledge Update Event: {}", event);
        try {
            // validateEvent(event);
            knowledgeWorker.processKnowledgeBaseUpdate(event);
30    } catch (Exception e) {
        logger.error("Error processing Knowledge Update Event:
            {}", e.getMessage(), e);

```

```

        throw new AmqpRejectAndDontRequeueException(e);
    }
}

35
    @RabbitListener(queues = TEST_GENERATION_DLQ)
    public void handleTestGenerationDLQ(Message message) {
        String rawMessage = new String(message.getBody());
        System.out.println("Failed Test Generation Event: " +
40         rawMessage);
    }

    @RabbitListener(queues = KNOWLEDGE_BASE_UPDATE_DLQ)
    public void handleKnowledgeBaseUpdateDLQ(Message message) {
        String rawMessage = new String(message.getBody());
45         System.out.println("Failed Knowledge Update Event: " +
            rawMessage);
    }

    private void validateEvent(@NotNull QueueEvent event) {
        if (event.getRepositoryUrl() == null || event.
            getRepositoryUrl().isEmpty()) {
50         throw new IllegalArgumentException("Repository is
            missing in the event");
        }
        if (event.getPullRequestId() <= 0) {
            throw new IllegalArgumentException("Invalid Pull Request
                ID in the event");
        }
55     }
}

    @ExtendWith(MockitoExtension.class)
    public class QueueListenerTest {

5        @Mock
        private TestGenerationWorker testGenerationWorker;

        @Mock
        private KnowledgeUpdatingWorker knowledgeWorker;
10

        @InjectMocks
        private QueueListener queueListener;

        @Test

```

```

15 void handleTestGenerationEvent_invalidEvent_throwsException
    () {
    // Arrange
    QueueEvent invalidEvent = new QueueEvent();

    // Act & Assert
20 AmqpRejectAndDontRequeueException exception = assertThrows
    (AmqpRejectAndDontRequeueException.class, () ->
    queueListener.handleTestGenerationEvent(invalidEvent)
    );

    // Assert
25 Throwable cause = exception.getCause();
    assertNotNull(cause);
    assertEquals("Repository is missing in the event", cause.
        getMessage());
    }

30 @SneakyThrows
    @Test
    void
        handleTestGenerationEvent_exceptionDuringProcessing_logsError
        () {
        // Arrange
        QueueEvent event = new QueueEvent();
35 event.setRepositoryUrl("https://github.com/example/repo");
        event.setPullRequestId(1);
        doThrow(new RuntimeException("Processing error")).when(
            testGenerationWorker).processTestGeneration(event);

        // Act & Assert
40 AmqpRejectAndDontRequeueException exception = assertThrows
        (AmqpRejectAndDontRequeueException.class, () ->
        queueListener.handleTestGenerationEvent(event)
        );
        assertNotNull(exception.getCause());
        assertEquals("Processing error", exception.getCause().
            getMessage());
45 }
    }

```

П2.4. Класс EmbeddingService.java

```

@Service
public class EmbeddingService {
    private final OllamaEmbeddingModel embeddingModel;

    public EmbeddingService(OllamaEmbeddingModel
        ollamaEmbeddingModel) {
        this.embeddingModel = ollamaEmbeddingModel;
    }

    public EmbeddingResponse getEmbedding(String input) {
        if (input == null || input.isEmpty()) {
            throw new IllegalArgumentException("Input string cannot
                be null or empty");
        }

        EmbeddingRequest embeddingRequest = new EmbeddingRequest(
            List.of(input),
            OllamaOptions.builder()
                .withModel(OllamaModel.NOMIC_EMBED_TEXT.id())
                .withTruncate(false)
                .build()
        );

        try {
            EmbeddingResponse response = this.embeddingModel.call(
                embeddingRequest);

            if (response == null || response.getResult() == null) {
                throw new RuntimeException("Invalid response from
                    embedding model: null");
            }

            return response;
        } catch (Exception e) {
            throw new RuntimeException("Error during embedding model
                call: " + e.getMessage(), e);
        }
    }
}

public class EmbeddingServiceTest {

```

```

@Mock
private OllamaEmbeddingModel mockEmbeddingModel;

@InjectMocks
private EmbeddingService embeddingService;

@BeforeEach
void setUp() {
    MockitoAnnotations.openMocks(this);
}

@Test
void testGetEmbedding_ThrowsExceptionForEmptyInput() {
    // Arrange
    String input = "";

    // Act & Assert
    IllegalArgumentException exception = assertThrows(
        IllegalArgumentException.class,
        () -> embeddingService.getEmbedding(input)
    );
    assertEquals("Input string cannot be null or empty",
        exception.getMessage());
}

@Test
void testGetEmbedding_ThrowsExceptionForModelCallError() {
    // Arrange
    String input = "test input";

    when(mockEmbeddingModel.call(any())) .thenThrow(new
        RuntimeException("Model call failed"));

    // Act & Assert
    RuntimeException exception = assertThrows(
        RuntimeException.class,
        () -> embeddingService.getEmbedding(input)
    );
    assertEquals("Error during embedding model call: Model
        call failed", exception.getMessage());
}

@Test

```



```

45 void testGetEmbedding_ThrowsExceptionForNullResponse() {
    // Arrange
    String input = "test input";

    when(mockEmbeddingModel.call(any())).thenReturn(null);

    // Act & Assert
50 RuntimeException exception = assertThrows(
    RuntimeException.class,
    () -> embeddingService.getEmbedding(input)
    );
    assertEquals("Error during embedding model call: Invalid
        response from embedding model: null", exception.
        getMessage());
55 }

@Test
void testGetEmbedding_ThrowsExceptionForNullResultInResponse
    () {
    // Arrange
60 String input = "test input";
    EmbeddingResponse response = mock(EmbeddingResponse.class)
        ;

    when(response.getResult()).thenReturn(null);
    when(mockEmbeddingModel.call(any())).thenReturn(response);
65

    // Act & Assert
    RuntimeException exception = assertThrows(
    RuntimeException.class,
    () -> embeddingService.getEmbedding(input)
70 );
    assertEquals("Error during embedding model call: Invalid
        response from embedding model: null", exception.
        getMessage());
    }
}

```

П2.5. Класс GitHubService.java

```

public class GitHubService {

```

```

private static final Logger logger = LoggerFactory.getLogger
    (GitHubService.class);
private static final String REMOTE_NAME = "origin";
5 private static final String DEFAULT_BRANCH = "main";

@Value("${github.access.token}")
private String githubAccessToken;

10 @Value("${repository.base.path}")
private String repositoryBasePath;

private final GitHub gitHub;
private final GitCredentialsProvider credentialsProvider;

15 public GitHubService(@Value("${github.access.token}") String
    githubAccessToken) throws IOException {
    this.githubAccessToken = githubAccessToken;
    this.gitHub = GitHub.connectUsingOAuth(githubAccessToken);
    this.credentialsProvider = new GitCredentialsProvider(
        githubAccessToken);
20 validateConnection();
}

private void validateConnection() throws IOException {
    if (!gitHub.isCredentialValid()) {
25 throw new IOException("Invalid GitHub credentials");
    }
    logger.info("Successfully connected to GitHub");
}

30 public List<EndpointCodeDetails> analyzePullRequestChanges(
    String repositoryUrl, String repositoryName, int
    pullRequestId) {
    try {
        GHPullRequest pullRequest = getPullRequest(
            repositoryName, pullRequestId);
        String baseBranch = pullRequest.getBase().getRef();
        String headBranch = pullRequest.getHead().getRef();

35 Git git = getRepository(repositoryUrl, repositoryName);
        return analyzeChanges(git, baseBranch, headBranch);
    } catch (Exception e) {

```

```

        logger.error("Failed to analyze pull request changes", e
    );
40     throw new GitHubOperationException("Failed to analyze
        pull request changes", e);
    }
}

private Git getRepository(String repositoryUrl, String
    repositoryName) throws GitAPIException, IOException {
45     Path localPath = getLocalRepositoryPath(repositoryName);
    return cloneOrUpdateRepository(repositoryUrl, localPath.
        toString());
}

private Path getLocalRepositoryPath(String repositoryName) {
50     String repoName = extractRepositoryName(repositoryName);
    return Paths.get(repositoryBasePath, repoName);
}

private String extractRepositoryName(String repositoryUrl) {
55     return repositoryUrl.substring(repositoryUrl.lastIndexOf
        ('/') + 1).replace(".git", "");
}

private Git cloneOrUpdateRepository(String repoUrl, String
    localPath) throws GitAPIException, IOException {
60     File repoDir = new File(localPath);

    if (isExistingRepository(repoDir)) {
        return updateExistingRepository(repoDir);
    }
    return cloneRepository(repoUrl, repoDir);
65 }

private boolean isExistingRepository(File repoDir) {
    return repoDir.exists() && new File(repoDir, ".git").
        exists();
}
70

private Git updateExistingRepository(File repoDir) throws
    GitAPIException, IOException {
    logger.info("Updating existing repository at {}", repoDir)
        ;

```

```

    Git git = Git.open(repoDir);
    git.fetch()
75    .setCredentialsProvider(credentialsProvider)
    .setRemote(REMOTE_NAME)
    .setRefSpecs(new RefSpec("+refs/heads/*:refs/remotes/
        origin/*"))
    .call();
    git.pull()
80    .setCredentialsProvider(credentialsProvider)
    .call();
    return git;
}

85 private Git cloneRepository(String repoUrl, File repoDir)
    throws GitAPIException {
    logger.info("Cloning repository from {} to {}", repoUrl,
        repoDir);
    return Git.cloneRepository()
        .setURI(repoUrl)
        .setDirectory(repoDir)
90    .setCredentialsProvider(credentialsProvider)
        .call();
}

private List<EndpointCodeDetails> analyzeChanges(Git git,
    String baseBranch, String headBranch) throws Exception {
95    checkoutBranches(git, baseBranch, headBranch);
    RevCommit baseCommit = getLatestCommit(git);
    RevCommit headCommit = switchToHeadBranch(git, headBranch)
        ;

    List<DiffEntry> diffs = getDiffs(git, baseCommit,
        headCommit);
100    return processChangedFiles(git, diffs);
}

private void checkoutBranches(Git git, String baseBranch,
    String headBranch) throws GitAPIException, IOException {
105    git.checkout().setName(baseBranch).call();
    cleanupExistingBranch(git, headBranch);
}

```

```

private RevCommit switchToHeadBranch(Git git, String
    headBranch) throws Exception {
    git.checkout()
110    .setCreateBranch(true)
    .setName(headBranch)
    .setStartPoint(REMOTE_NAME + "/" + headBranch)
    .call();
    return getLatestCommit(git);
115 }

private void cleanupExistingBranch(Git git, String
    branchName) throws GitAPIException, IOException {
    boolean branchExists = git.getRepository().getRefDatabase
        ()
120    .getRefsByPrefix("refs/heads/")
    .stream()
    .anyMatch(ref -> ref.getName().endsWith(branchName));

    if (branchExists) {
125        git.branchDelete()
        .setBranchNames(branchName)
        .setForce(true)
        .call();
        logger.info("Deleted existing branch: {}", branchName);
    }
130 }

private List<DiffEntry> getDiffs(Git git, RevCommit
    baseCommit, RevCommit headCommit) throws Exception {
    return git.diff()
135    .setOldTree(prepareTreeParser(git, baseCommit))
    .setNewTree(prepareTreeParser(git, headCommit))
    .call();
}

private List<EndpointCodeDetails> processChangedFiles(Git
    git, List<DiffEntry> diffs) throws Exception {
140    List<EndpointCodeDetails> endpoints = new ArrayList<>();
    File repoDir = git.getRepository().getWorkTree();
    String basePackagePath = BasePackageFinder.
        findBasePackagePath(repoDir);
    Map<String, Map<String, Object>> models = collectModels(
        basePackagePath);

```

```

145     for (DiffEntry diff : diffs) {
        if (isControllerFile(diff.getNewPath())) {
            processControllerFile(diff, git, repoDir, models,
                endpoints);
        }
    }
150     return endpoints;
}

private Map<String, Map<String, Object>> collectModels(
    String basePackagePath) {
    String modelsPath = basePackagePath + "/model";
155     ModelCollector modelCollector = new ModelCollector(
        modelsPath);
    return modelCollector.collectModels();
}

private boolean isControllerFile(String path) {
160     return path.endsWith("Controller.java");
}

private void processControllerFile(DiffEntry diff, Git git,
    File repoDir,
    Map<String, Map<String, Object>> models,
165     List<EndpointCodeDetails> endpoints) throws Exception {
    List<Integer> changedLines = DiffParser.getChangedLines(
        git.getRepository(), diff);
    File file = new File(repoDir, diff.getNewPath());
    CompilationUnit compilationUnit = StaticJavaParser.parse(
        file);

170     List<MethodDeclaration> endpointMethods =
        EndpointMethodLocator.findEndpointMethods(
            compilationUnit, changedLines);

    if (!endpointMethods.isEmpty()) {
        String className = extractClassName(compilationUnit,
            diff.getNewPath());
        endpoints.add(createEndpointDetails(className,
            endpointMethods, models));
175     }
}

```

```

private String extractClassName(CompilationUnit
    compilationUnit, String fallbackPath) {
    return compilationUnit.findFirst(
        ClassOrInterfaceDeclaration.class)
180    .map(ClassOrInterfaceDeclaration::getNameAsString)
    .orElse(fallbackPath);
}

private EndpointCodeDetails createEndpointDetails(String
    className,
185    List<MethodDeclaration> methods,
    Map<String, Map<String, Object>> models) {
    EndpointCodeDetails details = new EndpointCodeDetails();
    details.setClassName(className);
    details.setEndpointMethods(methods);
190    details.setModels(models);
    return details;
}

public void createPullRequestWithChanges(String
    repositoryUrl, String repositoryName, int pullRequestId,
    Map<String, String> content) {
195    try {
        String branchName = generateBranchName();
        Git git = prepareRepository(repositoryUrl, branchName);

        GHPullRequest pullRequest = getPullRequest(
            repositoryName, pullRequestId);
200        String headBranch = pullRequest.getHead().getRef();

        File repoDir = git.getRepository().getWorkTree();
        String testDirectory = BasePackageFinder.
            findBasePackageTestPath(repoDir);

205        for (var entry : content.entrySet()) {
            String testFileName = entry.getKey() + "ApiTest.java";
            TestFileWriter.writeTestFile(testDirectory,
                testFileName, entry.getValue());
        }

210        String message = "Generated tests";
        commitAndPushChanges(git, message, branchName);
    }
}

```



```

        createPullRequest(repositoryName , message , branchName ,
            headBranch);
    } catch (Exception e) {
        logger.error("Failed to create pull request", e);
215        throw new GitHubOperationException("Failed to create
            pull request", e);
    }
}

private String generateBranchName() {
220    return "feature -" + UUID.randomUUID().toString().substring
        (0, 8);
}

private Git prepareRepository(String repository , String
    branchName) throws Exception {
    Path localPath = getLocalRepositoryPath(repository);
225    Git git = cloneOrUpdateRepository(repository , localPath .
        toString());
    git.checkout().setCreateBranch(true).setName(branchName).
        call();
    return git;
}

230 private void commitAndPushChanges(Git git , String message ,
    String branchName) throws GitAPIException {
    git.add().addFilepattern(".").call();
    git.commit().setMessage(message).call();
    git.push()
235        .setRemote(REMOTE_NAME)
        .setCredentialsProvider(credentialsProvider)
        .setRefSpecs(new RefSpec("refs/heads/" + branchName))
        .call();
    logger.info("Changes pushed to branch: {}", branchName);
}

240 private void createPullRequest(String repository , String
    message , String branchName , String headBranch) throws
    IOException {
    GHRepository ghRepository = gitHub.getRepository(
        repository);
    GHPullRequest pullRequest = ghRepository.createPullRequest
        (

```

```

245     "TESTS: " + message ,
        branchName ,
        headBranch ,
        "Automated pull request\n\n" + message
    );
    logger.info(" Pull Request created: {}", pullRequest.
        getHtmlUrl());
250 }

    private GHPullRequest getPullRequest(String repository , int
        pullRequestId) throws IOException {
        return gitHub.getRepository(repository).getPullRequest(
            pullRequestId);
    }
255

    private static AbstractTreeIterator prepareTreeParser(Git
        git , RevCommit commit) throws IOException {
        try (RevWalk walk = new RevWalk(git.getRepository())) {
            return new CanonicalTreeParser(null , git.getRepository()
                .newObjectReader() , walk.parseTree(commit.getTree()))
                ;
        }
260 }

    private static RevCommit getLatestCommit(Git git) throws
        IOException {
        try (RevWalk walk = new RevWalk(git.getRepository())) {
            return walk.parseCommit(git.getRepository().resolve("
                HEAD"));
265 }
        }

    private static class GitCredentialsProvider extends
        UsernamePasswordCredentialsProvider {
        public GitCredentialsProvider(String token) {
270             super(token , "");
        }
    }

    public static class GitHubOperationException extends
        RuntimeException {
275         public GitHubOperationException(String message , Throwable
            cause) {

```

```

        super(message, cause);
    }
}

```

П2.6. Класс RAGService.java

```

public class RAGService {
    private final ChatService chatService;
    private final EmbeddingService embeddingService;
5   private final VectorStorageService vectorStorage;

    public RAGService(
        ChatService chatService,
        EmbeddingService embeddingService,
10   VectorStorageService vectorStorageService
    ) {
        this.chatService = chatService;
        this.embeddingService = embeddingService;
        this.vectorStorage = vectorStorageService;
15   }

    private Map<String, String> retrieveContext(
        EmbeddingResponse queryEmbedding) throws IOException {
        SearchResp searchResp = this.vectorStorage.searchEmbedding
            (queryEmbedding, 5);
        Map<String, String> context = new HashMap<>();
20   List<SearchResp.SearchResult> searchResult = searchResp.
            getSearchResults().get(0);
        for (SearchResp.SearchResult item : searchResult) {
            String document = item.getEntity().get("embedding").
                toString();
            String documentType = item.getEntity().get("
                document_type").toString();
            if (Objects.equals(documentType, KnowledgeUpdatingWorker
                .DocumentTypes.TESTS.name())) {
25   context.put("endpointDescription", document);
            } else {
                context.put("examples", document);
                context.put("previousTestFileContent", "");
            }
30   }
}

```

```

        return context;
    }

35 private Prompt buildTestGenerationPrompt(Map<String, String>
        context, String endpointCode) {
    StringBuilder sb = new StringBuilder();
    sb.append("You are an assistant for generating Java-based
        API tests. Generate test for endpoint below: \n\n");
    sb.append("Technical description of the new API-endpoint
        for which you have to generate tests:\n");
    sb.append("[ENDPOINT_DESCRIPTION]").append("\n\n");
40 sb.append(context.get("endpointDescription")).append("\n\n");
    sb.append("/ENDPOINT_DESCRIPTION").append("\n\n");

    sb.append("Models of endpoints to cover and entities that
        already exist. Use them if they are needed in the
        context of describing the endpoint to be covered::\n");
    sb.append("[ENDPOINT_CODE_TO_COVER]").append("\n\n");
45 sb.append(endpointCode).append("\n\n");
    sb.append("/ENDPOINT_CODE_TO_COVER").append("\n\n");

    if (!context.get("previousTestFileContent").isEmpty()) {
        sb.append("Previous test file content:\n");
50 sb.append("[PREVIOUS_TEST_FILE_CONTENT]").append("\n\n");
        ;
        sb.append(context.get("previousTestFileContent") + "\n\n");
        sb.append("/PREVIOUS_TEST_FILE_CONTENT").append("\n\n");
    } else {
        sb.append("Relevant examples of existing tests that you
            can use as a template and example when writing new
            technical documentation tests:\n");
55 sb.append("[EXAMPLE_TESTS]").append("\n\n");
        List<String> examples = Collections.singletonList(
            context.get("examples"));
        for (String ex : examples) {
            sb.append(ex).append("\n\n");
        }
60 sb.append("/EXAMPLE_TESTS").append("\n\n");
    }
}

```

```

        sb.append("Create a well-structured JUnit test class that
            covers the positive, negative, and boundary cases for
            endpoints described in the technical documentation. ");
        sb.append("Use the recommended patterns from the examples,
            describe existing entity and endpoint models if
            required, and follow best practices.\n\n");
65      sb.append("Return only the Java code placed between [
            JAVA_CODE] and [/JAVA_CODE] tags.\n");

        return new Prompt(sb.toString());
    }

70      public String generateTests(EndpointCodeDetails
            endpointCodeDetails) {
        try {
            StringBuilder sb = new StringBuilder();
            int filesCount = 0;
            for (MethodDeclaration endpoint : endpointCodeDetails.
                getEndpointMethods()) {
75              String input = "API tests for " + endpoint.getName() +
                  "endpoint in class " + endpointCodeDetails.
                      getClassName();
                EmbeddingResponse queryEmbedding = this.
                    embeddingService.getEmbedding(input);
                Map<String, String> context = this.retrieveContext(
                    queryEmbedding);
                Prompt prompt = this.buildTestGenerationPrompt(context
                    , endpointCodeDetails.serializeResults());
                ChatResponse response = this.chatService.sendRequest(
                    prompt);
80              sb.append(response.getResult().getOutput().getContent
                  () + "\n");
                filesCount++;
            }

            String tests;
85            if (filesCount > 1) {
                ChatResponse response = this.chatService.sendRequest(
                    new Prompt("Collect tests to one file and placed
                        between [JAVA_CODE] and [/JAVA_CODE] tags: \n" + sb
                            .toString()));
                tests = response.getResult().getOutput().getContent();
            }
        }
    }

```

```

    } else {
        tests = sb.toString();
    }

    int startIndex = tests.indexOf("[JAVA_CODE]") + "[
        JAVA_CODE]".length();
    int endIndex = tests.indexOf("/[JAVA_CODE]", startIndex)
        ;

    if (startIndex != -1 && endIndex != -1) {
        return tests.substring(startIndex, endIndex).trim();
    } else {
        return tests;
    }
} catch (Exception e) {
    throw new RuntimeException("Failed to generate tests: "
        + e.getMessage(), e);
}
}

public class RAGServiceTest {

    private RAGService ragService;

    @Mock
    private ChatService chatService;

    @Mock
    private EmbeddingService embeddingService;

    @Mock
    private VectorStorageService vectorStorageService;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
        ragService = new RAGService(chatService, embeddingService,
            vectorStorageService);
    }

    @Test
    void testRetrieveContext_Success() throws IOException {

```

```

EmbeddingResponse mockEmbeddingResponse = mock(
    EmbeddingResponse.class);
SearchResp mockSearchResp = mock(SearchResp.class);
25 SearchResp.SearchResult searchResult1 = mock(SearchResp.
    SearchResult.class);
SearchResp.SearchResult searchResult2 = mock(SearchResp.
    SearchResult.class);

Map<String, Object> entity1 = new HashMap<>();
entity1.put("embedding", "Document 1");
30 entity1.put("document_type", "TESTS");
when(searchResult1.getEntity()).thenReturn(entity1);

Map<String, Object> entity2 = new HashMap<>();
entity2.put("embedding", "Document 2");
35 entity2.put("document_type", "OTHER");
when(searchResult2.getEntity()).thenReturn(entity2);

when(mockSearchResp.getSearchResults()).thenReturn(List.of(
    List.of(searchResult1, searchResult2)));
when(vectorStorageService.searchEmbedding(any(), eq(5))).
    thenReturn(mockSearchResp);
40

Map<String, String> context = ragService.retrieveContext(
    mockEmbeddingResponse);

assertEquals("Document 1", context.get("
    endpointDescription"));
assertEquals("Document 2", context.get("examples"));
45 assertEquals("", context.get("previousTestFileContent"));
}

@Test
void testBuildTestGenerationPrompt() {
50 Map<String, String> context = new HashMap<>();
context.put("endpointDescription", "Test Endpoint
    Description");
context.put("examples", "Test Example");
context.put("previousTestFileContent", "");

55 String endpointCode = "public void testEndpoint() {}";

```



```

        Prompt prompt = ragService.buildTestGenerationPrompt(
            context, endpointCode);

        assertTrue(prompt.getContents().contains("Test Endpoint
            Description"));
60    assertTrue(prompt.getContents().contains("Test Example"));
        assertTrue(prompt.getContents().contains(endpointCode));
    }

    @Test
65    void
        testBuildTestGenerationPrompt_WithoutPreviousTestFileContent
            () {
        Map<String, String> context = new HashMap<>();
        context.put("endpointDescription", "Test Endpoint
            Description");
        context.put("examples", "Test Example");
        context.put("previousTestFileContent", "");

70    String endpointCode = "public void testEndpoint() {}";

        Prompt prompt = ragService.buildTestGenerationPrompt(
            context, endpointCode);

75    String promptContent = prompt.getContents();
        assertTrue(promptContent.contains("Test Endpoint
            Description"));
        assertTrue(promptContent.contains("Test Example"));
        assertTrue(promptContent.contains("public void
            testEndpoint() {}"));
        assertTrue(promptContent.contains("[EXAMPLE_TESTS]"));
80    assertFalse(promptContent.contains("[
            PREVIOUS_TEST_FILE_CONTENT]"));
    }

    @SneakyThrows
    @Test
85    void testGenerateTests_MultipleMethods_Success() {
        RAGService ragServiceSpy = spy(new RAGService(chatService,
            embeddingService, vectorStorageService));

        EndpointCodeDetails mockEndpointCodeDetails = mock(
            EndpointCodeDetails.class);

```

```

MethodDeclaration mockMethodDeclaration1 = mock(
    MethodDeclaration.class);
90 MethodDeclaration mockMethodDeclaration2 = mock(
    MethodDeclaration.class);
EmbeddingResponse mockEmbeddingResponse = mock(
    EmbeddingResponse.class);
ChatResponse mockChatResponse = mock(ChatResponse.class);
Generation mockChatResult = mock(Generation.class);
AssistantMessage mockChatOutput = mock(AssistantMessage.
    class);
95
when(mockEndpointCodeDetails.getEndpointMethods()).
    thenReturn(List.of(mockMethodDeclaration1,
        mockMethodDeclaration2));
when(mockEndpointCodeDetails.getClassName()).thenReturn("
    TestClass");
when(mockEndpointCodeDetails.serializeResults()).
    thenReturn("Serialized Endpoint Code");
100
when(embeddingService.getEmbedding(anyString())).
    thenReturn(mockEmbeddingResponse);

Map<String, String> context = new HashMap<>();
context.put("endpointDescription", "Test Description");
context.put("examples", "Example Test");
105 context.put("previousTestFileContent", "");

doReturn(context).when(ragServiceSpy).retrieveContext(
    mockEmbeddingResponse);

when(chatService.sendRequest(any(Prompt.class))).
    thenReturn(mockChatResponse);
110 when(mockChatResponse.getResult()).thenReturn(
    mockChatResult);
when(mockChatResult.getOutput()).thenReturn(mockChatOutput
    );
when(mockChatOutput.getContent()).thenReturn("[JAVA_CODE]
    public class Test {}[/JAVA_CODE]");

String generatedTests = ragServiceSpy.generateTests(
    mockEndpointCodeDetails);
115

```

```

        assertTrue(generatedTests.contains("public class Test {}")
        );

        verify(embeddingService, times(2)).getEmbedding(anyString
        ());
        verify(chatService, times(3)).sendRequest(any(Prompt.class
        ));
120    }
    }

```

П2.7. Класс VectorStorageService.java

```

public class VectorStorageService {
    private final MilvusClientV2 vectorClient;
    private static final String COLLECTION_NAME = "
        knowledge_base";
5    private static final String ID_FIELD = "id";
    private static final String VECTOR_FIELD = "embedding";
    private static final Integer VECTOR_DIM = 768;
    private static final String DOCUMENT_ID_FIELD = "document_id
        ";
    private static final String DOCUMENT_TYPE_FIELD = "
        document_type";
10    private static final String CHUNK_ID_FIELD = "chunk_id";

    public VectorStorageService(MilvusClientV2 milvusClientV2) {
        this.vectorClient = milvusClientV2;
        initializeCollection();
15    }

    private boolean isCollectionExists() {
        return this.vectorClient.hasCollection(
            HasCollectionReq.builder()
20            .collectionName(COLLECTION_NAME)
            .build()
        );
    }

    private void initializeCollection() {
25        try {
            if (this.isCollectionExists()) return;

```

```

30      AddFieldReq idField = AddFieldReq.builder()
        .fieldName(ID_FIELD)
        .dataType(DataType.Int64)
        .isPrimaryKey(Boolean.TRUE)
        .autoID(true)
        .build();
35      AddFieldReq embeddingField = AddFieldReq.builder()
        .fieldName(VECTOR_FIELD)
        .dataType(DataType.FloatVector)
        .dimension(VECTOR_DIM)
        .build();
40      AddFieldReq docTypeField = AddFieldReq.builder()
        .fieldName(DOCUMENT_TYPE_FIELD)
        .dataType(DataType.VarChar)
        .build();
        AddFieldReq docIdField = AddFieldReq.builder()
45      .fieldName(DOCUMENT_ID_FIELD)
        .dataType(DataType.Int64)
        .build();
        AddFieldReq chunkIdField = AddFieldReq.builder()
50      .fieldName(CHUNK_ID_FIELD)
        .dataType(DataType.Int64)
        .build();

        CreateCollectionReq.CollectionSchema collectionSchema =
            CreateCollectionReq
55      .CollectionSchema
        .builder()
        .build();

        collectionSchema.addField(idField);
        collectionSchema.addField(embeddingField);
60      collectionSchema.addField(docTypeField);
        collectionSchema.addField(docIdField);
        collectionSchema.addField(chunkIdField);

        CreateCollectionReq requestCreate = CreateCollectionReq.
            builder()
65      .collectionName(COLLECTION_NAME)
        .collectionSchema(collectionSchema)
        .build();

        this.vectorClient.createCollection(requestCreate);

```

```

70         this.createIndex();
    } catch (Exception e) {
        throw new RuntimeException("Failed to initialize Milvus
            collection: " + e.getMessage(), e);
    }
}

75
public void createIndex() {
    try {
        IndexParam indexParam = IndexParam.builder()
            .fieldName(VECTOR_FIELD)
80         .indexType(IndexParam.IndexType.HNSW)
            .metricType(IndexParam.MetricType.L2)
            .extraParams(Collections.singletonMap("M", 16))
            .build();

        CreateIndexReq createIndexReq = CreateIndexReq.builder()
            .collectionName(COLLECTION_NAME)
            .indexParams(Collections.singletonList(indexParam))
            .build();

85
        this.vectorClient.createIndex(createIndexReq);
    } catch (Exception e) {
        throw new RuntimeException("Failed to create index in
            Milvus: " + e.getMessage(), e);
    }
}

90
}

95
public void loadCollection() {
    try {
        this.vectorClient.loadCollection(
            LoadCollectionReq.builder()
100         .collectionName(COLLECTION_NAME)
            .build()
        );
    } catch (Exception e) {
        System.err.println("Error loading collection into memory
            : " + e.getMessage());
105
    }
}

public void insertEmbeddings(KnowledgeChunk chunk) {
    try {

```

```

110     JSONObject jsonObject = new JSONObject();
        jsonObject.put(VECTOR_FIELD, chunk.getEmbedding());
        jsonObject.put(DOCUMENT_TYPE_FIELD, chunk.
            getDocumentType());
        jsonObject.put(DOCUMENT_ID_FIELD, chunk.getDocumentId());
            ;
        jsonObject.put(CHUNK_ID_FIELD, chunk.getChunkId());
115
        InsertReq insertReq = InsertReq.builder()
            .collectionName(COLLECTION_NAME)
            .data(Collections.singletonList(jsonObject))
            .build();
120
        this.vectorClient.insert(insertReq);
    } catch (Exception e) {
        throw new RuntimeException("Failed to insert embedding
            into Milvus: " + e.getMessage(), e);
    }
125 }

public SearchResp searchEmbedding(EmbeddingResponse
    embeddingResponse, int topK) {
    try {
        this.loadCollection();
130
        Embedding output = embeddingResponse.getResult();
        List<Float> result = new ArrayList<>();
        for (float value : output.getOutput()) {
            result.add(value);
135
        }

        if (result.isEmpty()) {
            throw new IllegalArgumentException("The embedding
                vector is empty. Ensure the embedding response
                contains valid data.");
        }
140

        SearchReq searchReq = SearchReq.builder()
            .collectionName(COLLECTION_NAME)
            .data(Collections.singletonList(result))
            .outputFields(List.of(VECTOR_FIELD, DOCUMENT_ID_FIELD,
                CHUNK_ID_FIELD, DOCUMENT_TYPE_FIELD))
145
            .topK(topK)

```

```

        .build();

        return this.vectorClient.search(searchReq);
    } catch (Exception e) {
150         throw new RuntimeException("Failed to search embedding
            in Milvus: " + e.getMessage(), e);
    }
}

public class VectorStorageServiceTest {

    private VectorStorageService vectorStorageService;

5    @Mock
    private MilvusClientV2 mockMilvusClient;

    @BeforeEach
10    void setUp() {
        MockitoAnnotations.openMocks(this);
        vectorStorageService = new VectorStorageService(
            mockMilvusClient);
    }

15    @Test
    void
        testSearchEmbedding_ThrowsIllegalArgumentExceptionWhenVectorIs
            () {
        EmbeddingResponse mockEmbeddingResponse = mock(
            EmbeddingResponse.class);
        Embedding mockEmbedding = mock(Embedding.class);
        when(mockEmbeddingResponse.getResult()).thenReturn(
            mockEmbedding);
20        when(mockEmbedding.getOutput()).thenReturn(new float[0]);

        RuntimeException exception = assertThrows(RuntimeException
            .class, () ->
            vectorStorageService.searchEmbedding(mockEmbeddingResponse
                , 5)
        );
25        assertEquals("Failed to search embedding in Milvus: The
            embedding vector is empty. Ensure the embedding
            response contains valid data.", exception.getMessage());
    }

```



```

        verify(mockMilvusClient, never()).search(any());
    }

30  @Test
    void testInsertEmbeddings_ThrowsRuntimeException() {
        KnowledgeChunk chunk = mock(KnowledgeChunk.class);
        when(chunk.getEmbedding()).thenReturn(Collections.
            singletonList(0.1f));
        when(chunk.getDocumentType()).thenReturn("testType");
35  when(chunk.getDocumentId()).thenReturn(1L);
        when(chunk.getChunkId()).thenReturn(2L);

        doThrow(new RuntimeException("Simulated insertion failure
            "))
            .when(mockMilvusClient).insert(any());
40

        RuntimeException exception = assertThrows(RuntimeException
            .class, () ->
            vectorStorageService.insertEmbeddings(chunk)
        );

45  assertTrue(exception.getMessage().contains("Failed to
            insert embedding into Milvus: Simulated insertion
            failure"));
    }
}

```

П2.8. Класс KnowledgeUpdatingWorker.java

```

@Component
public class KnowledgeUpdatingWorker {
    private static final Logger logger = LoggerFactory.getLogger
        (KnowledgeUpdatingWorker.class);
5  private final EmbeddingService embeddingService;
    private final VectorStorageService vectorStorage;

    public enum DocumentTypes {
        TESTS,
10  DOCUMENTATION
    }
}

```

```

@Autowired
public KnowledgeUpdatingWorker(EmbeddingService
    embeddingService, VectorStorageService
    vectorStorageService) {
15     this.embeddingService = embeddingService;
        this.vectorStorage = vectorStorageService;
    }

    static long generateLongFromString(String input) {
20         try {
            MessageDigest digest = MessageDigest.getInstance("SHA
                -256");
            byte[] hash = digest.digest(input.getBytes(
                StandardCharsets.UTF_8));
            long result = 0;
            for (int i = 0; i < 8; i++) {
25                 result <<= 8;
                result |= (hash[i] & 0xFF);
            }
            return result;
        } catch (NoSuchAlgorithmException e) {
30             throw new RuntimeException("SHA-256 algorithm not found
                !", e);
        }
    }

    protected List<KnowledgeChunk> processCodeContent(String
        fileName, String fileContent) {
35         String preprocessedCode = CodePreprocessor.preprocessCode(
            fileContent);
        EmbeddingResponse embeddingResponse = this.
            embeddingService.getEmbedding(preprocessedCode);

        if (embeddingResponse == null || embeddingResponse.
            getResult() == null) {
            throw new RuntimeException("EmbeddingResponse or its
                result is null");
40         }

        Embedding output = embeddingResponse.getResult();
        List<Float> embedding = new ArrayList<>();
        for (float value : output.getOutput()) {
45             embedding.add(value);
        }
    }

```

```

    }

    Long documentId = KnowledgeUpdatingWorker.
        generateLongFromString(fileName);

50    KnowledgeChunk knowledgeChunk = new KnowledgeChunk();
    knowledgeChunk.setEmbedding(embedding);
    knowledgeChunk.setDocumentType(DocumentTypes.TESTS.name());
        ;
    knowledgeChunk.setDocumentId(documentId);
    knowledgeChunk.setChunkId(1L);

55    return Collections.singletonList(knowledgeChunk);
}

protected List<KnowledgeChunk> processDocContent(String
    fileName, String fileContent) {
60    EmbeddingResponse embeddingResponse = this.
        embeddingService.getEmbedding(fileContent);

    if (embeddingResponse == null || embeddingResponse.
        getResult() == null) {
        throw new RuntimeException("EmbeddingResponse or its
            result is null");
    }

65    Embedding output = embeddingResponse.getResult();
    List<Float> embedding = new ArrayList<>();
    for (float value : output.getOutput()) {
        embedding.add(value);
70    }

    Long documentId = KnowledgeUpdatingWorker.
        generateLongFromString(fileName);

    KnowledgeChunk knowledgeChunk = new KnowledgeChunk();
75    knowledgeChunk.setEmbedding(embedding);
    knowledgeChunk.setDocumentType(DocumentTypes.DOCUMENTATION
        .name());
    knowledgeChunk.setDocumentId(documentId);
    knowledgeChunk.setChunkId(1L);

80    return Collections.singletonList(knowledgeChunk);

```

```

    }

    public void processKnowledgeBaseUpdate(@NotNull Map<String ,
        Object> event) {
        try {
85         String documentType = (String) event.get("documentType")
            ;
            String fileContent = (String) event.get("fileContent");
            String fileName = (String) event.get("fileName");
            List<KnowledgeChunk> knowledgeChunks;

90         if (Objects.equals(documentType, DocumentTypes.TESTS.
            name())) {
            knowledgeChunks = processCodeContent(fileName,
                fileContent);
        } else {
            knowledgeChunks = processDocContent(fileName,
                fileContent);
        }

95         for (KnowledgeChunk chunk : knowledgeChunks) {
            this.vectorStorage.insertEmbeddings(chunk);
            logger.info(
                "File type '{}' with id '{}' processed and data
                inserted into Milvus.",
100         chunk.getDocumentType(),
            chunk.getDocumentId()
            );
        }
    } catch (Exception e) {
105         throw new RuntimeException("Error processing file: " +
            (e.getMessage() != null
            ? e.getMessage()
            : "null"),
            e);
110     }
    }
}

public class KnowledgeUpdatingWorkerTest {
    @Mock
    private EmbeddingService embeddingService;

5    @Mock

```

```

private VectorStorageService vectorStorage;

private KnowledgeUpdatingWorker knowledgeUpdatingWorker;

10 @BeforeEach
void setUp() {
    MockitoAnnotations.openMocks(this);
    knowledgeUpdatingWorker = new KnowledgeUpdatingWorker(
        embeddingService, vectorStorage);
15 }

@Test
void testProcessKnowledgeBaseUpdate_WithTests() {
    // Arrange
20 Map<String, Object> event = new HashMap<>();
    event.put("documentType", "TESTS");
    event.put("fileContent", "public class Test {}");
    event.put("fileName", "TestFile.java");

    KnowledgeChunk mockChunk = new KnowledgeChunk();
    mockChunk.setDocumentType("TESTS");
    mockChunk.setDocumentId(123L);
    mockChunk.setEmbedding(List.of(0.1f, 0.2f, 0.3f));
    mockChunk.setChunkId(1L);
25

    KnowledgeUpdatingWorker spyWorker = spy(
        knowledgeUpdatingWorker);
    doReturn(List.of(mockChunk)).when(spyWorker).
        processCodeContent(anyString(), anyString());

    // Act
35 spyWorker.processKnowledgeBaseUpdate(event);

    // Assert
    verify(spyWorker).processCodeContent("TestFile.java", "
        public class Test {}");
    verify(vectorStorage).insertEmbeddings(mockChunk);
40 }

@Test
void testProcessKnowledgeBaseUpdate_WithDocumentation() {
    // Arrange
45 Map<String, Object> event = new HashMap<>();

```

```

event.put("documentType", "DOCUMENTATION");
event.put("fileContent", "This is documentation content.");
;
event.put("fileName", "DocFile.txt");

50 KnowledgeChunk mockChunk = new KnowledgeChunk();
mockChunk.setDocumentType("DOCUMENTATION");
mockChunk.setDocumentId(123L);
mockChunk.setEmbedding(List.of(0.4f, 0.5f, 0.6f));
mockChunk.setChunkId(1L);

55 KnowledgeUpdatingWorker spyWorker = spy(
    knowledgeUpdatingWorker);
doReturn(List.of(mockChunk)).when(spyWorker).
    processDocContent(anyString(), anyString());

// Act
60 spyWorker.processKnowledgeBaseUpdate(event);

// Assert
verify(spyWorker).processDocContent("DocFile.txt", "This
    is documentation content.");
verify(vectorStorage).insertEmbeddings(mockChunk);
65 }

@Test
void testProcessKnowledgeBaseUpdate_WithMultipleChunks() {
    // Arrange
70 Map<String, Object> event = new HashMap<>();
event.put("documentType", "TESTS");
event.put("fileContent", "public class Test {}");
event.put("fileName", "TestFile.java");

75 KnowledgeChunk chunk1 = new KnowledgeChunk();
chunk1.setDocumentType("TESTS");
chunk1.setDocumentId(123L);
chunk1.setEmbedding(List.of(0.1f, 0.2f, 0.3f));
chunk1.setChunkId(1L);

80 KnowledgeChunk chunk2 = new KnowledgeChunk();
chunk2.setDocumentType("TESTS");
chunk2.setDocumentId(124L);
chunk2.setEmbedding(List.of(0.4f, 0.5f, 0.6f));

```

```

85     chunk2.setChunkId(2L);

    KnowledgeUpdatingWorker spyWorker = spy(
        knowledgeUpdatingWorker);
    doReturn(List.of(chunk1, chunk2)).when(spyWorker).
        processCodeContent(anyString(), anyString());

90     // Act
    spyWorker.processKnowledgeBaseUpdate(event);

    // Assert
    verify(vectorStorage).insertEmbeddings(chunk1);
95     verify(vectorStorage).insertEmbeddings(chunk2);
}

@Test
void testProcessKnowledgeBaseUpdate_ExceptionWithMessage() {
100     // Arrange
    Map<String, Object> event = new HashMap<>();
    event.put("documentType", "TESTS");
    event.put("fileContent", "public class Test {}");
    event.put("fileName", "TestFile.java");

105     KnowledgeUpdatingWorker spyWorker = spy(
        knowledgeUpdatingWorker);

    doThrow(new RuntimeException("Custom exception message"))
        .when(spyWorker).processCodeContent(anyString(), anyString
            ());

110     // Act & Assert
    RuntimeException exception = assertThrows(RuntimeException
        .class, () -> {
        spyWorker.processKnowledgeBaseUpdate(event);
    });

115     assertTrue(exception.getMessage().contains("Error
        processing file: Custom exception message"));
}

@Test
120 void testProcessCodeContent_EmptyOutput() {
    // Arrange

```



```

String fileName = "TestFile.java";
String fileContent = "public class Test {}";

125 try (MockedStatic<CodePreprocessor> mockedPreprocessor =
    mockStatic(CodePreprocessor.class)) {
    mockedPreprocessor.when(() -> CodePreprocessor.
        preprocessCode(fileContent))
        .thenReturn("Preprocessed content");

    EmbeddingResponse mockResponse = mock(EmbeddingResponse.
        class);
130 Embedding mockEmbedding = mock(Embedding.class);
    when(embeddingService.getEmbedding("Preprocessed content
        ")).thenReturn(mockResponse);
    when(mockResponse.getResult()).thenReturn(mockEmbedding)
        ;
    when(mockEmbedding.getOutput()).thenReturn(new float[0])
        ;

135 // Act
    List<KnowledgeChunk> chunks = knowledgeUpdatingWorker.
        processCodeContent(fileName, fileContent);

    // Assert
    assertNotNull(chunks);
140 assertEquals(1, chunks.size());
    KnowledgeChunk chunk = chunks.get(0);
    assertNotNull(chunk.getEmbedding());
    assertTrue(chunk.getEmbedding().isEmpty());
    assertEquals("TESTS", chunk.getDocumentType());
145 assertEquals(knowledgeUpdatingWorker.
        generateLongFromString(fileName), chunk.getDocumentId
        ());
    assertEquals(1L, chunk.getChunkId());
    }
}

150 @Test
void testProcessCodeContent_FilledOutput() {
    // Arrange
    String fileName = "TestFile.java";
    String fileContent = "public class Test {}";
155

```

```

try (MockedStatic<CodePreprocessor> mockedPreprocessor =
    mockStatic(CodePreprocessor.class)) {
    mockedPreprocessor.when(() -> CodePreprocessor.
        preprocessCode(fileContent))
        .thenReturn("Preprocessed content");

160    EmbeddingResponse mockResponse = mock(EmbeddingResponse.
        class);
    Embedding mockEmbedding = mock(Embedding.class);
    when(embeddingService.getEmbedding("Preprocessed content
        ")).thenReturn(mockResponse);
    when(mockResponse.getResult()).thenReturn(mockEmbedding)
        ;
    when(mockEmbedding.getOutput()).thenReturn(new float
        []{0.1f, 0.2f, 0.3f});

165    // Act
    List<KnowledgeChunk> chunks = knowledgeUpdatingWorker.
        processCodeContent(fileName, fileContent);

    // Assert
170    assertNotNull(chunks);
    assertEquals(1, chunks.size());
    KnowledgeChunk chunk = chunks.get(0);
    assertNotNull(chunk.getEmbedding());
    assertEquals(List.of(0.1f, 0.2f, 0.3f), chunk.
        getEmbedding());
175    assertEquals("TESTS", chunk.getDocumentType());
    assertEquals(knowledgeUpdatingWorker.
        generateLongFromString(fileName), chunk.getDocumentId
        ());
    assertEquals(1L, chunk.getChunkId());
}
}

180
@Test
void testProcessCodeContent_ReturnsKnowledgeChunk() {
    // Arrange
    String fileName = "TestFile.java";
185    String fileContent = "public class Test {}";

    try (MockedStatic<CodePreprocessor> mockedPreprocessor =
        mockStatic(CodePreprocessor.class)) {

```

```

mockedPreprocessor.when(() -> CodePreprocessor.
    preprocessCode(fileContent))
    .thenReturn("Preprocessed content");
190

EmbeddingResponse mockResponse = mock(EmbeddingResponse.
    class);
Embedding mockEmbedding = mock(Embedding.class);
when(embeddingService.getEmbedding("Preprocessed content
    ")).thenReturn(mockResponse);
when(mockResponse.getResult()).thenReturn(mockEmbedding)
    ;
195
when(mockEmbedding.getOutput()).thenReturn(new float
    []{0.1f});

// Act
List<KnowledgeChunk> chunks = knowledgeUpdatingWorker.
    processCodeContent(fileName, fileContent);

200
// Assert
assertNotNull(chunks);
assertEquals(1, chunks.size());
KnowledgeChunk chunk = chunks.get(0);
assertEquals(List.of(0.1f), chunk.getEmbedding());
205
assertEquals("TESTS", chunk.getDocumentType());
assertEquals(knowledgeUpdatingWorker.
    generateLongFromString(fileName), chunk.getDocumentId
    ());
assertEquals(1L, chunk.getChunkId());
}
}
210

@Test
void testProcessDocContent_EmptyOutput() {
    // Arrange
    String fileName = "DocFile.txt";
    String fileContent = "This is documentation content.";
    215

    EmbeddingResponse mockResponse = mock(EmbeddingResponse.
        class);
    Embedding mockEmbedding = mock(Embedding.class);
    when(embeddingService.getEmbedding(fileContent)).
        thenReturn(mockResponse);
    220
    when(mockResponse.getResult()).thenReturn(mockEmbedding);

```

```

when(mockEmbedding.getOutput()).thenReturn(new float[0]);

// Act
List<KnowledgeChunk> chunks = knowledgeUpdatingWorker.
    processDocContent(fileName, fileContent);
225

// Assert
assertNotNull(chunks);
assertEquals(1, chunks.size());
KnowledgeChunk chunk = chunks.get(0);
230
assertNotNull(chunk.getEmbedding());
assertTrue(chunk.getEmbedding().isEmpty());
assertEquals("DOCUMENTATION", chunk.getDocumentType());
assertEquals(knowledgeUpdatingWorker.
    generateLongFromString(fileName), chunk.getDocumentId()
);
assertEquals(1L, chunk.getChunkId());
235
}

@Test
void testProcessDocContent_ReturnsKnowledgeChunk() {
    // Arrange
240
    String fileName = "DocFile.txt";
    String fileContent = "This is documentation content.";

    EmbeddingResponse mockResponse = mock(EmbeddingResponse.
        class);
    Embedding mockEmbedding = mock(Embedding.class);
245
    when(embeddingService.getEmbedding(fileContent)).
        thenReturn(mockResponse);
    when(mockResponse.getResult()).thenReturn(mockEmbedding);
    when(mockEmbedding.getOutput()).thenReturn(new float[]{0.1
        f});

    // Act
250
    List<KnowledgeChunk> chunks = knowledgeUpdatingWorker.
        processDocContent(fileName, fileContent);

    // Assert
    assertNotNull(chunks);
    assertEquals(1, chunks.size());
255
    KnowledgeChunk chunk = chunks.get(0);
    assertEquals(List.of(0.1f), chunk.getEmbedding());

```

```

    assertEquals("DOCUMENTATION", chunk.getDocumentType());
    assertEquals(KnowledgeUpdatingWorker.
        generateLongFromString(fileName), chunk.getDocumentId()
    );
    assertEquals(1L, chunk.getChunkId());
260 }

@Test
void testGenerateLongFromString() {
    // Act
265     long result = KnowledgeUpdatingWorker.
        generateLongFromString("TestFile.java");

    // Assert
    assertEquals(8985125642329676891L, result);
    }
270 }

```

И2.9. Класс TestGenerationWorker.java

```

@Component
public class TestGenerationWorker {
    private final GitHubService gitHubService;
5     private final RAGService ragService;

    @Autowired
    public TestGenerationWorker(GitHubService gitHubService,
        RAGService ragClient) {
        this.gitHubService = gitHubService;
10        this.ragService = ragClient;
    }

    public void processTestGeneration(@NotNull QueueEvent event)
        throws Exception {
        List<EndpointCodeDetails> changedEndpoints = gitHubService
            .analyzePullRequestChanges(
15        event.getRepositoryUrl(),
            event.getRepositoryName(),
            event.getPullRequestId()
        );
    }

```

```

20      Map<String , String> mappedClassToContent = new HashMap<>()
        ;
        for (EndpointCodeDetails endpoint : changedEndpoints) {
            String generateTests = ragService.generateTests(endpoint
                );
            mappedClassToContent.put(endpoint.getClassName() ,
                generateTests);
        }
25
        gitHubService.createPullRequestWithChanges(event.
            getRepositoryUrl() , event.getRepositoryName() , event.
            getPullRequestId() , mappedClassToContent);
    }
}

public class TestGenerationWorkerTest {

    @Mock
5    private GitHubService gitHubService;

    @Mock
    private RAGService ragService;

10    private TestGenerationWorker testGenerationWorker;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
15        testGenerationWorker = new TestGenerationWorker(
            gitHubService , ragService);
    }

    @Test
    void testProcessTestGeneration() throws Exception {
20        // Arrange
        QueueEvent mockEvent = new QueueEvent();
        mockEvent.setRepositoryUrl(" https:// github.com/ test / repo ")
            ;
        mockEvent.setRepositoryName(" repo ");
        mockEvent.setPullRequestId(123);
25

        MethodDeclaration method1 = mock(MethodDeclaration.class);
        MethodDeclaration method2 = mock(MethodDeclaration.class);

```

```

30 EndpointCodeDetails endpoint1 = new EndpointCodeDetails();
   endpoint1.setClassName("ClassA");
   endpoint1.setEndpointMethods(Arrays.asList(method1,
        method2));
   endpoint1.setModels(Map.of("ModelA", Map.of("field1", "
        value1"))));

35 EndpointCodeDetails endpoint2 = new EndpointCodeDetails();
   endpoint2.setClassName("ClassB");
   endpoint2.setEndpointMethods(Collections.singletonList(
        method1));
   endpoint2.setModels(Map.of("ModelB", Map.of("field2", "
        value2"))));

40 List<EndpointCodeDetails> changedEndpoints = Arrays.asList
   (endpoint1, endpoint2);

   when(gitHubService.analyzePullRequestChanges("https://
        github.com/test/repo", "repo", 123))
   .thenReturn(changedEndpoints);

   when(ragService.generateTests(endpoint1)).thenReturn("
        TestCodeForClassA");
45 when(ragService.generateTests(endpoint2)).thenReturn("
        TestCodeForClassB");

   ArgumentCaptor<Map<String, String>> mapCaptor =
        ArgumentCaptor.forClass(Map.class);

50 // Act
   testGenerationWorker.processTestGeneration(mockEvent);

   // Assert
   verify(gitHubService).analyzePullRequestChanges("https://
        github.com/test/repo", "repo", 123);
   verify(ragService).generateTests(endpoint1);
55 verify(ragService).generateTests(endpoint2);

   verify(gitHubService).createPullRequestWithChanges(eq("
        https://github.com/test/repo"), eq("repo"), eq(123),
        mapCaptor.capture());

   Map<String, String> capturedMap = mapCaptor.getValue();

```

```

60    assertEquals(2, capturedMap.size());
    assertEquals("TestCodeForClassA", capturedMap.get("ClassA
        "));
    assertEquals("TestCodeForClassB", capturedMap.get("ClassB
        "));
    }

65    @Test
    void testProcessTestGeneration_NoEndpoints() throws
        Exception {
        // Arrange
        QueueEvent mockEvent = new QueueEvent();
        mockEvent.setRepositoryUrl("https://github.com/test/repo")
            ;
70        mockEvent.setRepositoryName("repo");
        mockEvent.setPullRequestId(123);

        when(gitHubService.analyzePullRequestChanges("https://
            github.com/test/repo", "repo", 123))
            .thenReturn(Collections.emptyList());

75        // Act
        testGenerationWorker.processTestGeneration(mockEvent);

        // Assert
80        verify(gitHubService).analyzePullRequestChanges("https://
            github.com/test/repo", "repo", 123);
        verifyNoInteractions(ragService);
        verify(gitHubService).createPullRequestWithChanges(
            "https://github.com/test/repo", "repo", 123, new HashMap
                <>());
85    }
    }

```

П2.10. Класс EndpointMethodLocator.java

```

@UtilityClass
public class EndpointMethodLocator {
    public static List<MethodDeclaration> findEndpointMethods(
        CompilationUnit compilationUnit, List<Integer>
            changedLines) {

```



```

5      List<MethodDeclaration> affectedMethods = new ArrayList
      <>();

      compilationUnit.findAll(MethodDeclaration.class).forEach(
          method -> {
              int startLine = method.getBegin().map(position ->
                  position.line).orElse(-1);
              int endLine = method.getEnd().map(position -> position.
                  line).orElse(-1);

              boolean isLineChanged = changedLines.stream().anyMatch(
                  line -> line >= startLine && line <= endLine);

              if (isLineChanged && hasEndpointAnnotation(method)) {
                  affectedMethods.add(method);
15          }
      });

      return affectedMethods;
  }

20 private static boolean hasEndpointAnnotation(
    MethodDeclaration method) {
    for (AnnotationExpr annotation : method.getAnnotations())
    {
        String name = annotation.getNameAsString();
        if (name.matches("RequestMapping|GetMapping|PostMapping|
            PutMapping|DeleteMapping|PatchMapping")) {
25            return true;
        }
    }
    return false;
}

30 }

```

П2.11. Класс ModelCollector.java

```

public class ModelCollector {
    private final String modelsPath;

5    public ModelCollector(String modelsPath) {
        this.modelsPath = modelsPath;
    }
}

```

```

    }

    public Map<String, Map<String, Object>> collectModels() {
10      Map<String, Map<String, Object>> models = new HashMap<>();
        File modelDir = new File(modelsPath);

        if (!modelDir.exists() || !modelDir.isDirectory()) {
            throw new IllegalStateException("Model directory not
15              found: " + modelsPath);
        }

        List<File> files = Arrays.asList(modelDir.listFiles((dir,
            name) -> name.endsWith(".java")));
        files.sort((f1, f2) -> {
            try {
20              ClassOrInterfaceDeclaration class1 = StaticJavaParser.
                parse(f1)
                .getClassByName(f1.getName().replace(".java", "")).
                orElse(null);
                ClassOrInterfaceDeclaration class2 = StaticJavaParser.
                parse(f2)
                .getClassByName(f2.getName().replace(".java", "")).
                orElse(null);

25              boolean isParentOfClass2 = class2.getExtendedTypes().
                stream()
                .anyMatch(t -> t.getNameAsString().equals(class1.
                getNameAsString()));
                return isParentOfClass2 ? -1 : 1;
            } catch (Exception e) {
                System.err.println("Failed to parse file for sorting:
30                  " + e.getMessage());
            }
            return 0;
        });

        for (File file : files) {
35            try {
                ClassOrInterfaceDeclaration classDecl =
                    StaticJavaParser.parse(file)
                    .getClassByName(file.getName().replace(".java", ""))
                    .orElse(null);

```

```

40         if (classDecl != null) {
            String typeName = classDecl.getNameAsString();
            models.put(typeName, describeClass(classDecl, models));
        }
    } catch (Exception e) {
45         System.err.println("Failed to parse model: " + file.getName() + " - " + e.getMessage());
    }
}

return models;
50 }

protected Map<String, Object> describeClass(
    ClassOrInterfaceDeclaration classDecl, Map<String, Map<String, Object>> models) {
    Map<String, Object> classDetails = new HashMap<>();
    classDetails.put("className", classDecl.getNameAsString());
    ;
55     classDetails.put("annotations", classDecl.getAnnotations().toString());

    Map<String, Object> fields = new HashMap<>();
    for (FieldDeclaration field : classDecl.getFields()) {
        field.getVariables().forEach(var -> {
60             String fieldName = var.getNameAsString();
            String fieldType = var.getType().asString();
            fields.put(fieldName, fieldType);
        });
    }
65 }

    if (classDecl.getExtendedTypes().isNotEmpty()) {
        String parentType = classDecl.getExtendedTypes(0).getNameAsString();
        classDetails.put("extends", parentType);

70         if (models.containsKey(parentType)) {
            Map<String, Object> parentDetails = models.get(parentType);
            Map<String, Object> parentFields = (Map<String, Object>) parentDetails.get("fields");
            fields.putAll(parentFields);
        }
    }
}

```

```

    }
75    }

    classDetails.put("fields", fields);
    return classDetails;
    }
80 }

```

П2.12. Класс PatchParser.java

```

@UtilityClass
public class PatchParser {
    private static final Pattern CHUNK_HEADER_PATTERN = Pattern.
        compile("@@ -\\d+,\\d+ \\+(\\d+) ,(\\d+) @@");
5
    public static List<Integer> parseChangedLines(String patch)
    {
        List<Integer> changedLines = new ArrayList<>();
        String[] lines = patch.split("\\n");
        int currentLine = 0;
10
        for (String line : lines) {
            Matcher matcher = CHUNK_HEADER_PATTERN.matcher(line);

            if (matcher.find()) {
15                int startLine = Integer.parseInt(matcher.group(1));
                currentLine = startLine;
            } else if (line.startsWith("+") && !line.startsWith
                ("+++")) {
                changedLines.add(currentLine);
                currentLine++;
20            } else if (line.startsWith("-")) {
                changedLines.add(currentLine);
                currentLine++;
            } else {
                currentLine++;
25            }
        }
        return changedLines;
    }
}

```

```
public class PatchParserTest {

    @Test
5   void testParseChangedLines_NoChanges() {
        // Arrange
        String patch = ""
        @@ -1,3 +1,3 @@
        unchanged line
10   unchanged line
        unchanged line
        """;

        // Act
15   List<Integer> changedLines = PatchParser.parseChangedLines
        (patch);

        // Assert
        assertTrue(changedLines.isEmpty());
    }
20

    @Test
    void testParseChangedLines_EmptyPatch() {
        // Arrange
        String patch = "";
25

        // Act
        List<Integer> changedLines = PatchParser.parseChangedLines
        (patch);

        // Assert
30   assertTrue(changedLines.isEmpty());
    }

    @Test
    void testParseChangedLines_DeletedLines() {
35   // Arrange
        String patch = ""
        @@ -1,3 +1,2 @@
        -removed line
        unchanged line
40   +added line
        """;
```

```

// Act
List<Integer> changedLines = PatchParser.parseChangedLines
    (patch);

// Assert
assertEquals(2, changedLines.size(), "Changes 2 rows");
assertEquals(1, changedLines.get(0), "First row");
assertEquals(3, changedLines.get(1), "Second row");
    }
}

```

П2.13. Класс EndpointCodeDetails.java

```

@Setter
@Getter
public class EndpointCodeDetails {
    private String className;
    private List<MethodDeclaration> endpointMethods;
    private Map<String, Map<String, Object>> models;

    public String serializeResults() throws Exception {
        Map<String, Object> result = new HashMap<>();
        result.put("className", className);

        List<Map<String, Object>> methods = new ArrayList<>();
        for (MethodDeclaration method : endpointMethods) {
            methods.add(TypeDescriptor.describeMethod(method));
        }

        result.put("endpointMethods", methods);
        result.put("models", models);

        ObjectMapper mapper = new ObjectMapper();
        return mapper.writerWithDefaultPrettyPrinter().
            writeValueAsString(result);
    }
}

```

П2.14. Класс KnowledgeChunk.java

```

@Setter

```

```

5  @Getter
   public class KnowledgeChunk {
       private List<Float> embedding;
       private String documentType;
       private Long documentId;
       private Long chunkId;

10  @Override
   public String toString() {
       return "KnowledgeChunk{" +
           ", embedding='" + embedding + '\'' +
           ", documentType='" + documentType + '\'' +
15  ", documentId='" + documentId + '\'' +
           ", chunkId=" + chunkId +
           '}' ;
       }
   }
}

```

П2.15. Пайплайн

Основной пайплайн:

```

5  pipeline {
    agent any
    environment {
        BRANCH = "${BRANCH}"
        ACTION = "${ACTION}"
        PULL_REQUEST_URL = "${PULL_REQUEST_URL}"
        PULL_REQUEST_ID = "${PULL_REQUEST_ID}"
        PULL_REQUEST_TITLE = "${PULL_REQUEST_TITLE}"
10  REPO_ID = "${REPO_ID}"
        REPO_NAME = "${REPO_NAME}"
        REPO_CLONE_URL = "${REPO_CLONE_URL}"
        RESPONSE_SUCCESS = false
        APP_DIR = <HOME_DIR>
15  PORT = <PORT>
        HEAD_BRANCH = "${HEAD_BRANCH}"
        PROCESS_SUCCESS = false // Флаг для проверки запуска
        GITHUB_API_URL = <GITHUB_API_URL>
        HEAD_BASE_TEST = 'branch'
20  }
    stages {
        stage('Send Webhook to External Service') {

```

```

steps {
  script {
25     def payload = """
        {
          "action": "${env.ACTION}",
          "number": ${env.PULL_REQUEST_ID},
          "pull_request": {
30             "url": "${env.PULL_REQUEST_URL}",
             "id": ${env.PULL_REQUEST_ID},
             "title": "${env.PULL_REQUEST_TITLE}"
          },
          "repository": {
35             "id": ${env.REPO_ID},
             "full_name": "${env.REPO_NAME}",
             "clone_url": "${env.REPO_CLONE_URL}"
          }
        }
40     """

    echo payload

    def response = httpRequest(
45     url: <RAG_API_TEST_URL>,
    httpMode: 'POST',
    contentType: 'APPLICATION_JSON',
    requestBody: payload,
    validResponseCodes: '200'
50    )

    if (response.status == 200) {
      env.RESPONSE_SUCCESS = true
      echo "POST request successful. Response code: ${
        response.status}"
55    } else {
      error "Failed to send POST request. Response code:
        ${response.status}, Response body: ${response.
        content}"
    }
  }
}

60 stage('Wait for Specific Build') {
  steps {

```



```

script {
    def jobName = 'run-testing'
    def targetHeadBranch = "${HEAD_BRANCH}"
    def maxWaitTime = 6000 * 60
    def pollInterval = 30 секундaх
    def elapsedTime = 0
    def buildFound = false

    echo "Waiting for a successful build with
        head_branch='${targetHeadBranch}'"

    while (elapsedTime < maxWaitTime) {
        def job = Jenkins.instance.getItemByFullName(
            jobName)
        echo "elapsedTime='${elapsedTime}'"
        echo "maxWaitTime='${maxWaitTime}'"
        echo "Job='${job}'"
        if (job) {
            def lastSuccessfulBuild = job.
                getLastSuccessfulBuild()
            if (lastSuccessfulBuild) {

                def buildEnvVars = lastSuccessfulBuild.
                    getEnvironment()
                def baseBranch = buildEnvVars.get('BASE_BRANCH')

                echo "Checking build #${lastSuccessfulBuild.
                    getNumber()} with HEAD_BRANCH=${baseBranch}
                    )"

                if (baseBranch == targetHeadBranch) {
                    echo "Found build #${lastSuccessfulBuild.
                        getNumber()} with head_branch='${
                            targetHeadBranch}'"
                    env.HEAD_BRANCH_TEST = buildEnvVars.get('
                        HEAD_BRANCH')
                    def pullRequestUrl = buildEnvVars.get('
                        PULL_REQUEST_URL')
                    echo "Head branch: ${env.HEAD_BRANCH_TEST}"
                    echo "Pull Request URL: ${pullRequestUrl}"
                    buildFound = true
                    break

```

```

95         }
        } else {
            echo "No successful builds found for job '${
                jobName} '.'"
        }
    } else {
100         error "Job '${jobName}' not found!"
    }

    echo "Waiting for ${pollInterval} seconds before
        checking again..."
    sh 'sleep 30'
105     elapsedTime += pollInterval
}

if (!buildFound) {
    error "Timeout reached: No successful build with
        head_branch='${targetHeadBranch}' was found
        within ${maxWaitTime / 60} minutes."
110 }
}
}
}
stage('Testing') {
115     when {
        expression { true }
    }
    steps {
        script {
120             if (env.PROCESS_SUCCESS == 'true') {
                echo "Application is running. Running tests..."
            } else {
                echo "Application failed to start. Proceeding to
                    tests for additional diagnostics..."
            }
        }
125     try {
        sh ""
        echo "Navigating to application directory: ${
            APP_DIR}"
        cd ${APP_DIR}

130         echo "Switching to branch: ${env.HEAD_BRANCH_TEST
            }"
    }
}

```

```

git fetch origin
git checkout ${env.HEAD_BRANCH_TEST}
git pull origin ${env.HEAD_BRANCH_TEST}

135 echo "Testing application..."
    ./mvnw spring-javaformat:apply
    ./mvnw test -Dtest=PetControllerApiTest
    ""
    } catch (Exception e) {
140 echo "An error occurred: ${e}"
        env.PROCESS_SUCCESS = false
    }
    }
    }
145 }
    }
    }
    }

```

Вспомогательный пайплан ожидания сгенерированных тестов, который взаимодействует с основным:

```

pipeline {
    agent any
    environment {
5      BASE_BRANCH = "${BASE_BRANCH}"
      HEAD_BRANCH = "${HEAD_BRANCH}"
      PULL_REQUEST_URL = "${PULL_REQUEST_URL}"
    }
    stages {
10      stage('Notify Waiting Job') {
          steps {
              script {
15                def baseBranch = env.BASE_BRANCH
                def headBranch = env.HEAD_BRANCH
                def pullRequestUrl = env.PULL_REQUEST_URL

                def crumbResponse = httpRequest(
20                  url: <JENKINS_URL>,
                  authentication: 'jenkins-credentials-id',
                  httpMode: 'GET'
                )

                def crumbJson = readJSON text: crumbResponse.content
                def crumbField = crumbJson.crumbRequestField

```

```

25      def crumb = crumbJson.crumb

      httpRequest(
        url: <JENKINS_URL>,
        httpMode: 'POST',
30      authentication: 'jenkins-credentials-id',
        contentType: 'APPLICATION_FORM',
        customHeaders: [[name: crumbField, value: crumb]],
        requestBody: "param1=value1&param2=value2"
      )

35      httpRequest(
        url: <JENKINS_URL>,
        httpMode: 'POST',
        authentication: 'jenkins-credentials-id',
40      contentType: 'APPLICATION_FORM',
        customHeaders: [[name: crumbField, value: crumb]],
        requestBody: "JOB_TRIGGERED=true&baseBranch=${
          URLEncoder.encode(baseBranch, 'UTF-8')}&
          headBranch=${URLEncoder.encode(headBranch, 'UTF
          -8')}&pull_request.url=${URLEncoder.encode(
          pullRequestUrl, 'UTF-8')}"
      )
45    }
  }
}

```

П2.16. Конфигурация запуска окружения для системы генерации автотестов

```

    version: "3.8"
    services:
      rabbitmq:
5      image: rabbitmq:management
      ports:
        - "5672:5672"
        - "15672:15672"

10      etcd:
        container_name: milvus-etcd
        image: quay.io/coreos/etcd:v3.5.14

```

```

environment:
- ETCD_AUTO_COMPACTION_MODE=revision
- ETCD_AUTO_COMPACTION_RETENTION=1000
- ETCD_QUOTA_BACKEND_BYTES=4294967296
- ETCD_SNAPSHOT_COUNT=50000
volumes:
- ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/etcd:/etcd
command: etcd -advertise-client-urls=http://127.0.0.1:2379 -
listen-client-urls http://0.0.0.0:2379 --data-dir /etcd
healthcheck:
test: ["CMD", "etcdctl", "endpoint", "health"]
interval: 30s
timeout: 20s
retries: 3

minio:
container_name: milvus-minio
image: minio/minio:RELEASE.2023-03-20T20-16-18Z
environment:
MINIO_ACCESS_KEY: minioadmin
MINIO_SECRET_KEY: minioadmin
ports:
- "9001:9001"
- "9000:9000"
volumes:
- ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/minio:/minio_data
command: minio server /minio_data --console-address ":9001"
healthcheck:
test: ["CMD", "curl", "-f", "http://localhost:9000/minio/
health/live"]
interval: 30s
timeout: 20s
retries: 3

standalone:
container_name: milvus-standalone
image: milvusdb/milvus:v2.5.0
command: ["milvus", "run", "standalone"]
security_opt:
- seccomp:unconfined
environment:
ETCD_ENDPOINTS: etcd:2379
MINIO_ADDRESS: minio:9000

```

```

55 volumes :
- ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/milvus:/var/lib/milvus
healthcheck:
test: ["CMD", "curl", "-f", "http://localhost:9091/healthz"]
interval: 30s
start_period: 90s
60 timeout: 20s
retries: 3
ports:
- "19530:19530"
- "9091:9091"
65 depends_on:
- "etcd"
- "minio"

networks:
70 default:
name: milvus

```

П2.17. Сгенерированные автотесты при помощи аналогов

Сгенерированные автотесты инструментов Zencoder:

```

package com.aiqa.ragapitestgenerator.controller;

import com.aiqa.ragapitestgenerator.model.QueueEvent;
5 import com.aiqa.ragapitestgenerator.model.WebhookPayload;
import com.aiqa.ragapitestgenerator.model.Repository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mock;
10 import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.http.MediaType;
import org.springframework.mock.web.MockMultipartFile;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.setup.
    MockMvcBuilders;
15
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.doNothing;
import static org.mockito.Mockito.verify;
20 import static org.mockito.MockitoAnnotations.openMocks;

```

```

import static org.springframework.test.web.servlet.request.
    MockMvcRequestBuilders.multipart;
import static org.springframework.test.web.servlet.request.
    MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.
    MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.
    MockMvcResultMatchers.status;
25 import static com.aiqa.ragapitestgenerator.queue.
    RabbitMQConfig.KNOWLEDGE_BASE_UPDATE_QUEUE;
import static com.aiqa.ragapitestgenerator.queue.
    RabbitMQConfig.TEST_GENERATION_QUEUE;

class WebhookControllerTest {

30     private MockMvc mockMvc;

    @Mock
    private RabbitTemplate rabbitTemplate;

35     private WebhookController webhookController;

    @BeforeEach
    void setUp() {
        openMocks(this);
40     webhookController = new WebhookController(rabbitTemplate);
        mockMvc = MockMvcBuilders.standaloneSetup(
            webhookController).build();
    }

    @Test
45     void generateTests_InvalidPayload_ShouldReturnBadRequest()
        throws Exception {
        // When & Then
        mockMvc.perform(post("/webhook/generate-tests")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{}"))
50     .accept(MediaType.APPLICATION_JSON))
        .andExpect(status().isBadRequest())
        .andExpect(content().string("Invalid payload"));
    }

55     @Test

```

```

void uploadKnowledgeFile_ValidFile_ShouldReturnOk() throws
    Exception {
    // Given
    MockMultipartFile file = new MockMultipartFile(
    "file",
    "test.txt",
    MediaType.TEXT_PLAIN_VALUE,
    "test content".getBytes()
    );

    doNothing().when(rabbitTemplate).convertAndSend(eq(
        KNOWLEDGE_BASE_UPDATE_QUEUE), any(Object.class));

    // When & Then
    mockMvc.perform(multipart("/webhook/upload-knowledge-file")
        .file(file)
        .param("fileName", "test.txt")
        .param("documentType", "text")
        .accept(MediaType.APPLICATION_JSON))
        .andExpect(status().isOk())
        .andExpect(content().string("File uploaded and processing
            started."));

    verify(rabbitTemplate).convertAndSend(eq(
        KNOWLEDGE_BASE_UPDATE_QUEUE), any(Object.class));
}

@Test
void uploadKnowledgeFile_EmptyFile_ShouldReturnBadRequest()
    throws Exception {
    // Given
    MockMultipartFile file = new MockMultipartFile(
    "file",
    "test.txt",
    MediaType.TEXT_PLAIN_VALUE,
    new byte[0]
    );

    // When & Then
    mockMvc.perform(multipart("/webhook/upload-knowledge-file")
        .file(file)

```



```

        .param("fileName", "test.txt")
        .param("documentType", "text")
        .accept(MediaType.APPLICATION_JSON))
95    .andExpect(status().isBadRequest())
        .andExpect(content().string("Invalid payload"));
    }

    @Test
100    void
        uploadKnowledgeFile_MissingParameters_ShouldReturnBadRequest
            () throws Exception {
        // Given
        MockMultipartFile file = new MockMultipartFile(
            "file",
            "test.txt",
105        MediaType.TEXT_PLAIN_VALUE,
            "test content".getBytes()
        );

        // When & Then
110        mockMvc.perform(multipart("/webhook/upload-knowledge-file")
            .file(file)
            .accept(MediaType.APPLICATION_JSON))
            .andExpect(status().isBadRequest())
            .andExpect(content().string("Invalid payload"));
115    }
}

```

Сгенерированные автотесты инструментов Cursor:

```

package com.aiqa.ragapitestgenerator.controller;

import com.aiqa.ragapitestgenerator.model.QueueEvent;
5  import com.aiqa.ragapitestgenerator.model.WebhookPayload;
import com.aiqa.ragapitestgenerator.model.Repository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.ArgumentCaptor;
10 import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.http.MediaType;
import org.springframework.mock.web.MockMultipartFile;
import org.springframework.http.ResponseEntity;

```

```

15 import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
import static com.aiqa.ragapitestgenerator.queue.
    RabbitMQConfig.KNOWLEDGE_BASE_UPDATE_QUEUE;
import static com.aiqa.ragapitestgenerator.queue.
    RabbitMQConfig.TEST_GENERATION_QUEUE;

20 class WebhookControllerTest {

    private WebhookController webhookController;
    private RabbitTemplate rabbitTemplate;

25 @BeforeEach
void setUp() {
    rabbitTemplate = mock(RabbitTemplate.class);
    webhookController = new WebhookController(rabbitTemplate);
}

30 @Test
void generateTests_InvalidPayload_ReturnsBadRequest() {
    WebhookPayload payload = new WebhookPayload(); // Empty
    payload

35     ResponseEntity<String> response = webhookController.
        generateTests(payload);

    assertEquals(400, response.getStatusCodeValue());
    assertEquals("Invalid payload", response.getBody());
    verify(rabbitTemplate, never()).convertAndSend(anyString(),
40         any());
}

@Test
void uploadKnowledgeFile_EmptyFile_ReturnsBadRequest()
    throws Exception {
    MockMultipartFile file = new MockMultipartFile(
45     "file",
    "test.txt",
    MediaType.TEXT_PLAIN_VALUE,
    new byte[0]
50     );

```

```
55     ResponseEntity<String> response = webhookController.  
        uploadKnowledgeFile(  
            file ,  
            "test.txt",  
            "documentation"  
        );  
  
        assertEquals(400, response.getStatusCodeValue());  
        assertEquals("Invalid payload", response.getBody());  
        verify(rabbitTemplate, never()).convertAndSend(anyString()  
            , any());  
60     }  
    }
```